RQF LEVEL 3

SWDWD301

SOFTWARE DEVELOPMENT

Website Development

TRAINEE'S MANUAL

October, 2024

WEBSITE DEVELOPMENT

KOICA Korea International Cooperation Agency

TQUM TVET Quality Management Project

2024

AUTHOR'S NOTE PAGE (COPYRIGHT)

Copies available from:

- *HQs: Rwanda TVET Board-RTB*
- *Web: www.rtb.gov.rw*
- **KIGALI-RWANDA**

Original published version: October 2024

ACKNOWLEDGEMENTS

The publisher would like to thank the following for their assistance in the elaboration of this training manual:

Rwanda TVET Board (RTB) extends its appreciation to all parties who contributed to the development of the trainer's and trainee's manuals for the TVET Certificate III in Software development, specifically for the module **"SWDWD301: Website Development."**

We extend our gratitude to KOICA Rwanda for its contribution to the development of these training manuals and for its ongoing support of the TVET system in Rwanda
We extend our gratitude to the TQUM Project for its financial and technical support in the development of these training manuals.

We would also like to acknowledge the valuable contributions of all TVET trainers and industry practitioners in the development of this training manual.

The management of Rwanda TVET Board extends its appreciation to both its staff and the staff of the TQUM Project for their efforts in coordinating these activities.

This training manual was developed:

Under Rwanda TVET Board (RTB) guiding policies and directives

Under Financial and Technical support of

COORDINATION TEAM

RWAMASIRABO Aimable

MARIA Bernadette M. Ramos

MUTIJIMA Asher Emmanuel

PRODUCTION TEAM

Authoring and Review

BUKIZI Eric

MUKAMARIDI Marie Rose

Validation

IRADUKUNDA Gilbert

AGIRANEZA Benitha

KWIZERA Emmanuel

Conception, Adaptation and Editorial works

HATEGEKIMANA Olivier

GANZA Jean Francois Regis

HARELIMANA Wilson

NZABIRINDA Aimable

DUKUZIMANA Therese

NIYONKURU Sylvestre

BIZIMANA Eric

Formatting, Graphics, Illustrations, and infographics

YEONWOO Choe

SUA Lim

SAEM Lee

SOYEON Kim

WONYEONG Jeong

MANIRAKORA Alexis

Financial and Technical support

KOICA through TQUM Project

TABLE OF CONTENT

ACRONYMS

CPM: Characters Per Minute

CSS: Cascading style sheet

HTML: Hypertext Markup Language

HTTP: Hypertext Transfer Protocol

RSS: Really Simple Syndication

RTB: Rwanda TVET Board

TQUM Project: TVET Quality Management Project

UAC: User Account Control

URL: Uniform Resource Locator

WPM: Words Per Minute

XML: Extensible Markup Language

INTRODUCTION

This trainee's manual includes all the knowledge and skills required in Software Development specifically for the module of **"Website Development".** Trainees enrolled in this module will engage in practical activities designed to develop and enhance their competencies. The development of this training manual followed the Competency-Based Training and Assessment (CBT/A) approach, offering ample practical opportunities that mirror real-life situations.

The trainee's manual is organized into Learning Outcomes, which is broken down into indicative content that includes both theoretical and practical activities. It provides detailed information on the key competencies required for each learning outcome, along with the objectives to be achieved.

As a trainee, you will start by addressing questions related to the activities, which are designed to foster critical thinking and guide you towards practical applications in the labour market. The manual also provides essential information, including learning hours, required materials, and key tasks to complete throughout the learning process.

All activities included in this training manual are designed to facilitate both individual and group work. After completing the activities, you will conduct a formative assessment, referred to as the end learning outcome assessment. Ensure that you thoroughly review the key readings and the 'Points to Remember' section.

MODULE CODE AND TITLE: SWDWD301-WEBSITE DEVELOPMENT

Learning Outcome 1: Apply keyboard skills
Learning Outcome 2: Create web structures
Learning Outcome 3: Style web elements


BACK SPACE
TAB
Q W E R T Y U I O P
ENTER
CAPS LOCK
A S D F G H J K L
B N
SHIFT
CTRL

Indicative contents
1.1 Use of keyboard characters
1.2 Combination keys and their use
1.3 Application of typing technique

Key Competencies for Learning Outcome 1: Apply keyboard skills

Knowledge	Skills	Attitudes
• Definition of the keyboard • Description of the different types of keyboard • Identification of keyboard parts • Description of the keyboard combination keys • Description of typing techniques	• Using keyboard characters • using the keyboard combination keys • using typing master software • Applying typing tips	• Be self-motivated • Be Detail oriented • Be a good observer

Duration: 20 hrs

Learning outcome 1 objectives:

By the end of the learning outcome, the trainees will be able to:

1. Define clearly the term keyboard as used in typing skills
2. Describe clearly the common keyboard layout based on language.
3. Identify correctly the different parts of keyboard based on their functions
4. Use properly the keyboard characters in accordance with the given task
5. Use properly keyboard combination keys as used in typing
6. Apply properly the typing techniques as used in typing skills
7. Use correctly the typing master software as used in typing skills

Resources

Equipment	Tools	Materials
● Computers ● Keyboard ● Ergonomic Accessories ● Projector/Smartboard	● Text editors ● Word processing ● Typing software (typing master, rapid typing)	● Internet

Indicative content 1.1: Use of keyboard characters

Duration: 7 hrs

Theoretical Activity 1.1.1: Description of keyboard

Tasks:

1. In small groups, you are requested to answer the following questions related to the images below:

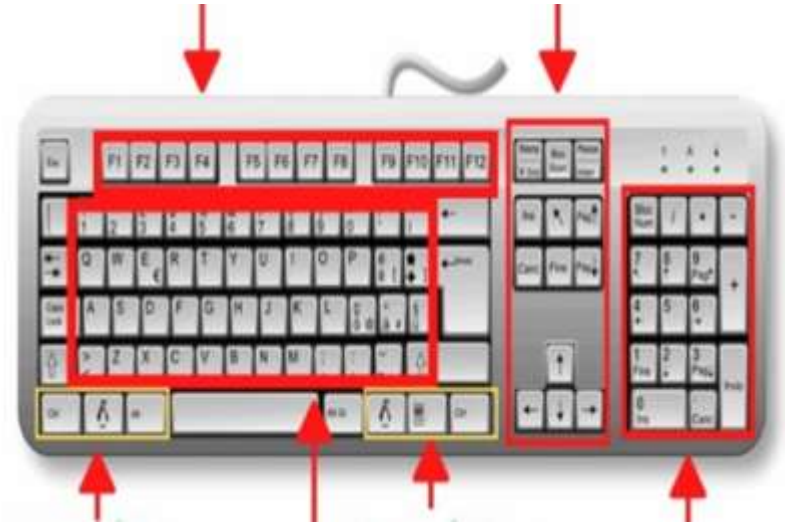


Figure 1: First Image

Figure 2: Second image

Question 1: Name and Define the above device.

Question 2: Based on how keys are arranged, name the different parts found on image.

Question 3: According to the arrangement of the first six characters (letters) on the first row for both images, name the basic layout we can have.

2. Provide the answer for the asked questions and write them on papers/flipchart
3. Present the findings/answers to the whole class

4. For more clarification, read the key **readings 1.1.1.** Ask questions where necessary.

Key readings 1.1.1:

•A computer keyboard is an input device used to enter characters and functions into the computer system by pressing buttons, or keys.

It is the primary device used to enter text. A keyboard typically contains keys for individual letters, numbers and special characters, as well as keys for specific functions.

Keyboard Layout

A keyboard layout is any specific physical, visual, or functional arrangement of the keys, legends, or key-meaning associations (respectively) of a computer keyboard, mobile phone, or other computer-controlled typographic keyboard.

A keyboard layout is the order of keys on your keyboard but it does not mean that it is tied to the keyboard's factory settings or looks. You can change your layout without changing the order of physical keys. That way you can use different layouts with the same keyboard (once you learn them)

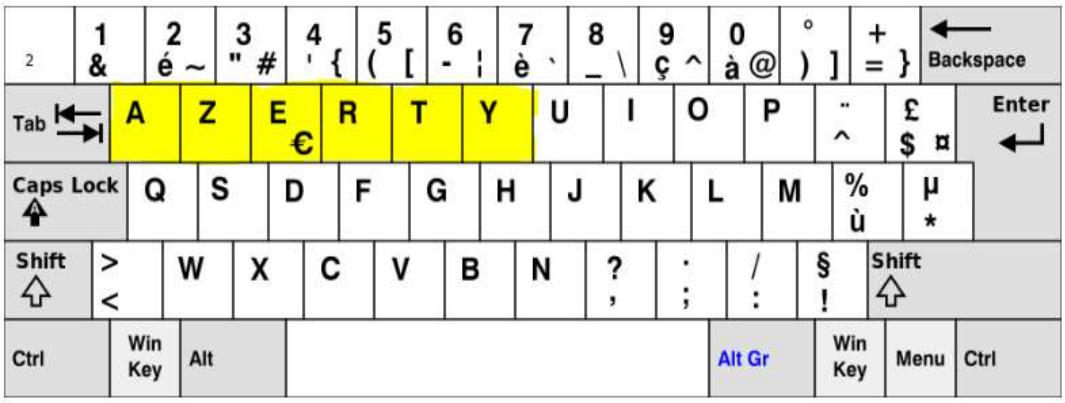
There are different keyboard layouts

Because people are not the same. Even the most basic example – there are many countries in the world. All of those countries have their own keyboard layout. Even if it is just a small change in the standard QWERTY – that is what they need.

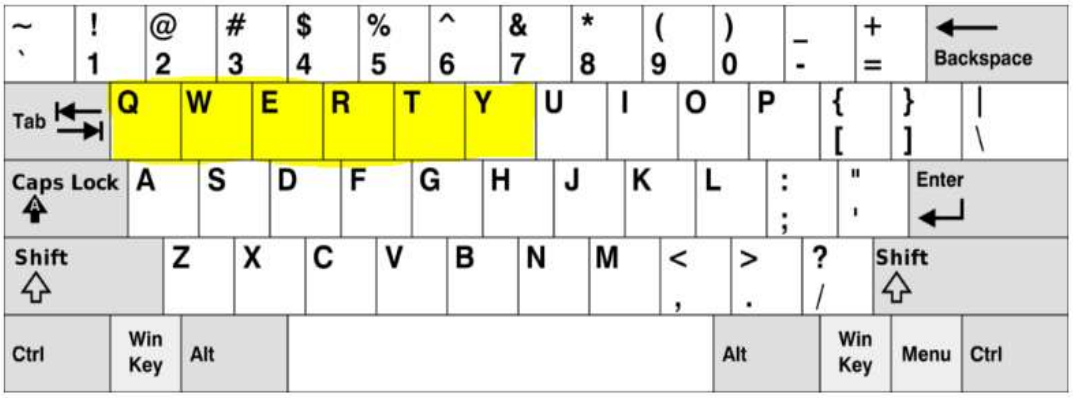
For example, there is a layout called AZERTY. It is a modified QWERTY for the French language. That is simply better for them to type.

AZERTY is a very useful layout for some countries. You could even say that AZERTY is required. It is used to type in French.

AZERTY is an official layout of France but it's used not only in France. People all over the world use this layout to type in the French language.



QWERTY is the first "keyboard" layout that was ever created. It is used for keyboards now but was not created for them. QWERTY was designed for typewriting machines. QWERTY is a layout that was created for typewriting machines. It is the combination of keys in a specific order (as the image shows). QWERTY is an old layout. It was created in the 1870s by Christopher Latham Sholes. It was the first layout ever created. Even though it is an old layout, QWERTY is still used by most people.



Difference between AZERTY Layout and QWERTY Layout

The first difference that we can see is a different name. Both of those layouts end with "ERTY" but they have different first and second letters.

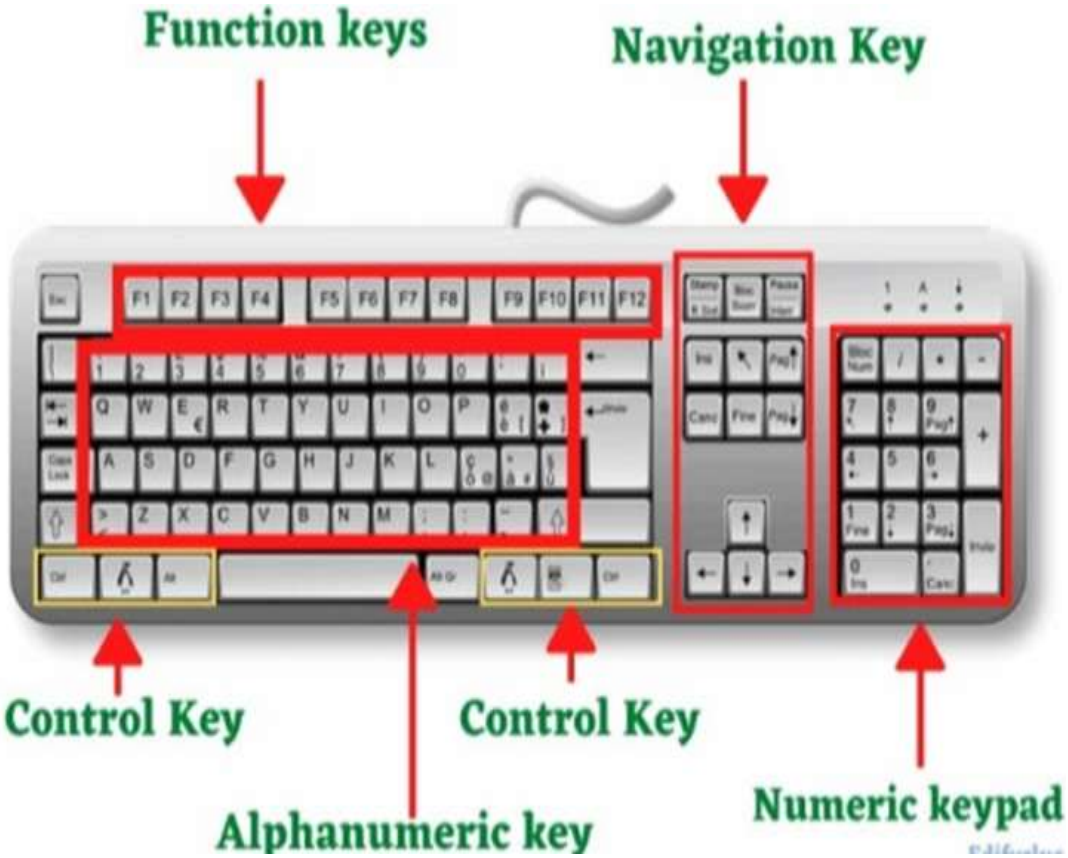
The difference in the name comes from the actual difference in the layout. The first two keys of the upper row are "Q" and "W" for QWERTY, and "A" and "Z" for AZERTY. Why is that so?

It's because it makes typing in French easier. Letter "A" is the second most frequently used letter in the French language. That's why it's moved up to the first row. To make it easier accessible.

Parts of keyboard

The keys on your keyboard can be divided into several groups based on function:

- **Typing (alphanumeric) keys.** These keys include the same letter, number, punctuation, and symbol keys found on a traditional typewriter
- **Control keys (Special Function).** These keys are used alone or in combination with other keys to perform certain actions. The most frequently used control keys are Ctrl, Alt, the Windows logo key picture of the Windows logo key, and Esc.
- **Function keys.** The function keys are used to perform specific tasks. They are labelled as F1, F2, F3, and so on, up to F12. The functionality of these keys differs from program to program.
- **Navigation keys.** These keys are used for moving around in documents or webpages and editing text. They include the arrow keys, Home, End, Page Up, Page Down, Delete, and Insert.
- **Numeric keypad.** The numeric keypad is handy for entering numbers quickly. The keys are grouped together in a block like a conventional calculator or adding machine



Useful keys on Keyboard for typing text

Whenever you need to type something in a program, e-mail message, or text box, you'll see a blinking vertical line (Picture of the cursor) called the cursor or insertion point. It shows where the text that you type will begin. You can move the cursor by clicking in the desired location with the mouse, or by using the navigation keys.

When typing text, we use alphanumeric key. In addition to letters, numerals, punctuation marks, and symbols, the typing keys also include Shift, Caps Lock, Tab, Enter, the Spacebar, and Backspace for performing specific task while typing.
Let us see the use of those keys

Key Name	How to use it	How it looks like
Shift	Press Shift in combination with a letter to type an uppercase letter. Press Shift in combination with another key to type the symbol shown on the upper part of that key	
Caps Lock	Press Caps Lock once to type all letters as uppercase. Press Caps Lock again to turn this function off. Your keyboard might have a light indicating whether Caps Lock is on.	

Tab	Press Tab to move the cursor several spaces forward. You can also press Tab to move to the next text box on a form.	
Enter	Press Enter to move the cursor to the beginning of the next line. In a dialog box, press Enter to select the highlighted button.	
Spacebar	Press the Spacebar to move the cursor one space forward.	
Backspace	Press Backspace to delete the character before the cursor, or the selected text.	

Insert	Turn Insert mode off or on. When Insert mode is on, text that you type is inserted at the cursor. When Insert mode is off, text that you type replaces existing characters.	
Esc	Cancel the current task	
Home keys	It is used to return you to the main page or Homepage of whatever application or program you are using.	
Arrow keys	They are used to move the insertion point in the direction indicated by	

	the arrow on each key.	

Practical Activity 1.1.2: Use of keyboard characters

Task:

1: Referring to the previous theoretical activities (1.1.1) you are requested to go to the computer lab to use keyboard characters. This task should be done individually.

Information and Communications Technology in education.

Schools use a diverse set of ICT tools to communicate, create, disseminate, store, and manage information. In some contexts, ICT has also become integral to the teaching learning interaction, through such approaches as replacing chalkboards with interactive digital whiteboards, using students own smart phones or other devices for learning during class time, and the flipped classroom model where trainees watch trainers at home on the computer and use classroom time for more interactive exercises.

2: Read key reading 1.1.2 and ask clarification where necessary
3: Apply safety precautions as recommended in Laboratory rules

4: Referring to the task given use all keys to do the task given
5: Present your work to the trainer and whole class

Key readings 1.1.2

To effectively use keyboard characters, follow these procedures:

1. Familiarise Yourself with Key Layout:

- Understand the layout of the keyboard, including the positions of letters, numbers, symbols, and special keys.

2.Correct Finger Placement:

- Position your fingers on the home row keys (**ASDF** for the left hand and **JKL;** for the right hand) for touch-typing.

3. Pressing Keys:

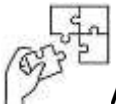
- Press keys with the appropriate fingers to type characters accurately. Use the correct finger for each key based on touch typing techniques.

4. Press Caps Lock for Uppercase or Lowercase:

- Press and hold the Shift key while typing a letter to capitalize it. Use the Shift key in combination with other keys to type symbols above numbers.

Points to Remember

- Keys on Keyboard are grouped into different groups based on its function such as: alphanumeric key, numeric keypad, function key and Navigation key
- Keyboards have different layouts such as QWERTY and AZERTY.
- There are several procedures to take under consideration in order to effectively use keyboard characters like being familiar with keyboard layout, correct finger placement on keyboard and Press key with appropriate finger.



Application of learning 1.1.

Project manager, needs to send an email to her team to provide updates on a project and schedule a meeting. You are asked to write the following Email for the team.

Subject: Project Update and Team Meeting

Hi Team,

I hope this email finds you well. I wanted to provide a quick update on the current status of our project and schedule a team meeting to discuss upcoming milestones.

Project Update:

- We have successfully completed Phase 1 of the project.

- The development team is working on implementing new features based on client feedback.

- QA testing is scheduled for next week.

Upcoming Team Meeting:

I propose that we have a team meeting on [Day], [Date] at [Time] to discuss the following:

1. Review the completed Phase 1 and address any questions.

2. Plan for the upcoming QA testing phase.

3. Discuss the timeline for the implementation of new features.

Please confirm your availability for the proposed meeting time. If you have any specific agenda items you'd like to add, please let me know by [Deadline].

Looking forward to a productive discussion.

Best regards,

Indicative content 1.2: Combination keys and their use

Duration: 5 hrs

Theoretical Activity 1.2.1: Description of keyboard Combination keys

Tasks:

1.In small groups, you are asked to answer the following questions related to the combination keys on the keyboard:

a. What do you understand about the combinations of keys?
b. What are primarily keys used to perform combinations or shortcut?

2.Provide the answer for the asked questions and write them on papers.
3.Present the findings/answers to the whole class
4. For more clarification, read the key readings 1.2.1. and ask questions where necessary.

Key readings 1.2.1:

A key combination, also known as a keyboard shortcut or hotkey, is a sequence of two or more keys pressed together on a computer keyboard to perform a specific function or command.

This allows users to execute tasks quickly and efficiently without navigating through menus or using the mouse. Key combinations are widely used in technology, computing, programming, and communications to enhance productivity and streamline operations.

Working principles of key combinations:

When you press a key or a combination of keys on your keyboard, the operating system or software recognizes the input and triggers a predefined action associated with that key combination. The specific actions performed by key combinations vary depending on the operating system, software application, or context in which they are used. Key combinations can be assigned by default or customized by the user to suit their preferences or requirements.

Key combinations against operating systems

Key combinations are not universal across all operating systems. While some key combinations have standardized functionality across different platforms, there are variations between operating systems, such as Windows and Linux. Additionally, individual software applications may also have their own unique key combinations that are specific to their features and functions. It is important to familiarize

yourself with the key combinations relevant to the operating system and software you are using.

Some popular key combinations or short cut.

In programming, certain key combinations are widely used by developers to expedite their coding tasks. Here are a few popular key combinations in programming:

Ctrl + S: Save changes made to the current file.

Ctrl + F: Find and search for a specific piece of text within the code.

Ctrl + D: Duplicate the current line or selection.

Ctrl + /: Comment or uncomment selected lines of code.

Ctrl + Space: Trigger code autocompletion.

Ctrl + Shift + L: Select all occurrences of the selected text.

These key combinations can vary depending on the programming language, integrated development environments (IDE), or code editor you are using. It is essential to familiarize yourself with the specific key combinations relevant to your programming environment.

Primarily Keyboard keys used to perform combinations

Many shortcuts use **Ctrl, Alt, Shift, Windows and function** keys alone or in combination for quick access to commands from the keyboard.

Use of combinations on mobile devices

Combinations Keys, often referred to as gestures, can be used on mobile devices with touchscreens. These gestures involve specific actions performed by tapping or swiping multiple fingers simultaneously on the screen. For example, on some smartphones, you can use a key combination of pressing the power button and volume down button together to take a screenshot.

Use of key combinations to navigate a document or text editor

Text editors and document processing software often support key combinations for navigation. For example, in Microsoft Word, you can use "Ctrl + Left Arrow" to move the cursor to the beginning of the previous word and "Ctrl + Right Arrow" to move the cursor to the beginning of the next word.

Switch tabs in web browsers using key combinations

Web browsers offer key combinations for tab navigation. For example, in Google Chrome, you can use "Ctrl + Tab" to switch to the next tab and "Ctrl + Shift + Tab" to switch to the previous tab.

Navigate through a long document or web page using Key combinations

In many applications and web browsers, you can use key combinations like "Ctrl + Home" to go to the beginning of a document or "Ctrl + End" to go to the end. Additionally, "Ctrl + Page Up" and "Ctrl + Page Down" allow you to scroll through the document or webpage.

Opening a new private browsing window in web browsers.

In most web browsers, you can use the key combination “Ctrl + Shift + N” on Windows and “Command + Shift + N” to open a new private browsing window.

Keyboard shortcuts are essential for improving efficiency across many applications, from web browsers to productivity software and design tools.

Here are some of the most popular and widely used shortcuts that can be applied across various application

Sample shortcuts

Windows:

Copy: Ctrl + C
Cut: Ctrl + X
Paste: Ctrl + V
Undo: Ctrl + Z
Redo: Ctrl + Y
Select All: Ctrl + A
Find: Ctrl + F
Save: Ctrl + S
Open: Ctrl + O
Print: Ctrl + P
New Window: Ctrl + N
Close Window: Alt + F4
Switch Applications: Alt + Tab
Minimize Window: Win + Down Arrow
Maximize Window: Win + Up Arrow

Microsoft Office (Word, Excel, PowerPoint):

1. **Bold:**
   Ctrl + B
2. **Italic:**
   Ctrl + I
3. **Underline:**
   Ctrl + U
4. **Save As:**
   F12 or Ctrl + Shift + S

5. **Print:**
   Ctrl + P
6. **Open File:**
   Ctrl + O
7. **Create New Document:**
   Windows: Ctrl + N

Practical Activity 1.2.2: Apply combination keys

Task:

1. Referring to previous theoretical activities (1.2.1.) you are asked to go to the computer Lab and perform key combinations. This task should be done individually.
2. Read key reading 1.2.2 and ask clarification where necessary
3. Apply safety precautions
4. Present out the selected keys to use for modify sample text and the combinations you are going to perform.
5. Referring to the selection of keys and combinations provided on step 3, perform the keys combinations by holding modifiers and action keys.
6. Present your work to the trainer and whole class

Key readings 1.2.2

To apply combination keys effectively, follow these steps:

1. Identify the Task:

- Determine the specific action or command you want to perform using a combination key shortcut.

2. Locate the Correct Keys:

- Identify the correct combination of keys needed to execute the desired shortcut (e.g., Ctrl, Alt, Shift, or a function key).

3.Press and Hold Modifier Key(s):

- Press and hold the modifier key(s) (Ctrl, Alt, Shift) while keeping them held down throughout the combination.

4.Press the Action Key:

- While holding down the modifier key(s), press the corresponding action key to trigger the desired function or command.

5. Release All Keys:

- Release all keys simultaneously after pressing the action key to complete the

shortcut.

6. Practice and Repeat:

- Practice applying the combination keys repeatedly to become comfortable and proficient with the shortcut.

7. Verify the Result:

- Ensure that the desired action or command was executed correctly after applying the combination keys.

8. Use in Context:

- Apply the combination keys within the appropriate context of the software application or task you are working on.

By following these steps and practicing the application of combination keys regularly, you can streamline your workflow, perform tasks more efficiently, and navigate software applications with ease using keyboard shortcuts.

Steps to apply Ctrl, shift, alt, Fn keys:

Points to Remember

- The Steps to use combination keys
  - Step1: Identify the desired combination key
  - Step2: Locate the keys on your keyboard
  - Step3: Press and hold the modifier key(s)
  - Step4: While holding the modifier key(s), press the main key
  - Step5: Continue holding the modifier key(s) and press the additional keys,
  - Step6: Release all keys
- Key combinations vary according to the system (operating system) or application you are using

Application of learning 1.2.

As IT Support you are request to help project Manager to Modify and print the document from his computer while it is experiencing touchpad problems (Cursor is not moving), You are request to duplicate and Modify whole the text to become Capitalized and print it by using keyboard only.

Indicative content 1.3: Application of typing technique

Duration: 8hrs

Theoretical Activity 1.3.1: Description of typing techniques

Tasks:

Step1. In small groups, you are requested to answer the following questions related to the typing techniques:

a) Based on the image provided above, what do you think the images are expressing?
b) What do you understand by the following terms?
   i. Typing
   ii. Typing technics
   iii. Touch typing
   iv. Ergonomics
c) What are the typing tools or software that can help you to be familiar with touch typing?

Step 2. Provide the answer for the asked questions and write them on papers/Flipchart
Step 3. Present your findings to the classmate and Trainer
Step 4. For more clarification, read the key readings 1.3.1. Ask questions where necessary.

Key readings:1.3.1

Typing is the process of writing or inputting text by pressing keys on a typewriter, computer keyboard, mobile phone, or calculator. It can be distinguished from other means of text input, such as handwriting and speech recognition.

Typing techniques refer to the methods and strategies used to improve the efficiency, accuracy, and speed of typing. These techniques encompass various aspects, including the correct positioning of hands and fingers, proper posture, and the use of specific typing practices.

Touch typing Touch type refers to a method of typing on a computer keyboard without having to look at the keys. It involves using all ten fingers to type quickly and accurately, increasing productivity and reducing eye strain.

- The application of typing techniques involves implementing proper methods and strategies in various contexts to improve typing efficiency, accuracy, and speed. These techniques are utilized not only for general typing tasks but also for specific professional, academic, and personal activities.

There are several typing tips to follow while performing keyboarding (Typing)

1. Hand positioning
2. Body posture
3. Ergonomics
4. Typing speed

a) Hand positioning

Proper hand positioning and technique are fundamental for developing typing speed and accuracy, as well as preventing discomfort or injury over time

b) Body posture

Good posture is a position in which the body is aligned in a way that puts the least amount of strain on the muscles, joints, and ligaments. It involves maintaining a balance between different parts of the body, so that the spine, shoulders, and hips are properly aligned.

Good posture and how it looks like

1. The head is balanced directly over the shoulders, without leaning forward or backward.
2. The shoulders are relaxed and level, not hunched or rounded.
3. The spine is aligned in its natural curves, with the neck slightly curved inward, the upper back slightly curved outward, and the lower back slightly curved inward.
4. The pelvis is in a neutral position, with the hips level and the tailbone pointing downward.
5. The knees are slightly bent and in line with the feet.
6. The feet are flat on the ground, with the weight distributed evenly on both feet.

Maintaining good posture throughout the day can help reduce the risk of developing a range of health problems, including back and neck pain, headaches, and fatigue. It can also improve breathing, digestion, and circulation, as well as increase energy levels and self-confidence.

c) Ergonomics

Ergonomics is the process of designing or arranging workplaces, products and systems so that they fit the people who use them.

Ergonomic Tips:

1. Adjust Your Chair

Ensure that your chair supports the natural curve of your spine. Adjust the height so your feet are flat on the floor and your knees are at a right angle

2. Optimize Monitor Position

Place the monitor at eye level, about an arm's length away, to reduce neck and eye strain.

3. Keyboard and Mouse Placement:

Keep the keyboard and mouse at the same height, within easy reach, to avoid stretching and strain

4. Take Regular Breaks:

Take short breaks every 20-30 minutes to stand, stretch, and relieve muscle tension.

5. Use Proper Lighting

Ensure adequate lighting to reduce glare and eye strain. Use task lighting for close-up work.

Ergonomics is essential for creating environments that enhance human performance, comfort, and safety. By applying ergonomic principles, individuals and organizations can improve productivity, reduce the risk of injury, and create more satisfying work and living environments. Regular assessment and adaptation of ergonomic practices are crucial for maintaining optimal health and efficiency in various settings.

d) Typing speed

Typing speed is a measure of how quickly a person can type. It is typically quantified in terms of words per minute (WPM) or characters per minute (CPM). These metrics help evaluate a person's typing efficiency and proficiency.

Key Components of Typing Speed

1. Words Per Minute (WPM):

Definition: WPM is the number of words a person can type in one minute. For standardization, a "word" is often defined as five characters or keystrokes, including spaces and punctuation.

Calculation: It is calculated by dividing the total number of characters typed by 5 and then dividing by the number of minutes taken to type.

2. Characters Per Minute (CPM):

Definition: CPM measures the number of characters a person can type in one minute**.**

Calculation: It is calculated by dividing the total number of characters typed by the number of minutes taken to type.

3. Accuracy

Importance: While speed is important, accuracy ensures that the typed content is correct. High typing speed with low accuracy can result in more time spent on corrections.

Measurement: Accuracy is often measured as a percentage, calculated by dividing the number of correct keystrokes by the total number of keystrokes made and then multiplying by 100.

Factors Influencing Typing Speed

1. Familiarity with the Keyboard

Touch Typing: Mastery of touch typing, where the typist relies on muscle memory rather than looking at the keys, significantly increases speed.

Keyboard Layout: Familiarity with different keyboard layouts (e.g., QWERTY, AZERTY) can impact typing speed.

2. Posture and Ergonomics

Seating and Position: Proper posture and ergonomically designed workstations can reduce fatigue and increase typing efficiency.

Hand and Finger Placement: Correct positioning of hands and fingers on the keyboard facilitates faster typing.

3. Practice and Experience

Regular Practice: Consistent practice improves typing speed as muscle memory develops.

Typing Exercises: Engaging in specific typing drills and exercises can enhance speed and accuracy.

4. Typing Tools and Software:

Typing Tutors: Programs designed to teach touch typing and improve speed through structured lessons and exercises**.**

Example: typing Master

Typing Tests: Online typing tests help assess and improve typing speed and accuracy

- **Application and Use of typing master**

Typing Master is a comprehensive software program designed to help users improve their typing skills, including speed, accuracy, and overall efficiency. It provides a structured approach to learning touch typing through interactive lessons, drills, games, and tests.

a) Checking total number of words written per minute

To check the total number of words written per minute (WPM) in Typing Master, you typically need to complete a typing test or exercise within the software

b) Typing Games

Typing games are interactive and engaging tools designed to help improve typing speed and accuracy through fun and challenging activities. Here's a detailed description of some popular typing games and how they contribute to enhancing typing skills:

1. NitroType:

Description: Nitro Type is a competitive typing game where players race cars by typing sentences quickly and accurately. The faster and more accurately you type, the faster your car goes.

How It Helps: It encourages rapid typing under pressure, improving both speed and accuracy while making the experience competitive and fun.

2. TypeRacer:

Description: TypeRacer is an online multiplayer game where you race against others by typing passages from books, movies, and songs. Each race provides a different text, making it diverse and challenging.

How It Helps: It promotes speed and accuracy through competitive typing, allowing users to see how they rank against others in real-time.

3. TypingClub:

Description: Typing Club offers a range of typing lessons and games that gradually increase in difficulty. It includes interactive lessons, games, and challenges that help improve typing skills.

How It Helps: The structured lessons and fun games help users build muscle memory and typing speed incrementally.

4. ZType:

Description: ZType is a space shooting game where players destroy enemy ships by typing words associated with the ships. The game becomes progressively harder, requiring faster typing to keep up.

How It Helps: It improves reflexes and speed by combining fast typing with immediate visual feedback in a game setting.

5. Keybr:

Description: Keybr is an online tool that offers interactive exercises and games to improve typing speed and accuracy. It focuses on touch typing and provides personalized exercises based on performance.

How It Helps: By targeting specific weaknesses and gradually increasing difficulty, Keybr helps users build efficient typing habits.

6. 10FastFingers:

Description: 10FastFingers provides various typing tests and competitive typing games that challenge users to type as many words as possible within a time limit.

How It Helps: The timed nature of the tests helps users practice typing quickly and accurately under time constraints

Practical Activity 1.3.2: Use typing master

Tasks:

1: Referring to the previous theoretical activities (1.3.1) you are requested to go to the computer lab to use typing software (Typing Mater). This task should be done individually.

2. Read key reading 1.3.2 and ask clarification where necessary

3. Apply safety precautions

4. Present out the steps to follow when using typing master.

5. Referring to the steps provided on step 3, type the passage

6. Present your work to the trainer and whole class

Key readings:1.3.2

To apply proper hand position on a computer keyboard, you can follow these steps:

1. Sit in a comfortable and upright position: Sit with your back straight, shoulders relaxed, and feet flat on the floor. Maintain a relaxed yet alert posture.

2. Place your hands on the keyboard: Position your hands on the keyboard with your fingers curved naturally. Rest your palms lightly on the palm rest or desk, without putting too much pressure.

3. Position your fingers on the home row: The home row is the middle row of the keyboard (ASDF for the left hand and JKL; for the right hand). Rest your left-hand fingers on the ASDF keys and your right-hand fingers on the JKL; keys.

5. **Keep your fingers in the proper position:** Each finger should rest on a specific key. Your left-hand fingers should be on the keys **ASDF**, and your right-hand fingers should be on the keys **JKL;** Your thumbs should rest on the space bar

6. **Use the correct finger for each key:** Use the appropriate finger for each key, following touch typing techniques. For example, use your left pinky finger for the A key, left ring finger for the S key, left middle finger for the D key, and left index finger for the F key. Repeat this pattern for the right-hand keys.
7. **Maintain a light touch:** Type with a light touch, pressing the keys gently. Avoid pounding on the keys, as it can lead to fatigue and strain.

8. **Keep your wrists straight:** Ensure that your wrists are in a neutral, straight position. Avoid bending or angling your wrists excessively, as it can cause discomfort or repetitive strain injuries.

8. Practice proper finger movement: Use your fingers to press the keys, rather than relying on excessive hand or wrist movements. This will help improve speed and accuracy.
Note: By following these steps, you can develop proper hand position on the computer keyboard, which will improve your typing speed, accuracy, and overall efficiency.

Use of Typing Master:

Initial Configuration

Launch Typing Master:
After installation, open Typing Master by clicking on the desktop icon or searching for it in the Start menu.

Create a User Profile:

When you first launch Typing Master, you'll be prompted to create a user profile.
Enter your name and choose your preferred language. This profile will store your progress and personalized training data.

Set Skill Level:

Typing Master may ask you to set your current typing skill level (e.g., beginner, intermediate, advanced). This helps the software tailor lessons to your needs.

Select Keyboard Layout:

Choose the keyboard layout you're using (e.g., QWERTY, AZERTY). This ensures that the lessons match your keyboard's layout.

Start with a Typing Test:

Begin by taking a typing test to assess your current speed and accuracy. This will help Typing Master customize your learning path.

Follow the Lessons:

Typing Master will guide you through a series of lessons that focus on various aspects of touch typing, including:
Home row positioning
Typing individual letters, words, and sentences
Special characters and numbers

Practice Regularly:

Engage in daily practice sessions. Typing Master tracks your progress and adjusts the difficulty level based on your performance.

Use Typing Games:

Explore the typing games available within the software. These games make practice fun and help reinforce the skills you're learning.

Monitor Your Progress:

Check your progress reports regularly to see improvements in speed, accuracy, and overall typing efficiency.

Points to Remember

- To use typing master, you are requested first of all to perform initial configuration such as: Create a user Profile, Set Skill Level, Select Keyboard Layout, Start with a Typing Test…

Application of learning 1.3

Jane is an administrative assistant at a busy marketing firm. Her daily tasks include typing emails, creating reports, and managing schedules. Despite her experience, Jane often finds herself spending too much time on typing-related tasks due to her relatively slow typing speed and frequent errors. The firm recently decided to implement a productivity improvement program to help employees work more efficiently. Jane's manager introduces Typing Master as part of the productivity program. Jane takes an initial typing test on Typing Master to assess her current typing speed and accuracy. The results show that her typing speed is 40 WPM with an accuracy of 85%.

Jane dedicates 30 minutes each day to practice typing using Typing Master. She follows the structured lessons, which gradually increase in difficulty, and plays typing games to make the practice sessions enjoyable. Jane uses the progress tracking feature to monitor her improvements. After a few weeks, she notices her typing speed has increased to 55 WPM, and her accuracy has improved to 95%.

With the help of Typing Master, Jane has significantly improved her typing speed and accuracy. This has not only enhanced her productivity but also reduced her stress levels and increased her job satisfaction. Her manager is pleased with the positive impact on her performance, and the firm considers expanding the Typing Master program to other employees. So as trainee try to increase your typing speed and accuracy up to 45 WPM and accuracy of at least 80%.

Learning outcome 1 end assessment

Theoretical assessment

Question1: What is a keyboard?

A) A device used to display images
B) A device used to input text and commands into a computer
C) A device used to output sound
D) A device used to store data

Question 2: What is the most common keyboard layout in English-speaking countries?

A) AZERTY
B) QWERTY
C) DVORAK
D) QWERTZ

Question 3: Which key is typically found directly below the 'Q' key in a QWERTY layout?

A) A
B) W
C) Z
D) S

Question 4: Which keyboard layout is primarily used in France?

A) QWERTY
B) QWERTZ
C) AZERTY
D) DVORAK

Question 5: The primary home row keys for the left hand on a QWERTY keyboard are: A, S, D, and _____.

Question 6: The key used to delete the character to the left of the cursor is called the ___ key

Question 7: Typing speed is typically measured in _____ per minute (WPM).

Question 8: Ergonomics in typing involves developing _____ and _____ as well as to prevent strain and injury.

Question 9: To create a capital letter, you hold down the _____ key while pressing the letter key.

Question 10: The function keys, such as F1, F2, etc., are located at the _____ of the keyboard.

Question 11: Match the typing game to its description

Answer	Typing Game	Description
1……	1. Nitro Type	**A.** Provides typing exercises with real-time error detection and correction**.**
2……	2. Type Racer	**B.** A space shooting game where you destroy enemy ships by typing words
3……	3. TypingClub	**C.** Offers structured lessons and interactive games to improve typing skills
4……	4. ZType	**D.** An online multiplayer game where players race by typing passages from books, movies, and songs
5…….	5. Keybr	**E.** A competitive racing game where your typing speed controls the speed of your car

Practical assessment

XYD-Ltd is an agricultural organisation located in Musanze district. They have a problem of writing a company profile because it has many different and complicated characters.

They want someone to write for them a company profile of 9600 words to submit to the RDB for getting an agricultural licence. As a Level 3 student in software development and acquired keyboard skills, you are requested to do this task by respecting the body posture, ergonomics and hand positioning).

References

Mobility Lab - Ultra-Slim Wired PC Keyboard Space Grey - French AZERTY USB connection : Amazon.com.be: Electronics
QWERTY, QWERTZ, and AZERTY - All you need to know about them - TypingDoneWell.com
What is a Computer Keyboard? - Parts, Layout & Functions - Lesson | Study.com
What is a Key Combination? How to Use Them on Mobile Devices | Lenovo US
What is a keyboard layout? (with 90+ list of them) - TypingDoneWell.com


Notepad++
<!DOCTYPE html>
<html lang="ko">
<head>
<meta charset="UTF-8" />
<meta http-equiv="X-UA-Compatible" content="IE=edge" />
<meta name="viewport" content="width=device-width, initial-scale=1.0" />
<link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-awe-
some/6.0.0-beta2/css/all.min.css" integrity="sha512-YWzhKL2whUzgiheMoBFwW8CKV4qpHQAEuvilg9-
FAn5VJUDwKZZxkJNuGM4XkWuk94WCrrwslk8yWNGmY1EduTA==" crossorigin="anonymous" referrerpoli-
cy="no-referrer" />
<link rel="apple-touch-icon" sizes="180x180" href="favicon.ico" />
<link rel="manifest" href="images/favicon.ico/manifest.json" />
<meta name="msapplication-TileColor" content="#ffffff" />
<meta name="msapplication-TileImage" content="images/favico/ms-icon-144x144.png" />
<meta name="theme-color" content="#ffffff" />
Content here, content here
What is Lorem Ipsum?
What is Lorem Ipsum?
containing Lorem Ipsum passages, and more recently
with desktop publishing software like Aldus
containing Lorem Ipsum passages, and more recently with desktop publishing software like Aldus

Indicative contents
2.1. Setting up of text editor and web browser
2.2. Creating a web page in HTML
2.3. Managing page layout in HTML
2.4. Optimising web page in HTML

Key Competencies for Learning Outcome 2: Create web structures

Knowledge	Skills	Attitudes
● Description of website ● Definition of text editor ● Definition of web browser ● Introduction to HTML ● Description of HTML tags ● Description of web page layout	● Installing text editor ● Installing of web browser ● Developing page structure ● Developing a webpage using HTML tag categories ● Using hyperlink ● Using of HTML graphics ● Using HTML tags to organise page layout ● Using HTML tags to optimize webpage	● Be creative ● Be detail-oriented ● Be Problem solver ● Be Collaborative and having teamwork ability ● Have Attention to detail ● Have Passion for technology ● Have Critical thinking

Duration:70 hrs

Learning outcome 2 objectives:

By the end of the learning outcome, the trainees will be able to:

1. Define correctly the term Webpage, Website, Web browser, Text editor, URL and Hyperlink as used in website development
2. Set up correctly the text editor and web browser according to the system requirements
3. Describe correctly html tags as used in web development
4. Create properly a web page using HTML tags in line with web structures
5. Manage properly page layouts in HTML according to the web structures to be developed
6. Link correctly the different webpages by using hyperlinks as used in website development
7. Optimize accurately web page in HTML based on target devices

Resources

Equipment	Tools	Materials
● Computer ● Projector/smartboard	● Text editors ● web browsers	● Internet

Indicative content 2.1: Setting up of text editor and web browser

Duration: 10 hrs

Theoretical Activity 2.1.1: Description of website concepts

Tasks:

1: In small groups, you are asked to answer the following questions related to the website concepts

i. What do you understand about the following terms used in web structure: ?
   a. Website
   b. Webpage
   c. Web browser
   d. Text editor
   e. Hyperlink
   f. URL

ii. What are some Examples of text editor?

3: Provide the answer for the asked questions and write them on papers.

4: Present the findings/answers to the whole class

5: For more clarification, read the key readings 2.1.1. In addition, ask questions where necessary.

Key readings 2.1.1.:

A website is a related collection of web pages that are publicly accessible and share a single domain name.

A website is a collection of interlinked web pages, typically identified by a common domain name, and published on at least one web server.

Websites are accessible via the internet and can contain a variety of content including text, images, videos, and interactive elements.

A domain name is a string of text that maps to an alphanumeric IP address, used to access a website from client software. In plain English, a domain name is the text that a user types into a browser window to reach a particular website.

Example of a website domain name:

- www.igihe.com

A web page is a document written in hypertext (also known as HTML) that you can see online, using a web browser. Most web pages include text, photos or videos, and links to other web pages.

A webpage is a document on the World Wide Web that is displayed in a web browser. It is a single, usually HTML-formatted page within a website, which can contain various types of content such as text, images, videos, and interactive elements.

A browser is an application program that provides a way to look at and interact with all the information on the World Wide Web.

This includes Web pages, videos and images. The word "browser" originated prior to the Web as a generic term for user interfaces that let you browse (navigate through and read) text files online

A web browser is a software application that allows you to access and navigate web pages on the internet. It displays websites on your computer screen and helps you interact with them by clicking on links or entering text.
The most popular types/examples of web browsers are:

- Google Chrome
- Apple Safari
- Mozilla Firefox
- Microsoft Edge.
- Brave
- Uc Browser
- Opera mini

Web browsers functionality

- A web browser works by retrieving resources from a web server and displaying them on your computer screen. These resources, mostly web pages, are identified by URLs and include text, images, videos, and other content.
- Web browsers use the Hypertext Transfer Protocol (HTTP) to request these web pages and display them to you.
- Web browsers function by translating Hypertext Markup language (HTML) and Extensible Markup Language (XML) code into a viewable web page. The browser fetches this code from a web server, interprets it, and creates a visible web page on the screen of your device.

- Supporting extensions and plugins: Browsers often allow the installation of extensions and plugins that enhance functionality, such as ad-blockers, password managers, or developer tools.
- Handling multimedia: Browsers can play audio and video files directly within the browser window, without the need for external players
- Managing bookmarks: Users can save and organize their favourite websites as bookmarks for quick access later

Difference between a browser and a search engine

Browsers and search engines serve different purposes. Web browsers (Google Chrome, Apple Safari, Microsoft Internet Explorer) allow you to view, locate, and access websites, while search engines (Google, Bing, Yahoo) are particular websites that provide you with search results that you can access via a web browser.

A text editor is a tool that allows a user to create and revise documents in a computer.

An HTML editor is a software application that creates and modifies HTML code. HTML is the standard Markup language for creating web pages and applications. These editors are essential tools for web developers and designers as they provide a convenient way to build and edit web page layouts and content.

Examples of HTML text editor:

- Dreamweaver
- Sublime text
- Brackets
- Komodo edit
- Atom
- Coffee cup
- Notepad++
- Visual Studio Code

Hyperlink: In website development, a hyperlink (often simply called a "link") is an element that links one web page to another. Hyperlinks are fundamental components of the web, allowing users to navigate between different pages and resources on the internet.

Types of Hyperlinks:

Text Links: Hyperlinked text, usually indicated by a different colour or an underline, signifies that clicking on it will lead to another location.

Image Links: Images can also serve as hyperlinks. Clicking on the image redirects the user to the linked destination.

Internal and External Links:

Internal Links: These links direct users to other sections or pages within the same document or website.

External Links: External links point to resources on different websites or servers.

Anchor Text: The clickable text of a hyperlink is known as anchor text. It provides users with a clue about the content or purpose of the linked resource.

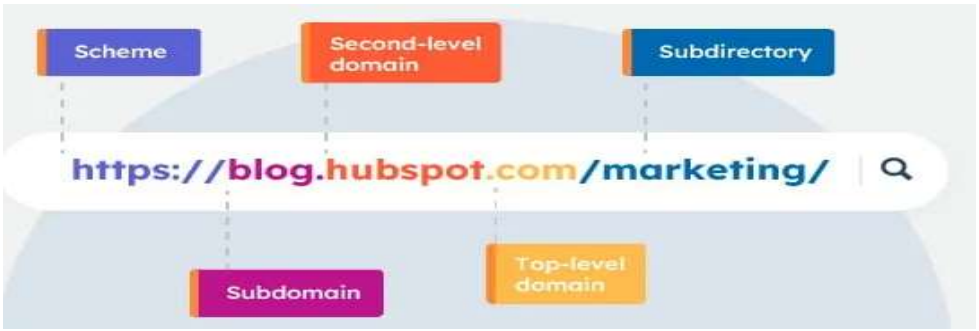
Example of a Hyperlink in HTML:

```
<a href="https://www.example.com">Visit Example Website</a>
```

- **<a>:** This is the HTML tag for creating hyperlinks.
- **href attribute**: This attribute specifies the destination URL.
- **Link Text:** The text between the opening <a> tag and the closing tag is the visible link text.

URL (Uniform Resource Locator) is a reference or address used to access resources on the internet. URLs are essential for locating web pages, images, videos, and other types of content available online.

Parts of URLs



Comparison between Domain Name and URL

A uniform resource locator (URL), sometimes called a web address, contains the domain name of a site as well as other information, including the protocol and the path. For example, in the URL '**https://cloudflare.com/learning/**', '**cloudflare.com'** is the domain name, while '**https**' is the protocol and '/learning/' is the path to a specific page on the website.

Practical Activity 2.1.2: Installation of text editor and web browser

Task:

1. Referring to the previous theoretical activities 2.1.1 you are requested to go to the computer lab to install a text editor and web browser. This task should be done individually.

2. Read key reading 2.1.2 and ask clarification where necessary
3. Apply safety precautions
4. Outline the steps of installing a text editor and web browser.
5. Referring to the outlined steps provided on task 3, Install text editor and web browser.
6. Present your work to the trainer and whole class

Key readings 2.1.2

Installing text editor and web browser from the internet

To install text editor from internet:(let us have an example for VSCode editor)

- Download the Installer
- Go to the Visual Studio Code website.
- Click on the "Download" button to download the installer
- Run the Installer
- Once the download is complete, open the downloaded file (VSCodeSetup.exe).
- If prompted by the User Account Control (UAC), click "Yes" to allow the Accept the License Agreement: installation
- **Follow installation wizard**
- This action will open the installation wizard, which is a series of dialog boxes designed to help you install the software step by step.
- You'll typically see a "Next" button that you click to proceed to the next step.
- You'll see a progress bar that shows the status of the installation.
- This process may take a few seconds to several minutes, depending on the software and your computer's speed.
- After the installation is complete, the wizard will inform you that the process is finished.
- Finally, click "Finish" to close the installation wizard.

To install web browser from internet

- Read the license agreement and click the "I accept the agreement" checkbox, then click "Next"
- Select Destination Location
- Choose the installation folder or leave the default location, then click "Next".

- Select Additional Tasks
- Choose additional tasks such as creating a desktop icon or adding to the PATH (recommended), then click "Next".
- Install
- Click the "Install" button to start the installation process.
- Wait for the installation to complete
- Finish Installation
- Once the installation is complete, you can choose to launch Visual Studio Code immediately by checking the "Launch Visual Studio Code" **checkbox**, then click "**Finish**".

Web Browser Installation

- Locate the web browser installer file from official website and download it .
- Double-click the installer file to begin the installation process.
- Follow the on-screen instructions provided by the installer.
- Review and accept any terms and conditions, if prompted.
- Customise any installation settings, such as installation location or shortcuts, if available.
- Wait for the installation to finish.
- Once the installation is complete, you can launch the web browser from the installed location or through shortcuts

To install text editor and web browser from storage drive

- Connect the storage drive (such as a USB drive) to your computer's USB port.
- Open the storage drive and navigate to the folder where the installation files are located.

Text Editor Installation:

- Find the text editor installer file (.exe or similar).
- Double-click the installer file to start the installation process.
- Follow the on-screen instructions provided by the installer.

Web Browser Installation

- Locate the web browser installer file (.exe or similar).
- Double-click the installer file to begin the installation process.
- Follow the on-screen instructions provided by the installer.

Points to Remember

- Below are general steps for installing text editors on Windows from internet:
- **Step1.**Download the Installer
- **Step2.**Run the Installer
- **Step 3.** Follow installation wizard
- Most Text editor and browsers are open source program, so it is easy to get it and install it into your computer.

Application of learning 2.1

RWANDA TVET BOARD has offered laptops to The TVET Trainers, John Is a trainer in Level three software development who got the laptop, as trainer in software wishes to have applications in his computer to help him to edit html and navigate different websites and search engine for making research. You are asked to help John to install a text editor and web browser in his computer.

Indicative content 2.2: Creating a web page in HTML

Duration: 25 hrs

Theoretical Activity 2.2.1: Description of web page in HTML

Tasks:

1: In small groups, you are requested to answer the following questions related to the web page in html.

i. Describe the different types of websites
ii. Explain the functions of websites based on their types?

2: Participate in group formulation.

3: Present the findings/answers to the whole class

4: Ask the clarification if any

5: For more clarification, read the key readings **2.2.1**. In addition, ask questions where necessary.

Key readings 2.2.1.:

Types of website and its function

Websites can be classified based on various criteria, including their purpose, functionality, content type, and audience. Here we are going to discuss on types of website according to their purpose and its functions:

1. E-commerce Websites: These websites allow businesses to sell products or services online. They typically include features like shopping carts, secure payment gateways, and product listings.

2. Blogging/Content Websites: Are platforms that are primarily focused on delivering informative, educational, or entertaining content to readers. They are designed to share articles, posts, or multimedia content on various topics of interest. These websites are often maintained by individuals, known as bloggers, or by organizations that have a dedicated team of writers or content creators.

The main characteristics of blogging/content websites include:

- **Regular Updates:** These websites are frequently updated with new content to keep readers engaged and attract new visitors. Bloggers often follow a publishing schedule to maintain consistency.

- **Categorized Content:** The content is usually organized into categories or topics, making it easier for readers to browse and find information on specific subjects of interest.

- **Commenting and Interaction:** Blogging websites often allow readers to leave comments, share their opinions, and engage in discussions with the blogger and other readers. This fosters a sense of community and interactivity.

- **Multimedia Content:** Along with written articles, blogging websites often utilize multimedia elements such as images, videos, infographics, and podcasts to enhance the user experience and engage readers.

- **Archives and Search Functionality:** Content websites typically have archives or search functionality that enables readers to find older posts or search for specific topics.

- **Social Sharing Features:** These websites commonly integrate social media sharing buttons to allow readers to easily share the content on various social platforms (Facebook, YouTube, WhatsApp, Instagram, and Facebook Messenger), increasing its reach and visibility.

- **Subscriber Options:** Many blogging websites offer subscription options such as RSS feeds or email newsletters for readers to stay updated with new content.
  Use of RSS feed

  RSS stands for "Really Simple Syndication" and it's just that - a way to syndicate(embed) your content to other sites or tools.
  Email subscription

  An email subscription is when people subscribe to your new posts via email. This means that they receive notifications via email without having to actually visit your blog.
- **Monetization Opportunities:** Some bloggers use advertising, sponsored content, or affiliate marketing to generate income from their websites. They may also offer premium content or products/services for sale.

8. **Portfolio Websites:** These websites are commonly used by professionals, such as artists, photographers, designers, or writers, to showcase their work and skills. They often feature galleries or portfolios to display their projects and accomplishments.

9. **Corporate/Business Websites:** These websites represent companies and organisations. They provide information about their business, services, products, and contact details.

10. **Social Media Websites:** Social media platforms enable users to connect and interact with others, share content, and participate in discussions. Examples include Facebook, Twitter, and LinkedIn.

11. **News/Media Websites:** These websites focus on delivering news articles, reports, and multimedia content to their readers. They often cover various topics like current events, sports, entertainment, and more.

12. **Educational Websites:** Educational websites provide resources and information related to learning and education. They can be online courses, tutorial platforms, or digital libraries.

13. **Non-profit Websites**: These websites belong to non-profit organisations. They aim to promote their cause, raise awareness, and solicit donations or support.

9. Personal Websites: Personal websites are created by individuals to showcase their personal brand, share their thoughts or experiences, or display their personal projects.

The function of a website depends on its purpose and goals.

1. Information Sharing: Websites are commonly used to provide information about a company, organization, individual, or a specific topic. They can include details about products, services, contact information, FAQs, and other relevant information.

2. E-commerce: Many websites serve as online marketplaces where businesses can showcase and sell products or services. They integrate features like shopping carts, secure payment gateways, and product listings to facilitate transactions.

4. **Communication and Interaction:** Websites can provide platforms for communication and interaction between individuals or groups. Examples include social media websites, forums, and messaging platforms that allow users to connect, share information, and engage in discussions.

5. **Content Publication**: Websites designed for content creation and publication, such as blogs and news websites, enable individuals or organizations to share articles, videos, podcasts, and other forms of multimedia content.

6. **Education and E-Learning:** Websites can be used as platforms for online learning and educational resources. They host educational content, videos, quizzes, and interactive modules to facilitate learning outside of traditional classrooms.

7. **Community Building:** Some websites are dedicated to fostering communities around specific interests, hobbies, or professions. They allow users to connect with like-minded individuals, share experiences, and collaborate on projects.

   7. Branding and Marketing: Websites play a crucial role in establishing and promoting a brand's online presence and identity. Companies and organizations showcase their values, mission, and portfolio to build credibility and attract customers.

8. **Customer Support:** Websites often provide customer support features, including FAQs, live chat, support tickets, and knowledge bases. These features help users find answers to their questions and resolve issues.

9. **Recruitment and Job Boards**: Websites dedicated to job postings and recruitment allow employers to post job openings, receive applications, and facilitate the hiring process.

   10. Entertainment and Media: Many websites are designed for entertainment purposes, such as online gaming platforms, streaming services for movies and music, or content sharing platforms for viral videos and memes.

   These are just a few examples of the function’s websites can serve. Depending on the specific needs and goals, websites can be customised with various features and functionalities to deliver unique user experiences.

Theoretical Activity 2.2.2: Introduction to HTML

Tasks:

1: In small groups, you are requested to answer the following questions related to the introduction to html:

a. What do you understand about an Html?
b. Describe an HTML document.

2. Provide the answer for the asked questions and write them on papers.

3.Present the findings/answers to the whole class

4. For more clarification, read the key readings 2.2.2. In addition, ask questions where necessary.

Key readings 2.2.2

Introduction to html

HTML stands for Hypertext Markup Language. It is a standard markup language for web page creation. It allows the creation and structure of sections, paragraphs, and links using HTML elements (the building blocks of a web page) such as tags and attributes**.**

HTML has a lot of use cases, namely:

Web development. Developers use HTML code to design how a browser displays web page elements, such as text, hyperlinks, and media files.

Internet navigation. Users can easily navigate and insert links between related pages and websites as HTML is heavily used to embed hyperlinks.

Web documentation. HTML makes it possible to organize and format documents, similarly to Microsoft Word.

HTML provides a set of tags or elements that define the structure and presentation of content on the internet.

HTML is a markup language, not a programming language. Its primary purpose is to structure content on the web by using tags to define elements such as headings, paragraphs, links, images, and more.

Structure of an HTML Document:

An HTML document is structured with a set of HTML tags. The basic structure includes an opening <html> tag and a closing </html> tag, which contains two main sections: the <head> section (metadata and links to external resources) and the <body> section (content of the page).

```
<!DOCTYPE html>
<html>
<head>
    <!-- Metadata, links to stylesheets, etc. go here -->
</head>
<body>
    <!-- Content of the web page goes here -->
</body>
</html>
```

HTML Elements and Tags:

HTML Tag: The specific characters that define the start and end of an HTML element**.**

HTML Element: The complete structure defined by the opening tag, content, and closing tag. Tags are enclosed in angle brackets. For example, a paragraph is defined by the <p> tag.

```
<p>This is a paragraph of text.</p>
```

Attributes:

HTML tags can have attributes that provide additional information about the element. Attributes are added within the opening tag. For example, the **<a>** (anchor) tag can have an href attribute specifying the link destination.

```
<a href="https://www.example.com">Visit Example Website</a>
```

HTML Version and History:

HTML has evolved over time, with several versions released to add new features and improvements. Here are the main versions of HTML that have been available in the marketplace:

1. HTML 1.0 (1993)

Introduction: The first version of HTML, introduced by Tim Berners-Lee.

Features: Basic structure for web pages, including elements like headings, paragraphs, and links.

2. HTML 2.0 (1995)

Introduction: Standardized by the Internet Engineering Task Force (IETF).

Features: Enhanced the basic structure, added support for forms, and improved overall functionality.

3. HTML 3.2 (1997)

Introduction: Released by the World Wide Web Consortium (W3C).

Features: Added support for tables, applets, and scripting languages like JavaScript.

4. HTML 4.0 (1997)

Introduction: Published by W3C.

Features: Introduced more complex layouts, improved form controls, support for stylesheets (CSS), and better internationalization**.**

Versions: HTML 4.0, HTML 4.01 (1999, minor update with bug fixes and improvements)

5. XHTML 1.0 (2000)

Introduction: A reformulation of HTML 4.01 using XML**.**

Features: Stricter syntax rules, aimed at ensuring cleaner and more consistent code**.**

Versions: XHTML 1.0 Transitional, XHTML 1.0 Strict, XHTML 1.0 Frameset**.**

XHTML 1.1 (2001): Further refinement of XHTML 1.0.

6. HTML5 (2014)

Introduction: Developed by W3C and WHATWG (Web Hypertext Application Technology Working Group).

Features: Significant update with new elements (e.g., <header>, <footer>, <article>, <section>), support for multimedia (audio and video elements), improved form controls, APIs for offline storage, and more semantic elements.

Status: Living standard, meaning it is continuously updated and improved**.**

7. HTML Living Standard (2014-present)

Introduction: Maintained by WHATWG.

Features: Continuous updates to HTML5, introducing new elements and APIs as the web evolves.

Goal: Ensure HTML stays up-to-date with modern web development practices and improved

Theoretical Activity 2.2.3: Description of html tags

Tasks:

1. In small groups, you are requested to answer the following questions related to the description of html tags.
   i. What do you understand about HTML tag?
   ii. What are the types of Html tags?
   iii. Give a description of html attributes.
2. Provide the answer for the asked questions and write them on papers/flipchart.
3. Present the findings/answers to the whole class
4. For more clarification, read the key readings 2.2.3. In addition, ask questions where necessary.

Key readings 2.2.3

HTML tags are the keywords on a web page that define how your web browser must format and display your web page.

Almost all tags contain two parts, an opening, and a closing tag. For example, <html> is the opening tag and </html> is the closing tag. Note that the closing tag has the same text as the opening tag, but has an additional forward-slash (/) character.

Types of HTML Tags

There are two kinds of HTML tags: paired and unpaired tag.

Paired tags require an opening tag that turns a formatting feature on and a closing tag that turns the feature off. Paired tags must surround the text you want formatted with that feature.

Examples:

1**. **This text is in italics. ****

Note: Here is called opening tag. and **** is called closing tag.

2. Paragraph Tag: ** ** and ** **

   Example: ** **This is a paragraph**. **

3.Heading Tags: **<h1>** to **<h6>** and **</h1>** to **</h6>**

   Example: **<h1>**This is a heading**.</h1>**

Unpaired tag take effect at once no need for closing tag. Unpaired tags are also known as Singular or Stand-Alone Tags.

1. Image Tag: **<img>**
   Example: <img src="image.jpg" alt="Image description">
2. Line Break Tag: **
**
   Example:
3. Horizontal Rule Tag: **<hr>**
   Example: <hr>
4. Input Tag: **<input>**
   Example: <input type="text" name="username">
5. Meta Tag: **<meta>**
   Example: <meta charset="UTF-8">
6. Link Tag: **<link>**
   Example: <link rel="stylesheet" href="styles.css">

HTML tags can be classified into various categories based on their functionality, structure, and purpose within a web document. Here's an overview of the main categories(groups) of HTML tags:

1. **Structural Tags**

   Structural tags define the layout and organization of a web document. These tags help structure the content logically.

   <html>: The root element of an HTML document.
   <head>: Contains meta-information about the document (e.g., title, scripts, styles).
   <body>: Contains the content of the document visible to users.
   <header>: Defines a header section for a document or section.
   <footer>: Defines a footer for a document or section.
   <section>: Defines a section within the document.
   <article>: Defines an independent, self-contained article.
   <nav>: Defines navigation links.
   <aside>: Defines content aside from the main content.

2. **Formatting Tags**

   Formatting tags apply specific styles or formats to text.

   <b>: Bold text (use for semantic emphasis).

   <i>: Italic text (use for semantic emphasis).

   <u>: Underlined text.

   <small>: Smaller text.

   <mark>: Highlighted text.

   <sub>: Subscript text.

<sup>: Superscript text

3. **Heading Tags**
   Heading tags define headings of different levels within the document.
   <h1> to **<h6>:** Define headings from level 1 (most important) to level 6 (least important).

4. **Linking Tags**
   Linking tags are used to create hyperlinks, connecting different parts of a document or external resources.
   <a>: Defines a hyperlink.

5. **List Tags**
   List tags create lists of items.
   <ul>: Unordered list.
   <ol>: Ordered list.
   <li>: List item.
   <dl>: Description list.
   <dt>: Description term.
   <dd>: Description definition.

6. **Media Tags**
   Media tags are used to embed multimedia content.
   <img>: Embeds an image.
   <audio>: Embeds sound content**.**
   <video>: Embeds video content**.**
   <source>: Specifies multiple media resources**.**
   <track>: Specifies text tracks for **<video>** and **<audio>**

7. **Table Tags**
   Table tags create tables for displaying data in rows and columns.
   <table>: Defines a table.
   <tr>: Defines a table row.
   <th>: Defines a table header cell.
   <td>: Defines a table data cell.
   <caption>: Defines a table caption.
   <thead>: Groups the header content.
   <tbody>: Groups the body content.
   <tfoot>: Groups the footer content

8. **Form tags**

Form tags create interactive forms for user input.

<form>: Defines an HTML form.
<input>: Defines an input control.
<textarea>: Defines a multiline text input.
<button>: Defines a clickable button.
<label>: Defines a label for an <input> element.
<select>: Defines a dropdown list.
<option>: Defines an option in a dropdown list.
<fieldset>: Groups related elements in a form.
<legend>: Defines a caption for a <fieldset>

9. **Semantic Tags**

Semantic tags provide meaning to the web page content, helping search engines and browsers understand the context.

<main>: Specifies the main content.

<figure>: Specifies self-contained content**.**

<figcaption>: Defines a caption for a <figure>.

<time>: Defines a specific time**.**

<address>: Defines contact information**.**

10. **Script and Style Tags**

These tags are used to include scripts and styles in a document.
<script>: Embeds or references a client-side script.
<style>: Embeds CSS styles within the document.
<link>: References an external CSS file.

HTML attributes

An HTML attribute is a piece of markup language used to adjust the behavior or display of an HTML element. For example, attributes can be used to change the color, size, or functionality of HTML elements.

Key Points About HTML Attributes

1. **Purpose**: HTML attributes define properties of elements, provide additional information, and control the behavior of elements**.**
2. **Syntax:**

- Attributes are written within the opening tag, after the tag name.
- Attributes are generally written in lowercase.
- Attribute values should be enclosed in double quotes ("), but single quotes (') are also acceptable.

Components of an HTML Attribute:

1. Attribute Name:

- This is the identifier for the attribute, indicating what property or characteristic is being defined. It is always written in lowercase.
- - Example: In href, class, and id, each of these is an attribute name.

2. Attribute Value:

- This is the value assigned to the attribute, providing specific information or settings. The value is typically enclosed in double quotes (" ") or single quotes (' ').
- Example: In href="https://www.example.com",

the value is https://www.example.com.

Syntax:

```
<tagname attribute="value">Content</tagname>
```

Examples:

<p **align**="**cente**r"> This text is aligned to center **</p>**

```
<img src="avatar.jpg" alt="User Avatar">
```

The required **alt** attribute for the **<img>** tag specifies an alternate text for an image, if the image for some reason cannot be displayed.

Points to Remember

- An HTML attribute is a piece of markup language used to adjust the behavior or display of an HTML element.
- There are two kinds of HTML tags: paired and unpaired.
- HTML tags are the keywords on a web page that define how your web browser must format and display your web page.

Practical Activity 2.2.4: Developing first simple webpage

Task:

1: Referring to the previous theoretical activities (2.2.3) you are requested to go to the computer lab to develop a simple web page. This task should be done individually.

```
<! DOCTYPE html>

<html><head> <title>First Simple developed Website</title>

</head> <body> Hello software developer </body> </html>
```

2: Read key reading 2.2.4 and ask clarification where necessary
3: Apply safety precautions.
4: Outline the steps to develop a simple web page.
5: Referring to the outlined steps provided on task 3, Develop the simple webpage
6. Present your work to the trainer and whole class

Key readings 2.2.4

Steps to develop a simple web page:

Writing an HTML tag involves several steps, and it depends on the type of HTML element you want to create. Here are the general steps to write an HTML tag:

1. **Open a Text Editor**:

   Use a text editor like Notepad, Visual Studio Code, Sublime Text, or any other code editor of your choice.

2. **Create a New Document:**

   Open a new document in the text editor. Save it with an appropriate file extension, such as .html (e.g., index.html).

3. **Setup a Document Structure:**

   Begin by setting up the basic structure of an HTML document:

   <!DOCTYPE html>

   <html lang="en">

   <head>

```
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Your Page Title</title>
</head>
<body>
  <!-- Your content goes here -->
</body>
</html>
```

4. **Choose an HTML Element:**
   Decide which HTML element you want to create. For example, a paragraph, heading, link, image, etc.

- **Start writing tags and include contents**
  Write the tags for the chosen element. The basic format is <tagname> </tagname>.

```
<p>This is a paragraph.</p>
```

Attributes (Optional):
If the element requires attributes, add them to the opening tag. Attributes follow the format attribute="value".

```
<a href="https://www.example.com">Visit Example.com</a>
```

5. **Save file**
   Step 1: Click on the "File" menu in the text editor.
   Step2: Choose "Save" or "Save As."
   Step3: Choose File Name: Enter a file name for your HTML file, including the .html extension. For example, index.html. Remember also to set file type as HTML or All types

   Step4: Choose Save Location: Select the directory or folder where you want to save the HTML file.

   Step 5: Confirm Save: Click the "Save" or "Save As" button to confirm and save the file.

6. **To display your html contents through the web browser (opening html file)**

a) Locate your file (Go where your file is saved).
b) Right-click on file icon
c) Select "Open with" from the context menu.
d) Choose a browser like "Google Chrome" or "Microsoft Edge."

Other methods (Drag and Drop Method:))

a) Open your web browser.
b) Locate your HTML file in your file explorer.
c) Drag the HTML file and drop it into the browser window.

Points to Remember

- **Developing a simple web page involves several steps. Here are steps develop your f irst web page:**

  Step1: Open a Text Editor
  Step2: create a document
  Step 3: Setup a Document Structure
  Step 4: Choose an HTML Elements and type them
  Step 5: save file
  Step 6: To display your html contents through the web browser

Theoretical Activity 2.2.5: Description of HTML tag categories

Tasks:

1. In small groups, you are requested to answer the following questions related to the html tag categories:

I. Step 1: Introduce the activity and request learners to respond to the following questions:

a) Can you explain the different categories of HTML Tags?

b) What is the difference between and tags in HTML?

2.Provide the answer for the asked questions and write them on papers.

3.Present the findings/answers to the whole class

4.For more clarification, read the key readings 2.2.5. In addition, ask questions where necessary.

Key readings 2.2.5.

HTML tags are categorized based on their functionality and how they structure or style the content of a webpage. Below are the different categories of HTML tags:

1. Text formatting tags

Text formatting tags in HTML refers to the way text is displayed on a web page. It is the process of applying various styles, colors, fonts, sizes, and other visual enhancements to text content within an HTML document. HTML offers a range of tags that can be used to format.

Text Formatting tag	Meaning
<b>	Bold text
****	Important text
<i>	Italic text
****	Emphasized text
<mark>	Marked text
<small>	Smaller text
<del>	Deleted text
<ins>	Inserted text
<sub>	Subscript text
<sup>	Superscript text

1. Tables tags

Tag	Description
<table>	Defines a table
<th>	Defines a header cell in a table
<tr>	Defines a row in a table
<td>	Defines a cell in a table
<caption>	Defines a table caption
<colgroup>	Specifies a group of one or more columns in a table for formatting
<col>	Specifies column properties for each column within a <colgroup> element
<thead>	Groups the header content in a table
<tbody>	Groups the body content in a table
<tfoot>	Groups the footer content in a table

2. Form tags

Tag	Description
<input>	element can be displayed in several ways, depending on the type attribute.

<label>	element defines a label for several form elements
<select>	element defines a drop-down list
<textarea>	element defines a multi-line input field (a text area):
<button>	element defines a clickable button
<fieldset>	element is used to group related data in a form
<legend>	element defines a caption for the <fieldset> element
<datalist>	The <datalist> element specifies a list of pre-defined options for an <input> element.
<output>	The <output> element represents the result of a calculation (like one performed by a script).

3. **HTML Headings**
   HTML headings are defined with the <h1> to <h6> tags.
   <h1> defines the most important heading. <h6> defines the least important heading.
4. **List tags**
   HTML lists allow web developers to group a set of related items in lists The <li> tag defines a list item. The <li> tag is used inside ordered lists(<ol>), unordered lists **(<ul>),** and in menu lists (<menu>). In <ul> and <menu>, the list items will usually be displayed with bullet points. In <ol>, the list items will usually be displayed with numbers or letters.
   Examples:
   An unordered HTML list:

- Item
- Item
- Item
- Item

An ordered HTML list:

i. First item
ii. Second item
iii. Third item
iv. Fourth item

5. **HTML Media Tags**
   Multimedia on the web is sound, music, videos, movies, and animations

The HTML **<video>** element is used to show a video on a web page.
To show a video in HTML, use the <video> element:

```
<video width="320" height="240" controls>
  <source src="movie.mp4" type="video/mp4">
  <source src="movie.ogg" type="video/ogg">
Your browser does not support the video tag.
</video>
```

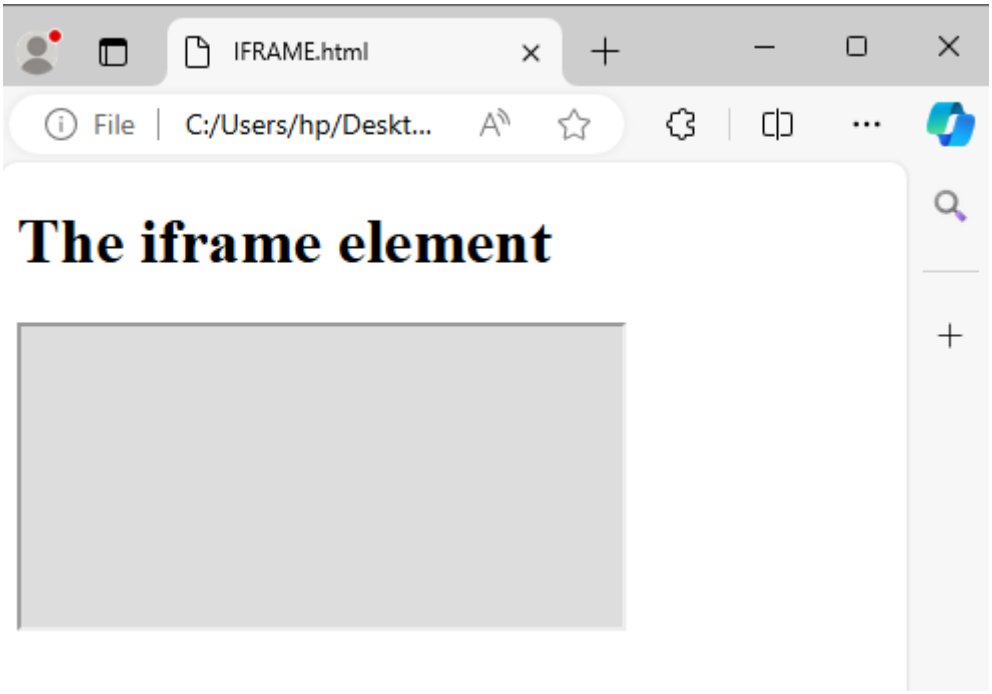
To play an audio file in HTML, use the **<audio>** element:

```
<audio controls>
  <source src="horse.ogg" type="audio/ogg">
  <source src="horse.mp3" type="audio/mpeg">
Your browser does not support the audio element.
</audio>
```

6. **Code Tags**
   The <code> tag is used to define a piece of computer code. The content inside is displayed in the browser's default monospace font.
7. **Frame tags (iFrame) in html 5**
   The <iframe> tag specifies an inline frame.
   An inline frame is used to embed another document within the current HTML document.
   Example of application:

```
<!DOCTYPE html>
<html>
<body>

<h1>The iframe element</h1>

<iframe src="https://www.w3schools.com" title="W3Schools Free Online Web Tutorials"></iframe></body></html>
```

   Output



8. **HTML Comments**

The comment tag is used to insert comments in the source code. Comments are not displayed in the browsers.

You can use comments to explain your code, which can help you when you edit the source code at a later date. This is especially useful if you have a lot of code.

<!--This is a comment. Comments are not displayed in the browser**-->**

This is a paragraph.

9. **Grouping Tags (Div and span)**

In HTML, the ** ** and **** tags are used for structuring and styling content. creates block-level containers for grouping elements, while creates inline containers for styling specific portions of text or elements within a block-level container.

The element is an inline container used to apply styles or scripting to a specific portion of text or inline elements.

Example:

This is a red word in the paragraph.

In this example, the element is used to apply a red color to the word "red."

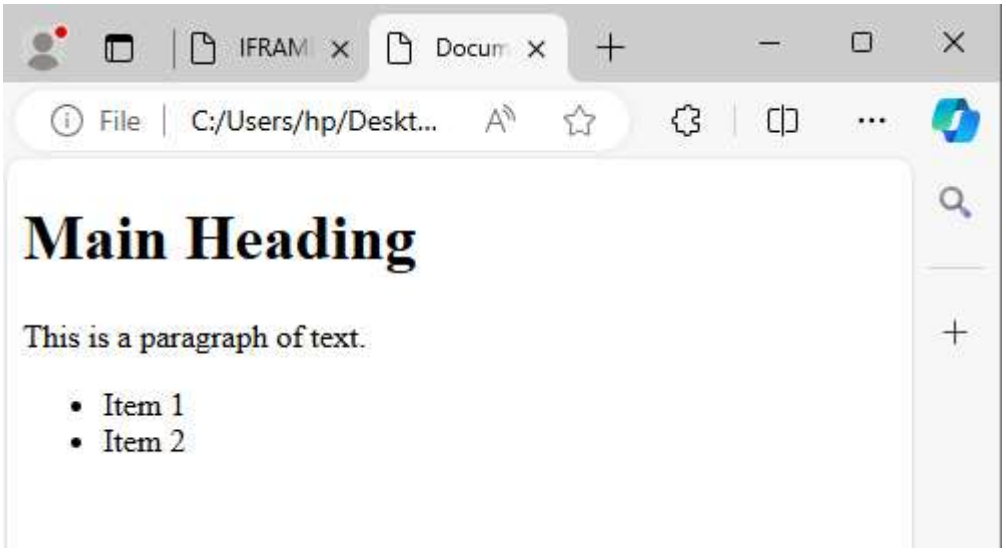
The element is a block-level container that is often used to group together and apply styles to a block of content. It does not have any inherent styling or behavior and is typically used to create layout structures.

Example:

  <h1>Main Heading</h1>

```
 <p>This is a paragraph of text.</p>
 <ul>   <li>Item 1</li>  <li>Item 2</li>
 </ul>
</div>
```

Output



In this example, the element with the id "container" wraps multiple elements, creating a block-level container.

Practical Activity 2.2.6: Developing webpage using HTML tag categories

Task:

1. Referring to the previous theoretical activity (2.2.5) you are requested to go to the computer lab to develop a web page using html categories. This task should be done individually.

2. Read key reading 2.2.6 and ask clarification where necessary
3. Apply safety precautions
4. Outline the steps to develop a webpage using html tag categories.
5. Referring to the outlined steps provided on task3, develop a webpage using tag categories.
6. Present your work to the trainer and whole class

Key readings 2.2.6

To develop a webpage using HTML categories, you can follow these steps:

1.Plan Your Webpage: Determine the purpose and content of your webpage. Identify the text, images, links, forms, and other elements you want to include.

2. Choose HTML Categories: Based on your content plan, select the appropriate HTML categories for organising and structuring your webpage content. Categories include text-level elements, block-level elements, form elements, semantic elements, and more.

3. **Create the HTML Document Structure:**

- Start with the <!DOCTYPE html> declaration to specify the HTML version.
- Use the <html> element as the root element of your webpage.
- Include the <head> section for meta-information like the page title, character encoding, and links to external resources.
- Add a <title> within the head section to define the title of your webpage.
- Use the <body> element to contain the visible content of your webpage.

4. Utilize Text-Level Elements:

- Use heading elements <h1> to <h6> for titles and headings.
- Employ the <p> element for paragraphs of text.
- Apply , , and for text formatting and styling.

5. Structure Content with Block-Level Elements:

- Organise content into sections using , <section>, <article>, <header>, <footer>, and <nav> elements.

6. Incorporate Lists and Links:

- Use <ul> and <ol> for unordered and ordered lists respectively.
- Employ <li> for list items.
- Create hyperlinks with the <a> tag.

7. Insert Images and Multimedia:

- Include images using the <img> tag with the src attribute.
- Embed multimedia content like videos and audio using <video> and <audio> tags.

8. Develop Forms for Interaction:

- Implement interactive forms using <form>, <input>, <textarea>, <select>, and <button> elements for user input and submission.

9. Add Meta Information:

- Include meta information such as character encoding, viewport settings, and keywords using <meta> tags within the <head> section.

HTML tag categories:

1. **Step to write a Formatting tags:**

To apply formatting to text in HTML, you can use various formatting tags. Here are the steps to write formatting tags in HTML:

I. Open a Text Editor

Open a text editor such as Notepad, Sublime Text, Visual Studio Code, or any other code editor of your choice.

ii. Identify the text that you want to format.

- Determine the specific portion of text that you want to apply formatting to, such as making it bold, italic, or underlined.

iii. Choose the appropriate formatting tag.

- Select the HTML tag that corresponds to the desired formatting style. Common formatting tags include for bold, for italic, <u> for underline, for strong emphasis, for emphasis, etc.

iv.Write the opening tag.

- Start by writing the opening tag before the text you want to format.

- The opening tag typically consists of the desired formatting tag enclosed in angle brackets (<>).

- Example:

v. Write the formatted text.

- Place the text you want to format immediately after the opening tag.

- Example: This text will be bold.

vi. Write the closing tag.

- End the formatted section by writing the closing tag after the formatted text.

- The closing tag is similar to the opening tag but includes a forward slash (/) before the tag name.

- Example:

vii. Repeat the process for additional formatting.

- If you want to apply multiple formatting styles to the same text, you can nest the formatting tags within each other.

- Example: This text will be bold and italic.

viii. Save and test your HTML file.

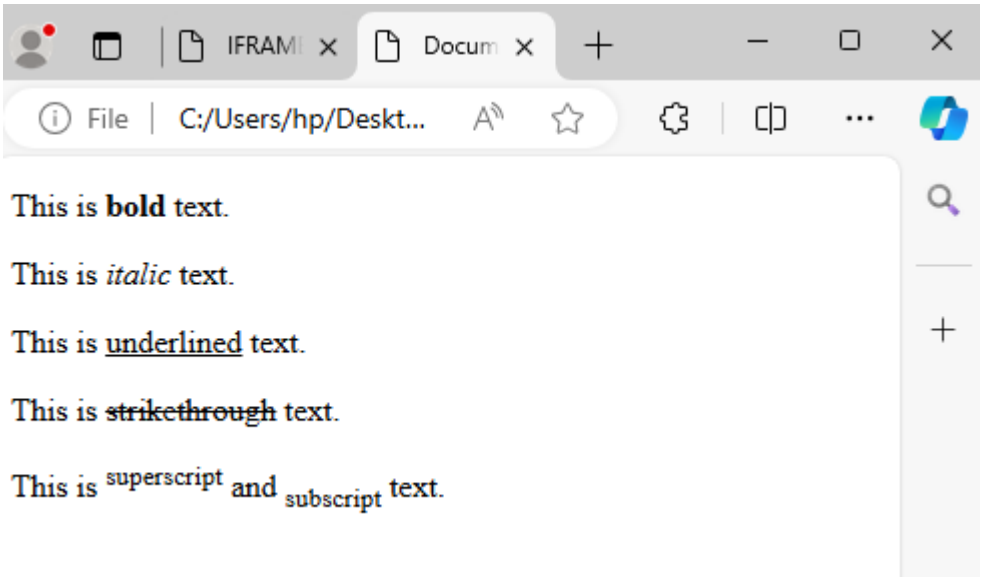
- Save the HTML file with a .html extension.

- Open the file in a web browser to see the formatted text.

Example:

```
<p>This is <strong>bold</strong> text.</p>

<p>This is <em>italic</em> text.</p>

<p>This is <u>underlined</u> text.</p>

<p>This is <s>strikethrough</s> text.</p>

<p>This is <sup>superscript</sup> and <sub>subscript</sub> text.</p>
```

Output



2. Steps to write Table tags

To create a table in HTML, you can use the <table>, <tr>, <th>, and <td> tags. Here are the steps to write table tags in HTML:

i. Open a Text Editor

Open a text editor such as Notepad, Sublime Text, Visual Studio Code, or any other code editor.

ii. Determine the structure of your table.

- Decide on the number of rows and columns your table will have.

iii. Start with the <table> tag.

- Begin by writing the opening <table> tag to define the start of the table.

- Example: <table>

iv. Define table rows with the <tr> tag.

- Write the opening <tr> tag to define a table row.

- Example: <tr>

v. Add table headers with the <th> tag.

- Write the opening <th> tag to define a table header cell.

- Place the header content between the opening and closing <th> tags.

- Example: <th>Header 1</th>

vi. Add table data cells with the <td> tag

- Write the opening <td> tag to define a table data cell.

- Place the cell content between the opening and closing <td> tags.

- Example: <td>Data 1</td>

vii. Repeat steps iv and vi to add more headers and data cells.

- Add additional <th> tags for each header cell in the row.

- Add additional <td> tags for each data cell in the row.

viii. Close the table row with the closing </tr> tag.

- Write the closing </tr> tag to close the table row.

ix. Repeat steps iv to viii to add more rows to the table.

- Add additional <tr> tags and repeat steps iv to viii to add more rows.

x. Close the table with the closing </table> tag.

- Write the closing </table> tag to close the table.

xi. Save and test your HTML file.

- Save the HTML file with a .html extension.

- Open the file in a web browser to see the table structure and content.

Here is an example of a table with Table Heading, three row and two columns.

<!DOCTYPE html><html> <head>

<title>HTML Table Header</title> </head>

<body>

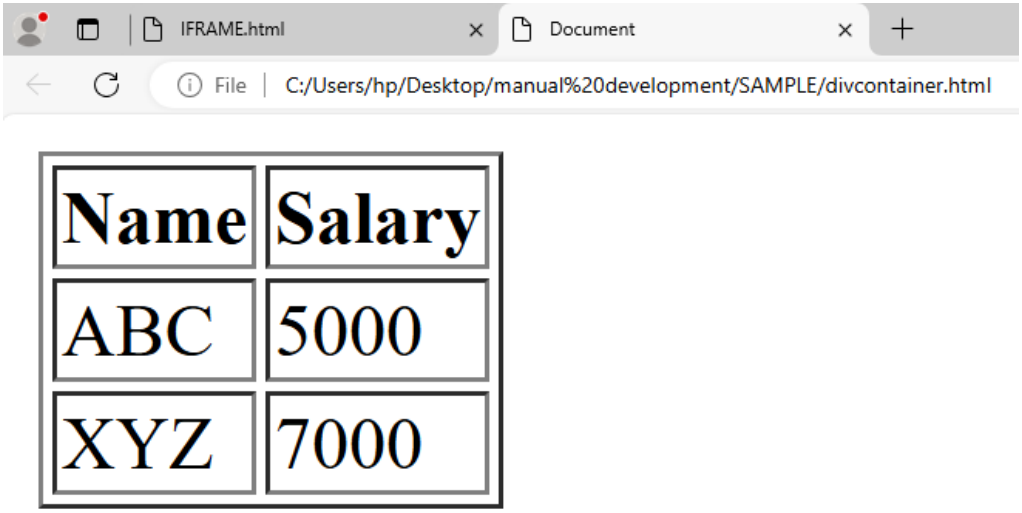
<table border = "1"> <tr> <th>Name</th> <th>Salary</th> </tr>

<tr> <td>ABC</td> <td>5000</td> </tr>

<tr> <td>XYZ</td> <td>7000</td> </tr> </table>

</body></html>

output



3. Steps to create Form (using form tags)

To create a form in HTML, you can use the <form> tag along with various form elements.

Here are the steps to write form tags in HTML:

1. Open a Text Editor

Open a text editor such as Notepad, Sublime Text, Visual Studio Code, or any other code editor.

2. Start with the <form> tag.

- Begin by writing the opening <form> tag to define the start of the form.

- Example: <form>

3. Specify the form's action and method attributes.

- Use the action attribute to specify the URL or file where the form data will be submitted.

- Use the method attribute to specify the HTTP method for submitting the form data (e.g., GET or POST).

- **Example**: <form action="submit.php" method="POST">

4.Add form elements within the form.

- Include various form elements such as <input>, <select>, <textarea>, etc., to collect user input.

- Example:

```
<form action="submit.php" method="POST">
 <input type="text" name="username" placeholder="Username">
 <input type="password" name="password" placeholder="Password">
 <input type="submit" value="Submit">
</form>
```

5. Customize form elements as needed.

- Adjust form elements' attributes, such as name, type, value, placeholder, etc., to suit your requirements.

- Add labels, styling, and validation attributes to enhance the form's usability and functionality.

6.Close the form with the closing </form> tag.

- Write the closing </form> tag to close the form element.

7.Save and test your HTML file.

- Save the HTML file with a .html extension.

- Open the file in a web browser to see and interact with the form.

Example:

```
<!DOCTYPE html><html> <head>
 <title>first page</title>
```

```
 </head> <body>
 <form action="">
<label> First Name </label><br>
<input type="text" name="fname" value=""><br>
<label> Last Name </label><br>
<input type="lname" name="lname" value=""><br>
<label> Address </label><br>
<input type="text" name="address" value=""><br><br>
<input type="radio" name="gender" value="male" /> Male <br />
<input type="radio" name="gender" value="female" /> Female <br />
<b><label> Select the module you learned </label><br></b>
<input type="checkbox" name="gender" value="1" /> Website Development <br />
<input type="checkbox" name="gender" value="2" /> UX/UI design <br />
<input type="checkbox" name="gender" value="2" /> Vue framework <br />
<input type="checkbox" name="gender" value="2" /> Version Control <br />
   <b> <label>Select file to upload:</label> </b>
<input type="file" name="newfile">  <br><br>
Pick your Favorite color:
<input type="color" name="color" value=""> <br><br>
<input type="date" name="date"> date of birth:<br><br>
<input type="submit" value="submit" onclick="alert('Are You sure You want to Submit?')">
</form> </body></html>
```

Output

First Name

Last Name

Address

Male

Female

Select the module you learned

Website Development

UX/UI design

Vue framework

Version Control

Select file to upload: Choose File No file chosen

Pick your Favorite color:

mm/dd/yyyy date of birth:

submit

Use select tags

The <select> tag is used to create a drop-down list in HTML. The <option> tag is used to define the available options in the list.

<html> <head> <title>first page</title> </head>

<body>

<form action="">

<label>Choose your District :</label>

```
        <select name="District">
          <option value="none">none</option>
          <option value="Kicukiro">Kicukiro</option>
          <option value="Muhanga">Muhanga</option>
          <option value="Rubavu">Rubavu</option>
          <option value="kirehe">kirehe</option>
          <option value="Rulindo">Rulindo</option>
        </select>
      </form>
</body>
</html>
```

Contact Form

The form will include Name, Email, and Message fields.

```
<html><head><title>first page</title></head>
    <body> <span>Contact Me</span>
        <form> <textarea name="message"></textarea>
        <input type="submit" value="SEND" class="submit" />
        </form>
    </body>
    </html>
```

4. Steps to write a Headings tags

To create headings in HTML, you can use the <h1> to <h6> tags, which represent different levels of headings. Here are the steps to write heading tags in HTML:

i. Open a Text Editor

Open a text editor such as Notepad, Sublime Text, Visual Studio Code, or any other code editor.

ii. Determine the hierarchy and importance of your headings.

- Decide which heading level is appropriate for each section of your content, considering the structure and importance of the information.

iii. Start with the appropriate <h1> to <h6> tag.

- Begin by writing the opening tag for the desired heading level.

- Example: <h1>

iv. Add the heading text.

- Place the text you want to use as the heading between the opening and closing tags.

- Example: <h1>This is a Heading</h1>

v. Repeat steps 2 and 3 for additional headings.

- Use the appropriate <h2>, <h3>, <h4>, <h5>, or <h6> tags for lower-level headings as needed.

- Example:

<h2>Subheading 1</h2>

<h3>Subheading 2</h3>

vi. Save and test your HTML file.

- Save the HTML file with a .html extension.

- Open the file in a web browser to see the headings rendered with their respective

styles.

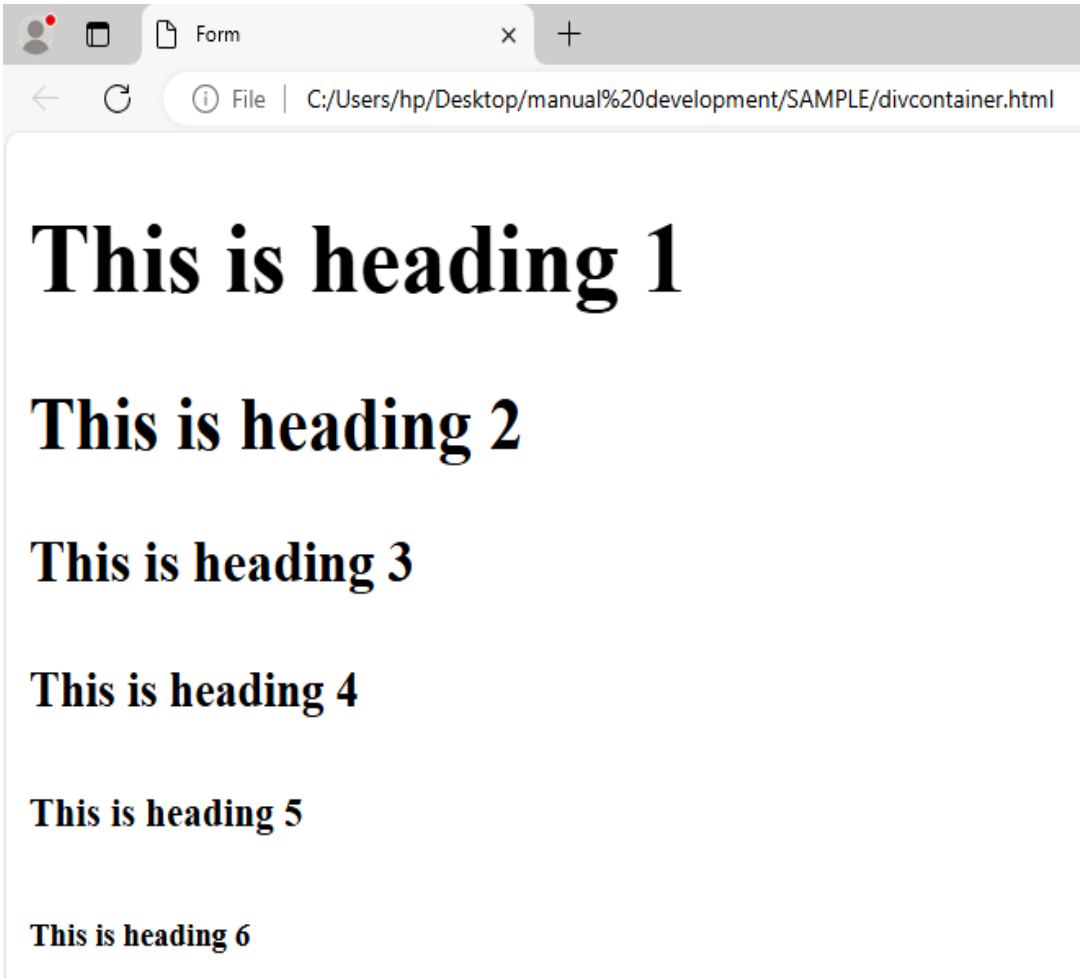
HTML includes six levels of headings, which are ranked according to importance.

These are <h1>, <h2>, <h3>, <h4>, <h5>, and <h6>.

The following code defines all of the headings:

```
<html>
<head>
<title>first page</title>
</head>
<body>
<h1>This is heading 1</h1>
<h2>This is heading 2</h2>
<h3>This is heading 3</h3>
<h4>This is heading 4</h4>
<h5>This is heading 5</h5>
<h6>This is heading 6</h6>
</body>
</html>
```

Output



5. Horizontal Lines

To create a horizontal line, use the <hr /> tag.

```
<html>
  <head>
  <title>first page</title>
  </head>
  <body>
     <h2>This is a horizontal line </h2>
  <hr />
  </body>
  </html>
```

Output

5.Steps to write list tags

To create lists in HTML, you can use the <ul>, <ol>, and <li> tags. Here are the steps to write list tags in HTML:

1. Open a Text Editor:

- Open a text editor such as Notepad, Sublime Text, Visual Studio Code, or any other code editor.

2.Unordered List:

Start with the <ul> tag.

- Begin by writing the opening <ul> tag to define the start of the unordered list.

- Example:

3. Add list items with the <li> tag.

- Write the opening <li> tag to define a list item.

- Place the content of each list item between the opening and closing <li> tags.

- Example:

```
<ul>
<li>List item 1</li>
<li>List item 2</li>
</ul>
```

4.Repeat step 2 to add more list items.

- Add additional <li> tags for each list item in the unordered list.

5.Close the unordered list with the closing tag.

- Write the closing tag to close the unordered list.

Ordered List:

Start with the <ol> tag.

- Begin by writing the opening <ol> tag to define the start of the ordered list.

- Example:

1. Start with the <ol> tag.

- Write the opening <li> tag to define a list item.

- Place the content of each list item between the opening and closing <li> tags.

- Example:

 List item 1

List item 2

2. Add list items with the <li> tag.

- Add additional <li> tags for each list item in the ordered list.

3. Repeat step 2 to add more list items.

- Write the closing tag to close the ordered list.

4. Close the ordered list with the closing tag.

Nested List:

1. Nest one list inside another.

Nest one list inside another.

- You can nest an ordered or unordered list inside another list item to create a nested list.

- Example:

List item 1 Nested item 1
Nested item 2

List item 2

6.Save and test your HTML file.

- Save the HTML file with a .html extension.

- Open the file in a web browser to see the list structure and content.

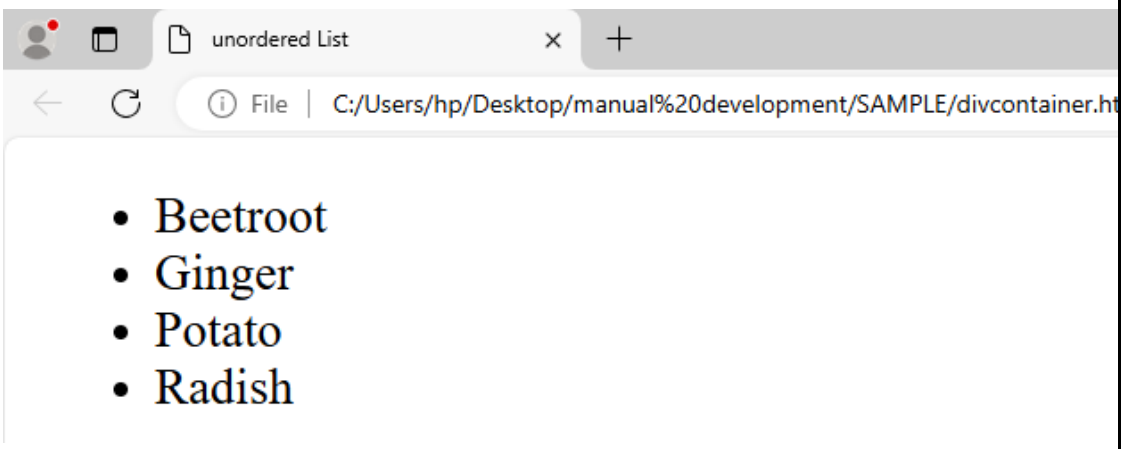
HTML Unordered List

<!DOCTYPE html>

<html>

```
<head>
<title>HTML unordered
List</title>
</head>
<body>
<ul>
<li>Beetroot</li>
<li>Ginger</li>
<li>Potato</li>
<li>Radish</li>
</ul>
</body>
</html>
```

Output



You can use type attribute for <ul> tag to specify the type of bullet you like. By default, it is a disc. Following are the possible options

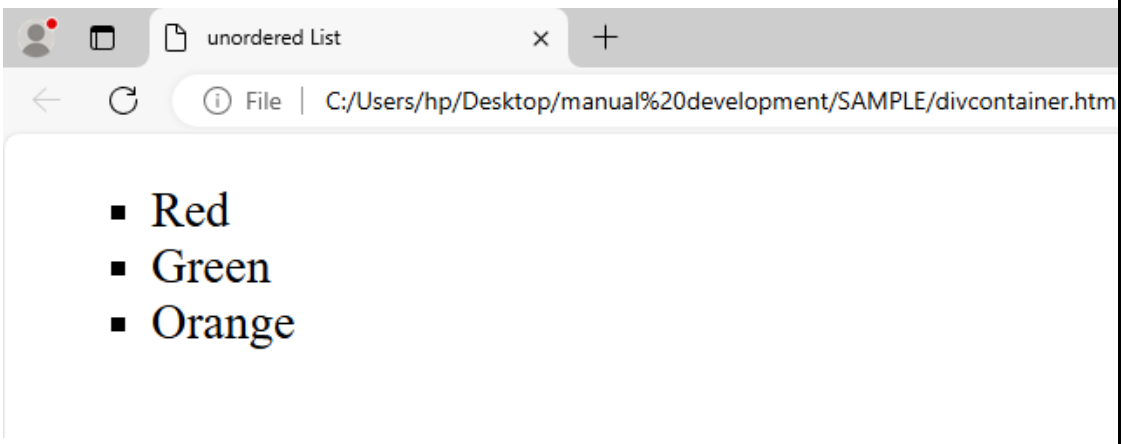
1. <ul type = "square">
2. <ul type = "disc">
3. <ul type = "circle">

Example for each type attribute

1. Example where we used <ul type = "square">

```
<!DOCTYPE html>
<html>
 <head>
 <title>HTML Unordered
List</title>
 </head>
 <body>
 <ul type = "square">
 <li>Red</li>
 <li>Green</li>
 <li>Orange</li>
 </ul>
 </body
</html>
```

Output

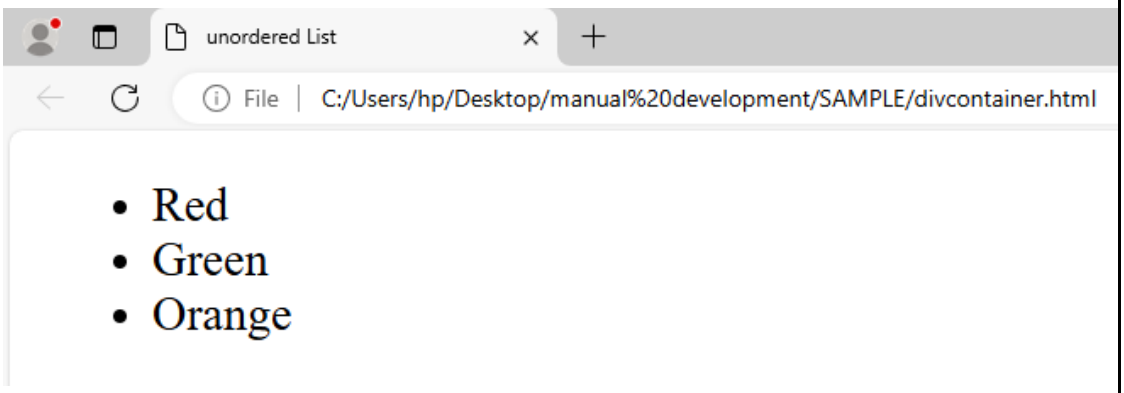


2. Example where we used <ul type = " disc">

```
<!DOCTYPE html>
<html>
 <head>
```

```
<title>HTML Unordered
List</title>
</head>
<body>
<ul type = "disc">
  <li>Red</li>
  <li>Green</li>
  <li>Orange</li>
</ul>
</body>
</html>
```

Output

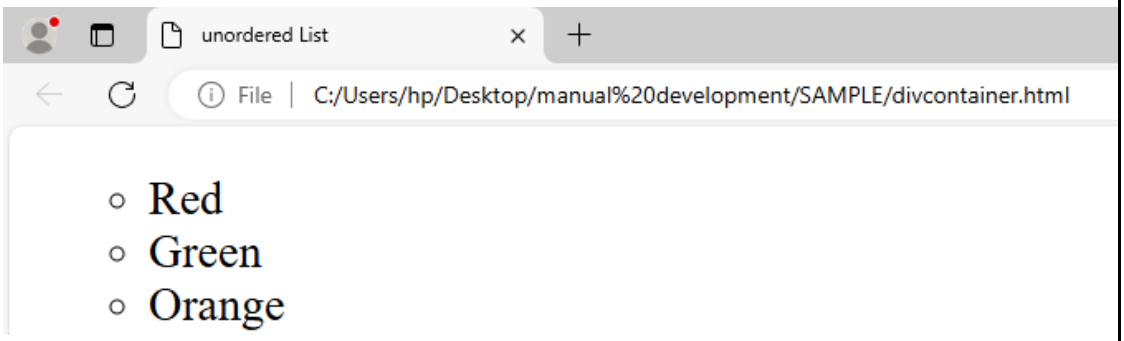


3. Example where we used <ul type = " circle">

```
<!DOCTYPE html>
<html>
<head>
<title>HTML Unordered
List</title>
</head>
<body>
<ul type = "circle">
```

```
  <li>Red</li>
  <li>Green</li>
  <li>Orange</li>
 </ul>
 </body>
</html>
```

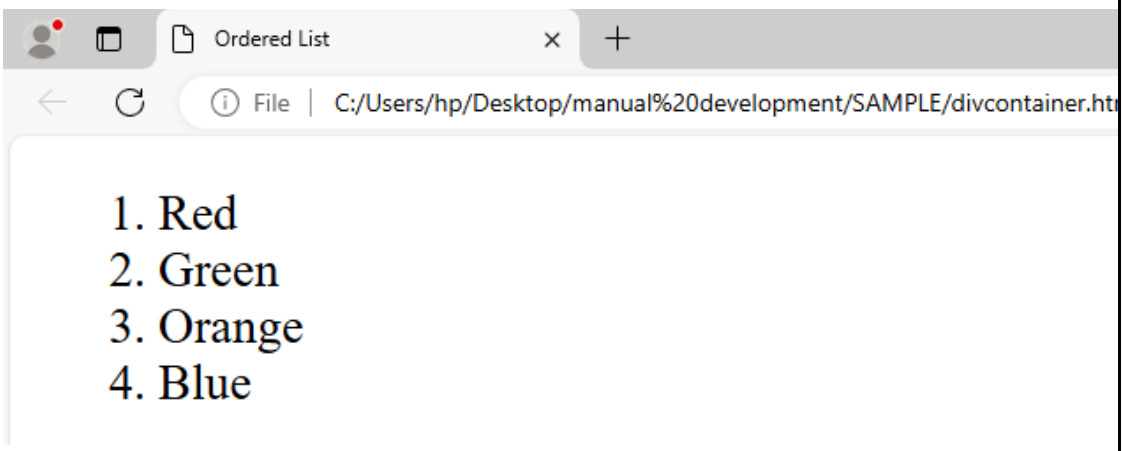
Output



HTML Ordered Lists.

```
<!DOCTYPE html>
<html>
 <head>
 <title>HTML Ordered List</title>
 </head>
<body>
 <ol>
  <li>Red</li>
  <li>Green</li>
  <li>Orange</li>
  <li>Blue</li>
 </ol>
 </body>
</html>
```

Output



Ordered List uses different type attributes for <ol> tag to specify the type of numbering you like. By default, it is a number.

Following are the possible option:

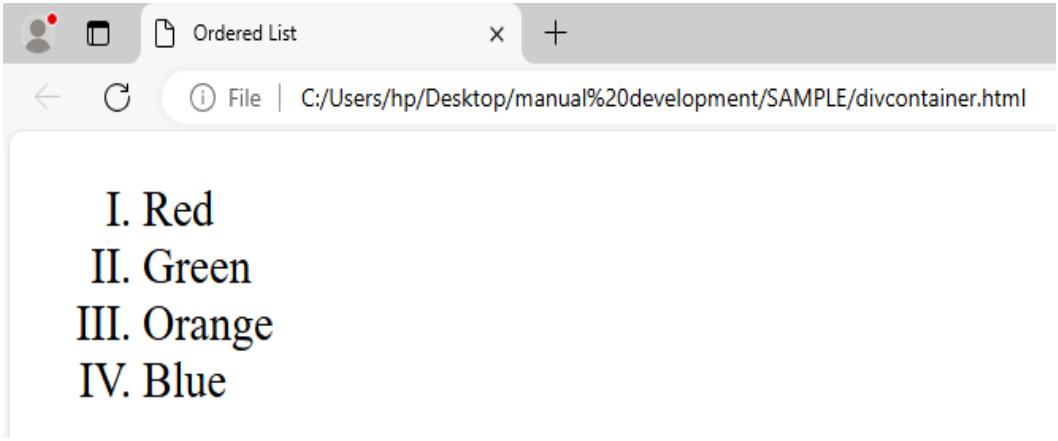
- <ol type = "I"> Upper-Case Numerals.
- <ol type = "i"> Lower-Case Numerals.
- <ol type = "A"> Upper-Case Letters.
- <ol type = "a"> Lower-Case Letters.
- <ol type = "1"> Default-Case Numerals.

Example where we used <ol type = "I">

```
<!DOCTYPE html>
<html>
 <head>
 <title>HTML ordered List</title>
 </head>
 <body>
 <ol type = "I">
   <li>Red</li>
   <li>Green</li>
   <li>Orange</li>
```

```
    <li>Blue</li>
  </ol>
  </body>
</html>
```

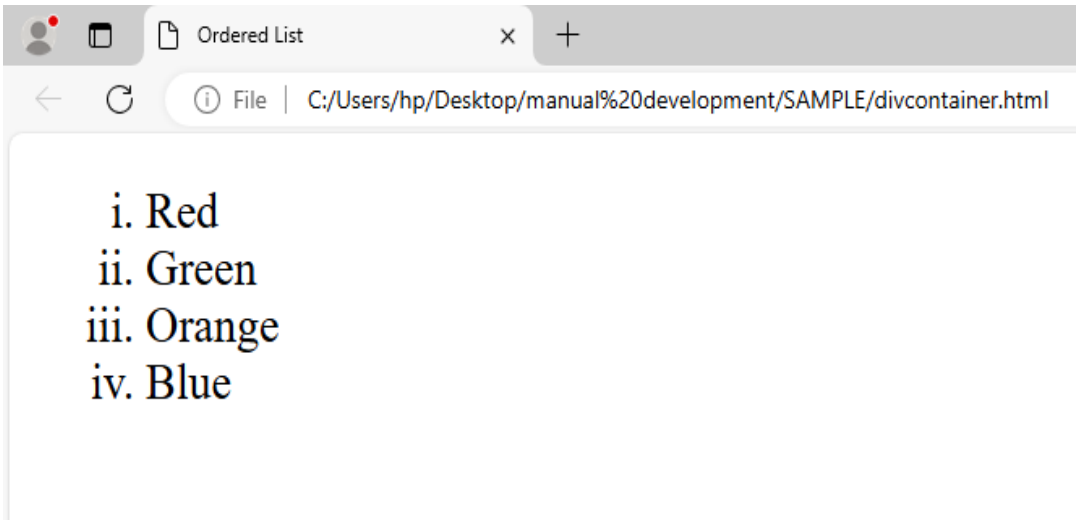
Output



Example where we used <ol type = "i">

```
<!DOCTYPE html>
  <html>
   <head>
   <title>HTML ordered List</title>
   </head>
   <body>
   <ol type = "i">
     <li>Red</li>
     <li>Green</li>
     <li>Orange</li>
     <li>Blue</li>
   </ol>
   </body>
  </html>
```

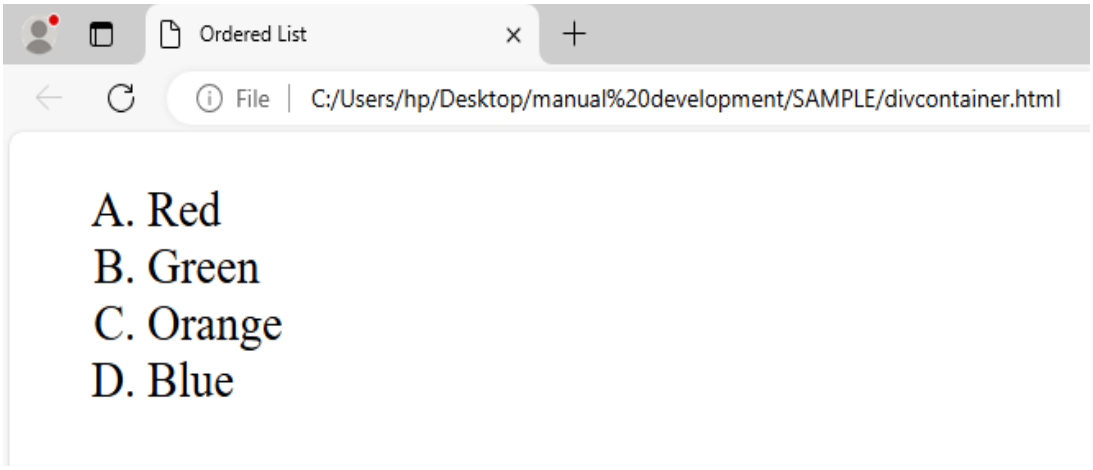
Output



Example where we used <ol type = "A">

```
<!DOCTYPE html>
  <html>
   <head>
   <title>HTML ordered List</title>
   </head>
   <body>
   <ol type = "A">
     <li>Red</li>
     <li>Green</li>
     <li>Orange</li>
     <li>Blue</li>
   </ol>
   </body>
  </html>
```

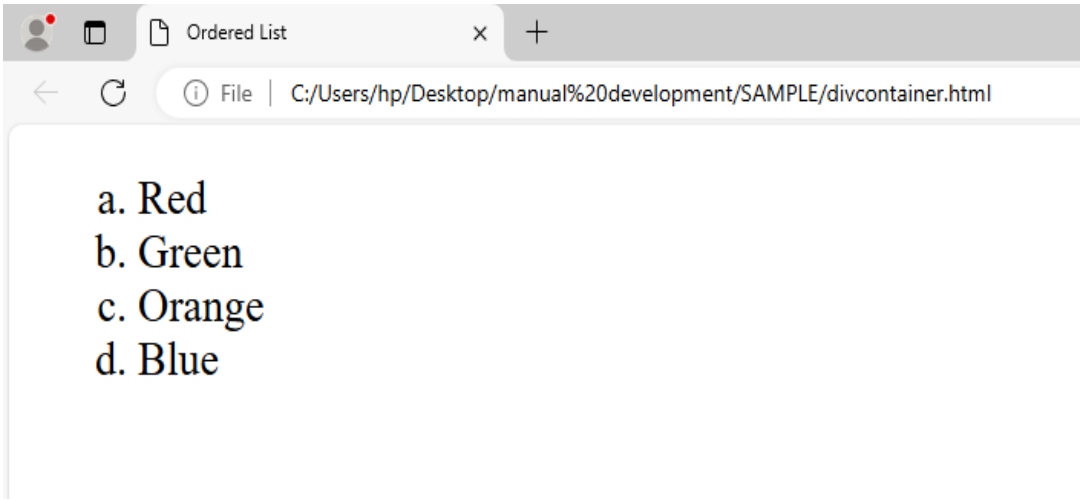
OUTPUT



Example where we used <ol type = "a">

```
<!DOCTYPE html>
<html>
 <head>
 <title>HTML ordered List</title>
 </head>
 <body>
 <ol type = "a">
   <li>Red</li>
   <li>Green</li>
   <li>Orange</li>
   <li>Blue</li>
 </ol>
 </body>
 </html>
```

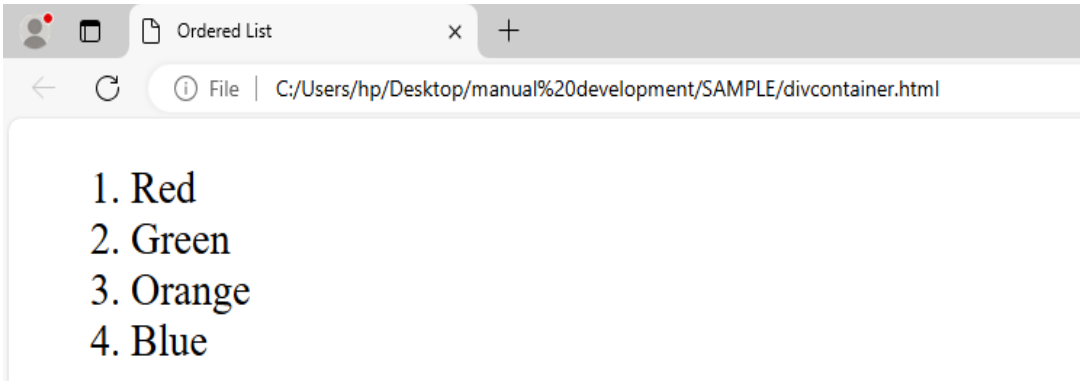
Output



Example where we used <ol type = "1">

```
<!DOCTYPE html>
<html>
<head>
<title>HTML ordered List</title>
</head>
<body>
<ol type = "1">
<li>Red</li>
<li>Green</li>
<li>Orange</li>
<li>Blue</li>
</ol>
</body>
</html>
```

Output



Ordered List use also different start attribute for <ol> tag to specify the starting point of numbering you need.

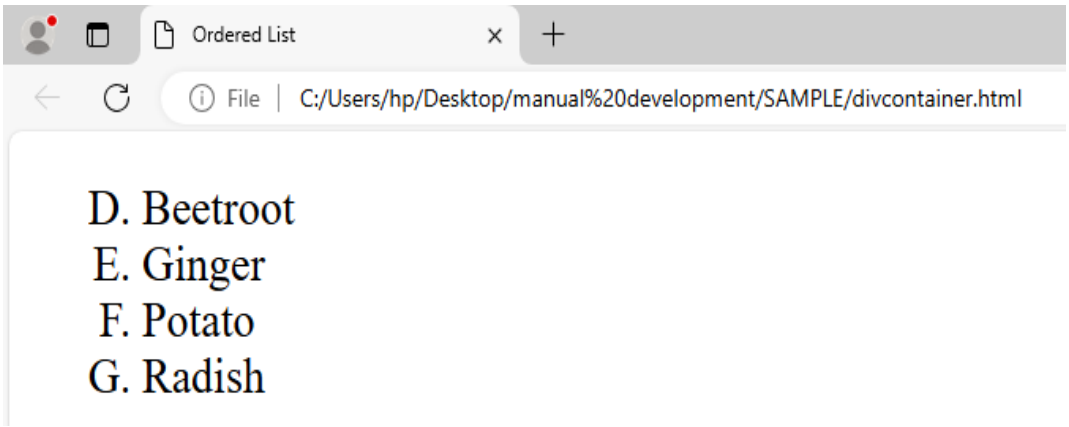
Possible options used “start attribute”

- <ol type = "1" start = "4"> Numerals starts with 4.
- <ol type = "I" start = "4"> Numerals starts with IV.
- <ol type = "i" start = "4"> Numerals starts with iv.
- <ol type = "a" start = "4"> Letters starts with d.
- <ol type = "A" start = "4"> Letters starts with D.

Example where we used <ol type = "i" start = "4">

```
<!DOCTYPE html>  <html>  <head>
<title>HTML ordered List</title>
</head>
<body>
<Ol type = "A" start = "4">
<li>Beetroot</li>
<li>Ginger</li>
<li>Potato</li>
<li>Radish</li>
</Ol>
</body>
</html>
```

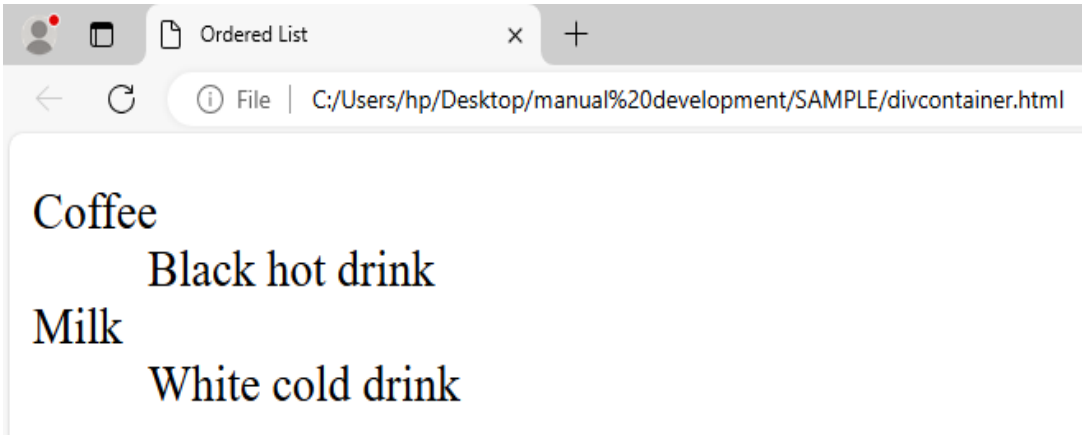
Output



Description List or Definition List <dl></dl>

```
<!DOCTYPE html>
<html>
<body>
<dl>
 <dt>Coffee</dt>
 <dd>Black hot drink</dd>
 <dt>Milk</dt>
 <dd>White cold drink</dd>
</dl>
</body>
</html>
```

Output



6. **Steps to write a Media tags**

To include media content, such as images, audio, and video, in HTML, you can use various tags. Here are the steps to write media tags in HTML:

1. Open a Text Editor

Open a text editor such as Notepad, Sublime Text, Visual Studio Code, or any other code editor.

a) Embed an Image(<img>)

- Use the <img> tag to include images in your HTML document.

- Specify the image source using the src attribute and provide alternative text using the alt attribute.

- Example:

<img src="image.jpg" alt="Description of the image">

b) Embed Audio (<audio>)

- Use the <audio> tag to embed audio content in your HTML document.

- Specify the audio source using the <source> tag within the <audio> tag.

- Example:

<audio controls>

<source src="audio.mp3" type="audio/mpeg">

</audio>

c) Embed Video (<video>)

- Use the <video> tag to embed video content in your HTML document.

- Specify the video source using the <source> tag within the <video> tag.

- Example:

```
html
<video controls>
  <source src="video.mp4" type="video/mp4">
</video>
```

3.Save and test your HTML file.

- Save the HTML file with a .html extension.

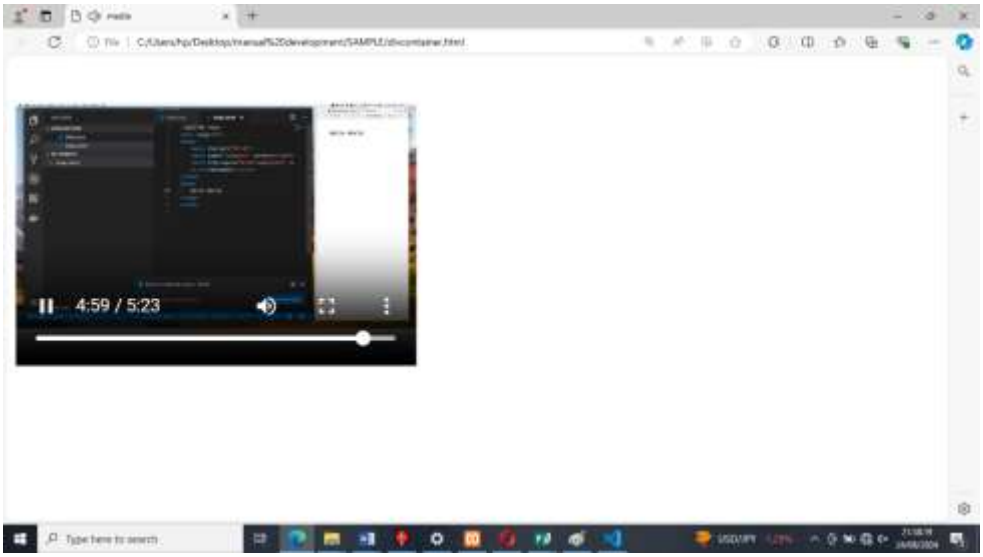
- Open the file in a web browser to see the media content rendered.

Here are some commonly used media tags:

i. HTML Video Tags :<video>

```
<html>
 <head>
 <title>first page</title>
 </head>
 <body>
<video width="320" height="240" controls>
  <source src=" mv_bb.mp4" type="video/mp4">
</video> </body> </html>
```

Output



HTML <video> Autoplay

To start a video automatically, use the autoplay attribute:

```
<html> <head>
 <title>first page</title>
 </head>
 <video width="320" height="240" controls autoplay muted >
  <source src="mv_bb.mp4" type="video/mp4">
</video>
 </body>
 </html>
```

HTML <video> Looping

To repeat video automatically, use the loop attribute:

```
<html>
 <head>
 <title>first page</title>
 </head>
 <video width="320" height="240" controls autoplay muted loop >
  <source src="mv_bb.mp4" type="video/mp4">
```

```
</video>
 </body>
 </html>
```

ii. HTML <audio> Element : <audio>:

The HTML <audio> Element

To play an audio file in HTML, use the <audio> element:

```
<!DOCTYPE html>
<html>
<body>
<audio controls>
  <source src="horse.mp3" type="audio/mpeg">
</audio>
</body>
</html>
```

Note : horse is file name of audio.

iii. HTML <embed> Tag: <embed>

Example

```
<!DOCTYPE html>
<html>
<body>
<h1>The embed element</h1>
<embed type ="video/mp4" src="mv_bb.mp4" width="300" height="200">
</body>
</html>
```

v. HTML <object> Tag:<object>:

```
<!DOCTYPE html><html><body>
<h1>The object element</h1>
```

<object data="mv_bb.mp4" width="300" height="200"></object>

</body>

</html>

vi. <img>: This tag is used to embed and display images on a webpage.

<!DOCTYPE html><html>

<body> <img src="codeimage.jpg" alt=""></body>

</html>

vii. <iframe>

HTML iframes

An HTML iframe is defined with the <iframe> tag:

<!DOCTYPE html>

<html>

<body>

<h2>HTML Iframes example</h2>

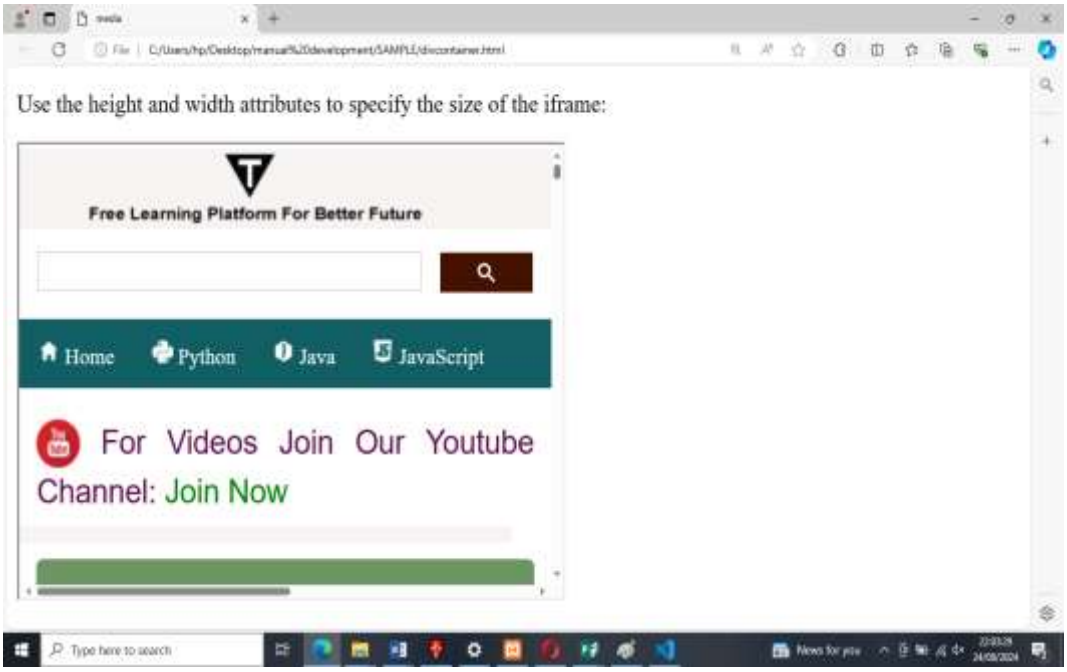
Use the height and width attributes to specify the size of the iframe:

<iframe src="https://www.javatpoint.com/" height="300" width="400"></iframe>

</body>

</html>

Output



The above output will be displayed on when computer is connected

7. Steps to write a frame tag

the steps to write <frame> tags, here they are:

1. Start with the <frameset> tag.

- Begin by writing the opening <frameset> tag to define the start of the frameset.

- Example: <frameset>

2. Add frame elements with the <frame> tag.

- Write the opening <frame> tag to define an individual frame.

- Specify attributes such as src to define the source URL for the frame, name to provide a name for the frame, and other attributes as needed.

- Example:

```
<frameset>
  <frame src="frame1.html" name="frame1">
  <frame src="frame2.html" name="frame2">
</frameset>
```

3. Close the frameset with the closing </frameset> tag.

- Write the closing </frameset> tag to close the frameset.

4. Save and test your HTML file.

- Save the HTML file with a .html extension.

- Open the file in a web browser to see the frames rendered.

Examples:

Create Vertical frames:

```
<!DOCTYPE html>
<html>
<head>
  <title>Frame tag</title>
</head>
 <frameset cols="25%,50%,25%">
   <frame src="frame1.html" >
   <frame src="frame2.html">
   <frame src="frame3.html">
 </frameset>
</html>
```

Create Horizontal frames:

```
<!DOCTYPE html>
<html>
<head>
   <title>Frame tag</title>
</head>
 <frameset rows="30%, 40%, 30%">
   <frame name="top" src="frame1.html" >
   <frame name="main" src="frame2.html">
   <frame name="bottom" src="frame3.html">
 </frameset>
```

</html>

HTML iframes

An HTML iframe is defined with the <iframe> tag:

<!DOCTYPE html>

<html>

<body>

<h2>HTML Iframes example</h2>

Use the height and width attributes to specify the size of the iframe:

<iframe src="https://www.javatpoint.com/" height="300" width="400"></iframe>

</body>

</html>

8.Html Comments

To add comments in HTML, you can use the <! -- --> syntax. Here are the steps to write comments in HTML:

1. Open a Text Editor

Open a text editor such as Notepad, Sublime Text, Visual Studio Code, or any other code editor.

2. Identify the portion of code where you want to add a comment.

- Determine the specific area or line of code that you want to comment on or provide additional information about.

3.Start the comment with <!--.

- Begin the comment by writing <!-- at the beginning of the line or before the code you want to comment on.

4. Add your comment.

- Write the comment text after <! -- and before -->.

- Include any relevant information or notes about the code or its purpose.

- Example: <! -- This is a comment about the following code block -->

5.End the comment with -->.

- Close the comment by writing --> at the end of the line or after the comment text.

6.Save your HTML file.

- Save the HTML file with a .html extension.

The browser does not display comments, but they help document the HTML and add descriptions, reminders, and other notes. <! -- Your comment goes here -->

Example:

```
<html>
 <head>
 <title>first page</title>
 </head>
 <body>
 <!--This is a comment -->
 </body>
 </html>
```

9. Grouping tags (div, span)

To group and style elements in HTML, you can use the and tags. Here are the steps to write grouping tags in HTML:

1. Open a Text Editor:

Open a text editor such as Notepad, Sublime Text, Visual Studio Code, or any other code editor.

2. Start with the tag.

- Begin by writing the opening tag to define the start of a division or container.

- Example:

3. Add content within the tag

- Place the elements or content you want to group or style inside the opening and closing tags.

- Example:

```
<div>
```

```
    <h1>Title</h1>
    <p>Paragraph 1</p>
    <p>Paragraph 2</p>
  </div>
```

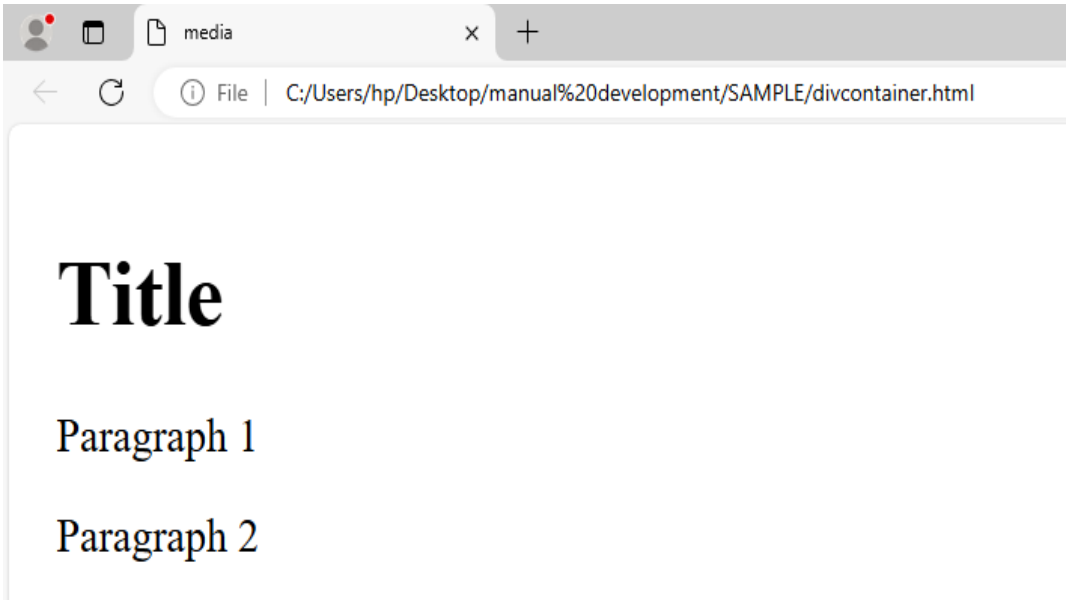
4.Customize the element with CSS.

- Use CSS to style the element and its contents as desired.

- Apply styles such as background color, padding, margin, etc., using CSS properties and values.

- Example:

```
  <style>
    .my-div {
      background-colour: #f2f2f2;
      padding: 10px;
      margin-bottom: 20px;
    }
  </style>
  <div class="my-div">
    <h1>Title</h1>
    <p>Paragraph 1</p>
    <p>Paragraph 2</p>
  </div>
```



For the above code css must locate into head section

5. Start with the tag.

- Begin by writing the opening tag to define the start of an inline container.

- Example:

6.Add content within the tag.

- Place the text or elements you want to group or style inside the opening and closing tags.

- Example:

This is a highlighted text.

7.Customize the element with CSS.

- Use CSS to style the element and its contents as desired.

- Apply styles such as colour, font weight, text decoration, etc., using CSS properties and values.

- Example:

```
<style>
.highlight {
colour: red;
font-weight: bold;
```

```
    text-decoration: underline;
  }
</style>
<p>This is a <span class="highlight">highlighted</span> text.</p>
```

8. Save and test your HTML file.

- Save the HTML file with a .html extension.

- Open the file in a web browser to see the grouping tags and their associated styles.

The element is a block-level element that is often used as a container for other HTML elements.

When used together with some CSS styling, the element can be used to style blocks of code

```
<html>
<body>
<h1>Headline</h1>
<div style="background-color:green; color:white; padding:20px;">
<p>Some paragraph text goes here.</p>
<p>Another paragraph goes here.</p>
</div>
</body>
</html>
```

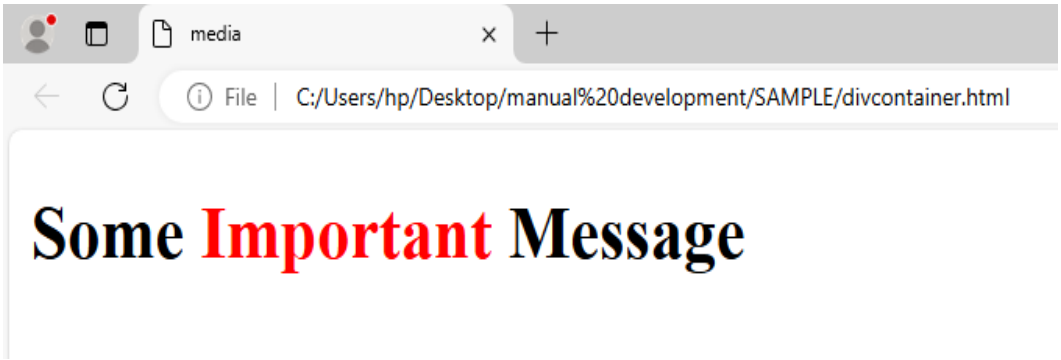
Output

Similarly, the element is an inline element that is often used as a container for some text.

When used together with CSS, the element can be used to style parts of the text:

```
<html>
 <body>
 <h2>Some
 <span style="color:red">Important</span>
 Message</h2>
 </body>
 </html>
```

Output



Theoretical Activity 2.2.7: Description of Hyperlinks

Tasks:

1.In small groups, you are requested to answer the following questions related to the use of hyperlinks.

a. What do you understand by the term hyperlink?
b. What are the default appearance styles of hyperlink on webpage?
c. What are common types of hyperlinks?

3. Provide the answer for the asked questions and write them on papers.
4. Present the findings/answers to the whole class

5. For more clarification, read the key readings 2.2.7. In addition, ask questions where necessary.

Key readings 2.2.7:

Hyperlinks: A hyperlink, also known as a link, is a reference or connection from one web page to another web page, document, or resource. It allows users to navigate between different web pages or sections within a web page by clicking on the linked text or image.

A hyperlink typically consists of two main components:

1. Anchor Text: This is the visible text or image that represents the hyperlink. It is usually underlined or highlighted and is clickable. When a user clicks on the anchor text, it triggers the hyperlink and directs the user to the linked destination.

2. Destination URL: This is the Uniform Resource Locator (URL) or web address that specifies the location of the linked content. It can be an external website, an internal page within the same website, or a specific resource like a document, image, or video.

Hyperlinks can be created using the HTML <a> tag, where the href attribute specifies the destination URL. By clicking on the hyperlink, the user's web browser will load the linked content.

Default link appearance

The default appearance of a link in HTML is typically determined by the browser's default styles or any custom styles applied to the link using CSS. However, here are the common default styles for links:

1. **Text Color:** By default, links are usually displayed in a different colour than regular text to indicate their clickable nature. The default colour is often blue.

2. Underline: Links are commonly underlined by default to provide a visual indication that they are clickable.

3. Hover and Visited States: Links may have different styles when hovered over or when they have been visited. The default behaviour is often to change the colour or add an underline when hovered or visited.

It's important to note that default link styles can vary slightly between different browsers and operating systems. Additionally, web developers often override the

default styles with their own custom CSS styles to match the design of their websites.

To change the default appearance of links, you can use CSS to apply custom styles to the <a> (anchor) elements. For example, you can change the color, remove the underline, or add additional effects like hover or focus states.

Here's an example of CSS code to change the default link appearance:

```
css
a {
  colour: red;
  text-decoration: none;
}

a: hover {
  colour: green;
  text-decoration: underline;
}
```

In the example above, the link color is set to red and the underline is removed for all <a> elements. When hovering over the link, the colour changes to green and an underline is added.

Syntax of hyperlinks:

The syntax of hyperlinks in HTML using the <a> tag is as follows:

****Link Text****

Let's break down the components of the hyperlink syntax:

- <a>: This is the opening tag of the hyperlink element.

- href: This is an attribute of the <a> tag that specifies the destination URL or web address to which the hyperlink will navigate when clicked. The value of the href attribute should be enclosed in double quotes.

- "destination_url": This is the actual URL or web address of the destination page or resource to which the hyperlink points. Replace "destination_url" with the appropriate URL, such as "https://example.com".

- "Link Text" : This is the text or content that will be displayed as the clickable hyperlink. Replace "Link Text" with the desired text or insert an image tag to display an image as the hyperlink.

There are several types of hyperlinks that can be used in HTML to create different linking behaviours and functionalities. Here are some common types of hyperlinks:

1. External Hyperlinks: These hyperlinks point to web pages or resources that are located on a different domain or website than the current page. They typically include the full URL, including the protocol (e.g., "https://") and the domain name. Example:

Visit Example

2.Internal Hyperlinks: These hyperlinks point to different sections or pages within the same website or domain. They are often used for website navigation and allow users to jump to specific sections or content on the same page or different pages of the website.

Example:

Go to Section 1

3. **Anchor Hyperlinks (Bookmark Hyperlink):** These hyperlinks are used to link to specific anchor points within a web page. Anchors are defined using the <a> tag with the name attribute, and the hyperlink uses the # symbol followed by the anchor name to navigate to that specific point on the page.
   Uses text or an image to link to another part of the same webpage.
   Example:

   <h2>Section 1</h2>
   This is the content of section 1.
   Go to Section 1
4. **Text Links:** Basic hyperlinks that consist of clickable text.
   Example:
   Visit Example Website
5. **Image Links:** Hyperlinks applied to images, making the entire image clickable.
   Example
   <img src="image.jpg" alt="Example Image">
   Here are a few examples of hyperlink syntax:

6. **E-mail Hyperlink**
   Allows visitors to send an e-mail message to the displayed e-mail address.
   Click here to send E-mail

Practical Activity 2.2.8: Use of Hyperlinks

Task:

1. Referring to the previous theoretical activity (2.2.7) you are requested to go to the computer lab to develop more than one web page and apply hyperlinks on both webpages. Apply safety precautions
2. Read key reading 2.2.8 and ask clarification where necessary
3. Outline common used Types of hyperlinks.
4. Referring to the outlined types of hyperlinks provided on task3, develop a webpage and use type of hyperlinks as you have outlined them.
5. Present your work to the trainer and whole class

Key readings 2.2.8.

Hyperlinks are one of the most exciting innovations the Web has to offer. They've been a feature of the Web since the beginning, and are what makes the Web a web. Hyperlinks allow us to link documents to other documents or resources, link to specific parts of documents, or make apps available at a web address.
A URL can point to HTML files, text files, images, text documents, video and audio files, or anything else that lives on the Web.

❖ **Steps to follow while creating hyperlinks:**

1. Start with the <a> tag

- Begin by writing the opening <a> tag to define the start of the hyperlink.
  - **Example**: <a>

2.Specify the destination URL

- Use the href attribute to specify the URL or file path that the hyperlink should point to.
  - **Example:**

3. Add link text

- Place the text or content you want to display as the hyperlink between the opening and closing <a> tags.
  - Example: Visit Example

4.Customize the hyperlink (optional)

- Apply additional attributes or styles to the <a> tag as needed.
  - **Examples:**

- (opens link in a new tab or window)
- (applies a CSS class for custom styling)

5. Close the hyperlink with the closing tag

- Write the closing tag to close the hyperlink.

Here's an example of a hyperlink:

Visit Example

In the example above, the hyperlink will display the text "Visit Example" and when clicked, it will navigate to the URL "https://example.com".

6. Save and test your HTML file

- Save the HTML file with a .html extension.
- Open the file in a web browser to see the hyperlink rendered.

❖ **Steps to use hyperlinks**

1. Identify the text or element you want to turn into a hyperlink:

- Determine the specific text or element that you want to make clickable and act as a hyperlink.

2. Wrap the text or element with the <a> tag:

- Begin by writing the opening <a> tag before the text or element.
- Place the closing tag after the text or element.
- Example: <a>Clickable Text

3. Specify the destination URL:

- Add the href attribute to the <a> tag and provide the URL or file path that the hyperlink should point to.
- Example: Clickable Text

4. Customise the hyperlink (optional):

- Apply additional attributes or styles to the <a> tag as needed.
- Examples:
  - <a href="https://example.com" target="_blank"> (opens link in a new tab or window)
  - <a href="https://example.com" class="custom-link"> (applies a CSS class for custom styling)

5. Save and test your HTML file:

- Save the HTML file with a .html extension.
- Open the file in a web browser to see the hyperlink rendered.
- Click on the hyperlink to verify that it navigates to the specified destination URL.

Here's an example of using a hyperlink:

Visit our website for more information.

In the example above, the word "website" is turned into a hyperlink that, when clicked, will navigate to the URL "https://example.com"

Theoretical Activity 2.2.9: Description of HTML graphics

Tasks:

1: In small groups, you are requested to answer the following questions related to the HTML Graphics

a. What do you understand about canvas and svg tags?
b. Where do we apply the canvas and svg tags?

2.Provide the answer for the asked questions and write them on papers.

3.Present the findings/answers to the whole class

4.For more clarification, read the key readings 2.2.9. In addition, ask questions where necessary.

Key readings 2.2.9:

Canvas tags in html.

The HTML <canvas> element is used to draw graphics, on the fly, via JavaScript. The <canvas> element is only a container for graphics. You must use JavaScript to actually draw the graphics. Canvas has several methods for drawing paths, boxes, circles, text, and adding images

Here are some key points about the <canvas> tag:

1. Drawing Surface: The <canvas> tag creates a rectangular area on the web page where you can draw graphics.

2. **JavaScript Interaction:** To draw on the canvas, you need to use JavaScript and the HTML5 Canvas API. With JavaScript code, you can create shapes, paths, images, and apply various transformations and styles.

3. **Dimensions:** The size of the canvas is specified using the width and height attributes of the <canvas> tag. These attributes determine the width and height of the drawing area in pixels.

4. **Context:** Before drawing on the canvas, you need to obtain the rendering context using JavaScript. The rendering context (2d or webgl) provides the methods and properties to perform drawing operations.
5. **Drawing Operations:** The Canvas API offers a wide range of methods to draw on the canvas, including drawing shapes (lines, rectangles, circles), paths, text, images, gradients, and more. You can also apply transformations, set colors, and control transparency.
6. **Animation and Interactivity**: The canvas can be used for creating animations and interactive elements by updating the canvas content and rendering it repeatedly using JavaScript timers or request Animation Frame.
7. **Compatibility:** The **<canvas>** tag is supported in all modern web browsers, including Chrome, Firefox, Safari, and Edge. However, older versions of Internet Explorer may have limited support or require polyfills.

Here's an example of a basic <canvas> tag:

```
<canvas id="myCanvas" width="500" height="300"></canvas>
```

In the example above, a canvas element is created with an id of "myCanvas" and a width of 500 pixels and height of 300 pixels. To draw on this canvas, you would need to use JavaScript code to obtain the rendering context and perform the desired drawing operations.

The <canvas> tag is a powerful tool for creating dynamic and interactive graphics on the web, allowing you to bring visual elements to life and engage users with rich visual experiences.

Structure:

Opening tag: <canvas>

Attributes:

width: Defines the width of the canvas in pixels.

height: Defines the height of the canvas in pixels.

id: Provides a unique identifier for the canvas element.

Content: Empty, as you'll use JavaScript to draw on the canvas.

Closing tag: </canvas>

SVG stands for Scalable Vector Graphics. It is an XML-based markup language used to define and create two-dimensional vector graphics. SVG graphics are resolution-independent, meaning they can be scaled without losing quality, making them ideal for various applications such as icons, illustrations, charts, and more.

Here are some key points about SVG tags:

1. XML-based Markup: SVG graphics are defined using XML-based markup, allowing for precise control over the appearance and behavior of the elements.

2. Vector Graphics: SVG graphics are composed of geometric shapes, paths, text, and other graphical elements that can be scaled, rotated, and transformed without losing quality.

3. Elements and Attributes: SVG uses a set of predefined elements and attributes to define and style the graphical elements. Elements include <rect>, <circle>, <path>, <text>, and more, while attributes control properties like color, size, position, and more.

4. Styling: SVG elements can be styled using CSS or inline attributes. This allows for easy customization and control over the appearance of the graphics.

5. Animation and Interactivity: SVG supports animation and interactivity through CSS animations, JavaScript, and SVG-specific animation elements like <animate> and <animateTransform>. This enables the creation of dynamic and interactive SVG graphics.

6. Integration with HTML: SVG graphics can be embedded directly into an HTML document using the <svg> tag. They can also be referenced from external SVG files using the <object> or <img> tags.

7. Browser Support: SVG is supported in all modern web browsers, including Chrome, Firefox, Safari, and Edge. However, older versions of Internet Explorer may have limited support or require polyfills.

Here's an example of a basic SVG tag:

```
<svg width="500" height="300">
  <rect x="50" y="50" width="200" height="100" fill="blue" />
  <circle cx="300" cy="150" r="50" fill="red" /> </svg>
```

In the example above, an SVG element is created with a width of 500 pixels and height of 300 pixels. Within the <svg> tag, there are two graphical elements: a blue rectangle and a red circle.

SVG provides a powerful and flexible way to create and display vector graphics on the web, allowing for rich visual experiences and responsive design.

Structure:

- **Opening tag: <svg>**
- **Attributes:** Define the size, viewport, and other settings of the SVG element:
  - **ViewBox**: Sets the visible portion of the SVG content.
  - **Width**: Defines the width of the SVG element in pixels, percentages, or other units.
  - **Height:** Defines the height of the SVG element.

- **PreserveAspectRatio**: Controls how the SVG scales to fit the available space.
- **Content:** Holds the actual SVG elements that draw your visuals:
- Shapes like circles, rectangles, lines, and paths.
- Text elements with various styles and fonts.
- Images and other embedded content.
- Styling elements like fill, stroke, and opacity.
- **Closing tag:** </svg>

Application of Canvas and SVG Elements

The <canvas> and <svg> tags are both used for creating graphics on the web, but they are applied in different scenarios based on their capabilities and use cases.

- **Application of CANVAS**

✓ **Games:** Rendering game elements and animations.
✓ **Data Visualization:** Creating custom charts, graphs, or real-time data visualizations.
✓ **Image Processing**: Manipulating images, such as filtering or transforming them.
✓ **Custom Animations:** Drawing and animating graphics that require precise control over each pixel.

- **Application of SVG**

✓ **Icons and Logos:** Creating responsive and scalable icons and logos.
✓ **Diagrams and Charts:** Drawing and interacting with vector-based diagrams, flowcharts, and graphs.
✓ **Illustrations:** Crafting detailed illustrations that need to be scalable without losing quality.
✓ **Interactive Graphics**: Adding interactivity to vector graphics, like hover effects, tooltips, or clickable areas.

Practical Activity 2.2.10: Use of HTML graphics

Task:

1.Referring to the previous theoretical activities (2.2.9) you are requested to go to the computer laboratory and perform the following task about using HTML Graphics. This task should be done individually.
2. Read key reading **2.2.10** and ask clarification where necessary
3. Apply safety precautions and follow the rules governing use of computer laboratory

4.Create a web page which will display a yellow rectangle using canvas tag and blue circle using svg tag.
5.Present your work to the trainer and whole class

Key readings 2.2.10

To write a <canvas> tag in HTML, follow these steps:

1. Start with the <canvas> tag:

- Begin by writing the opening <canvas> tag to define the canvas element.
- **Example: <canvas>**

2. Specify the dimensions of the canvas:

- Use the width and height attributes to set the dimensions of the canvas in pixels.
- Example: <canvas width="500" height="300">

3. Add fallback content (optional):

- The <canvas> tag supports fallback content, which is displayed if the browser does not support the <canvas> element.
- Place the fallback content between the opening and closing <canvas> tags.
- Example: <canvas width="500" height="300">Fallback content...</canvas>

4. Save and test your HTML file:

- Save the HTML file with a .html extension.
- Open the file in a web browser to see the canvas rendered.

Here's an example of a <canvas> tag:

<canvas width="500" height="300"></canvas>

Example:

Let us have an example to draw a rectangle.

Draw a red rectangle on the fly, and show it inside the **<canvas>** element:

```
<!DOCTYPE html>
<html>
<body>
<h1>HTML5 Canvas</h1>

<canvas id="myCanvas" width="300" height="150" style="border:4px solid grey"> Here</canvas>

</body>
</html>
```

Output will be

HTML5 Canvas

In the example above, the canvas element has a width of 500 pixels and a height of 300 pixels. You can use JavaScript and the HTML5 Canvas API to draw and manipulate graphics on this canvas.

Remember that the <canvas> tag provides a drawing surface, but you need to use JavaScript code to interact with it and perform drawing operations.

By following these steps, you can write a <canvas> tag in HTML to create a canvas element where you can dynamically render and manipulate graphics using JavaScript.

Examples:

- **Drawing a simple circle:**

Creating a circle in a web page using the <canvas> tag involves a few simple steps. Below is a step-by-step guide to help you create a circle using the canvas element in HTML and JavaScript.

1. Add the HTML Structure

Start by adding the <canvas> element to your HTML file. This element will serve as the drawing area for your circle.

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Canvas Circle Example</title>
</head>
<body>
    <!-- Canvas Element -->
<canvas id="myCanvas" width="200" height="200"
style="border:1px solid #000000;">
</canvas>
    <script src="script.js"></script>
</body>
</html>
```

2. Set Up the Canvas and Draw the Circle

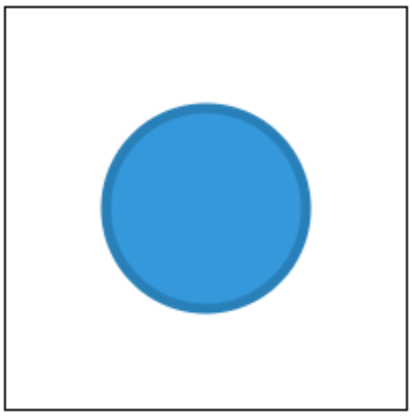
Next, use JavaScript to draw the circle on the canvas. Create a script.js file and add the following code:

```
// Step 1: Get the canvas element
var canvas = document.getElementById('myCanvas');
// Step 2: Get the drawing context
var context = canvas.getContext('2d');
// Step 3: Begin a new path for drawing
context.beginPath();
// Step 4: Define the circle's parameters
// context.arc(x, y, radius, startAngle, endAngle, anticlockwise);
context.arc(100, 100, 50, 0, 2 * Math.PI);
// Step 5: Style the circle (optional)
context.fillStyle = '#3498db';  // Fill color
context.fill();                 // Fill the circle
context.lineWidth = 5;          // Line width
context.strokeStyle = '#2980b9';// Stroke color
context.stroke();               // Draw the circle outline
// Step 6: Close the path (optional for a complete circle)
context.closePath();
```

3. View the Result

When you open the HTML file in a browser, you should see a blue circle with a darker blue border centered within the 200x200 pixel canvas. The circle's size, position, and colors can be adjusted by changing the parameters in the context.arc() method and the styling properties

Result will looks like this:



Steps to write SVG tag:

To write an SVG tag in HTML, follow these steps:

1. Start with the <svg> tag:

- Begin by writing the opening <svg> tag to define the SVG element.
- Example: <svg>

2. Specify the dimensions of the SVG:

- Use the width and height attributes to set the dimensions of the SVG element in pixels or other units.
- Example: <svg width="500" height="300">

3. Add SVG content:

- Within the <svg> tags, you can add various SVG elements to create the desired graphical content.
- SVG elements include shapes (e.g., <rect>, <circle>, <path>), text elements (e.g., <text>, <tspan>), and more.
- Example: <svg width="500" height="300"><rect x="50" y="50" width="200" height="100" fill="blue" /></svg>

4. Customize the SVG:

- You can apply styles to the SVG elements using CSS or inline attributes to control their appearance.
- Example: <svg width="500" height="300"><rect x="50" y="50" width="200" height="100" fill="blue" style="stroke: black; stroke-width: 2px;" /></svg>

5. Save and test your HTML file:

- Save the HTML file with a .html extension.
- Open the file in a web browser to see the SVG rendered.

Here's an example of an SVG tag:

```
<svg width="500" height="300">
  <rect x="50" y="50" width="200" height="100" fill="blue" />
  <circle cx="300" cy="150" r="50" fill="red" />
</svg>
```

In the example above, an SVG element is created with a width of 500 pixels and a height of 300 pixels. Within the <svg> tag, there are two graphical elements: a blue rectangle and a red circle.

By following these steps, you can write an SVG tag in HTML to create and display scalable vector graphics with precise control over their appearance and behaviour.

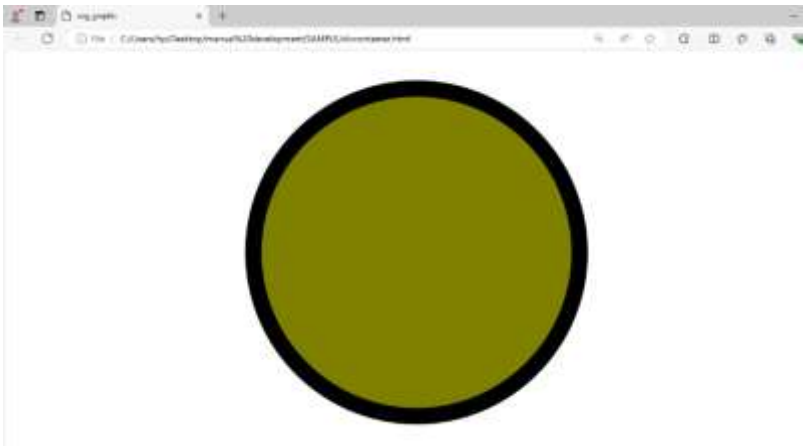
Examples:

- **Creating a simple pie chart**:

In HTML:

```
<svg viewBox="0 0 100 100">
  <circle cx="50" cy="50" r="40" fill="#ff0000" />
  <circle cx="50" cy="50" r="40" fill="#00ff00" stroke="#000" stroke-width="2" fill-opacity="0.5" />
```

```
</svg>
```



- **Implementing a hover animation on an icon:**

JavaScript

```
const svg = document.getElementById('myIcon');

svg.addEventListener('mouseover', () => {
  svg.style.fill = 'red';
});

svg.addEventListener('mouseout', () => {
  svg.style.fill = 'black';
});
```

The possibilities with <svg> are endless! You can create icons, illustrations, charts, animations, and interactive elements that bring your webpages to life. By understanding its capabilities and best practices, you can unlock the full potential of SVG and create stunning and engaging visuals for your audience.

Points to Remember

- There are different types of website each serving different purposes and catering to specific needs.
- There the basic structure of an HTML document includes an opening <html> tag and a closing </html> tag, which contains two main sections: the <head> section (metadata and links to external resources) and the <body> section (content of the page).

Application of learning 2.2.

XYZ Is company which sales different furniture for home use and decoration, in order to competitive on global market wishes to advertise their products through social network platform to attract more customers, after collecting different data from clients they decide to use e-commerce website which will have the following features:

Develop an e-commerce website that includes hyperlinks for navigation, HTML5 Canvas graphics for interactive elements, and a structured HTML layout using various HTML tags and categories.

Requirements:

- **Homepage (index.html):** Introduction and navigation menu.
- **Product Page(products.html):** List of products with descriptions and images.
- **Contact Page(contact.html):** Contact form for customer inquiries.

As Website developer you are hired to help xyz company to have the desired e-commerce website as describe above.

Indicative content 2.2: Managing page layout in HTML

Duration:20 hrs

Theoretical Activity 2.3.1: Description of page layout in HTML

Tasks:

1. In small groups, you are requested to answer the following questions related to the page layout in html
    a. What do you understand about term Page Layout in web development?
    b. What are the basics parts of webpage?
2. Provide the answer for the asked questions and write them on papers.
3. Present the findings/answers to the whole class
4. For more clarification, read the key readings 2.3.1. In addition, ask questions where necessary.

Key readings 2.3.1:

Page layout refers to the arrangement and organisation of elements on a web page. It involves structuring and positioning various components such as headers, navigation menus, content sections, sidebars, footers, and more. A well-designed page layout ensures a logical and visually pleasing structure for users to navigate and interact with the content.

Here are some key considerations for creating an effective page layout:

1. Content Hierarchy: Determine the hierarchy of your content and organise it accordingly. Important elements should be given prominence and positioned prominently, while secondary elements can be placed in less prominent areas.

2. Grid Systems: Use grid systems to create a consistent and balanced layout. Grids help in aligning elements, maintaining visual harmony, and making the page responsive to different screen sizes.

3. Responsive Design: Ensure that your page layout is responsive, meaning it adapts and adjusts to different screen sizes and devices. This allows for optimal viewing and usability on various devices, including desktops, tablets, and mobile phones.

4. White Space: Utilize white space effectively to provide breathing room between elements. White space helps improve readability, focus attention on important content, and create a clean and uncluttered design.

5. Navigation: Place navigation menus and links in easily accessible locations, such as the top of the page or a sidebar. Consistent navigation across pages helps users navigate your website efficiently.

6. Visual Hierarchy: Use visual cues like size, color, typography, and spacing to establish a clear visual hierarchy. This helps users quickly understand the importance and relationship between different elements on the page.

7. Balance and Alignment: Maintain balance and alignment of elements to create a visually pleasing layout. Align elements horizontally and vertically to create a sense of order and structure.

8. Consistency: Maintain consistency in the design and layout across pages within your website. Consistent placement of elements, fonts, colours, and styles helps users navigate and understand your website easily.

9. User-Friendly Design: Consider usability principles when designing your page layout. Ensure that important elements are easily discoverable, clickable areas are large enough, and the overall design is intuitive and user-friendly.

Benefits of a good page layout:

- **Improved user experience:** Users can easily find what they need and navigate the website efficiently.
- **Enhanced brand recognition**: Consistent layout contributes to a cohesive brand identity.
- **Increased engagement:** Users are more likely to stay on your website and interact with your content when it's well-organised and visually appealing.
- **Better conversion rates:** A clear and intuitive layout can lead to higher conversion rates for calls to action and desired user actions.

A web page consists of various parts that work together to create a complete user experience. Here are the key parts of a typical web page:

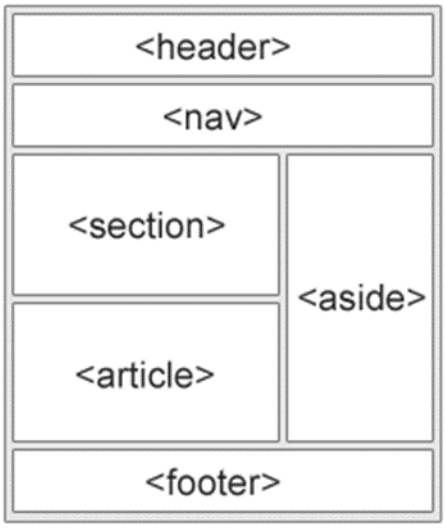
1. Header: The header is located at the top of the webpage and often contains the website's logo, branding elements, and primary navigation menu. It serves as a consistent element across pages, providing users with easy access to essential site features.

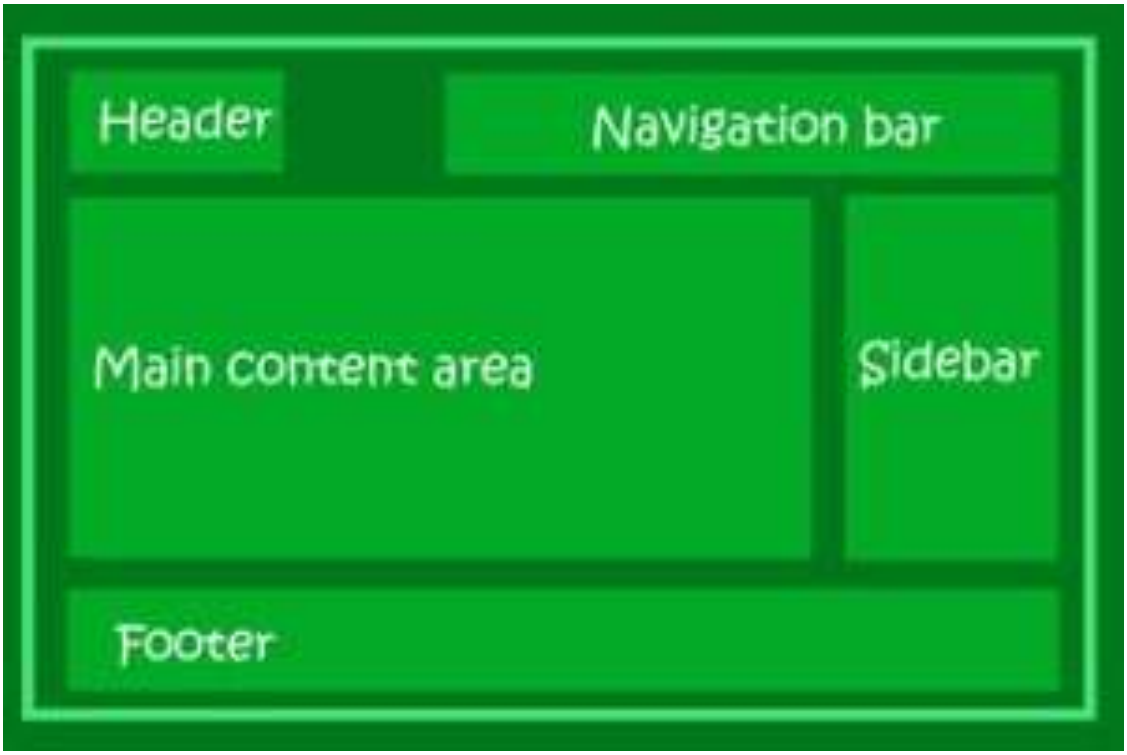
2. Navigation Menu: The navigation menu typically appears in the header or sidebar and contains links to different sections or pages of the website. It helps users navigate and explore the site's content.

3. Content Sections: Content sections make up the main body of the webpage. These sections contain the primary information, text, images, videos, and other media that convey the intended message or provide relevant content to the users. The content sections are organised based on the page's structure and hierarchy.

4. Sidebar: The sidebar is a vertical column that appears alongside the main content sections. It may contain additional navigation elements, secondary information, advertisements, or widgets like search bars, social media links, or recent posts.

5.Footer: The footer is located at the bottom of the webpage and typically contains secondary navigation links, copyright information, contact details, and additional links to important pages or resources. It provides a sense of closure and often appears consistently across the website.





HTML tags to organise page layout

HTML provides several tags that can be used to organize the layout of a webpage. Here are some commonly used HTML tags for structuring the page layout:

1. <header>: Represents the header section of a webpage, typically containing the site's logo, navigation menu, and other introductory elements.

2. <nav>: Defines a container for navigation links, such as a menu or a list of links to different sections of the website.

3. <main>: Encloses the main content of the webpage, excluding headers, footers, and sidebars. It represents the primary content area of the page.

4. <section>: Represents a distinct section of content within a webpage. It can be used to group related content, such as articles, blog posts, or different sections of a product page.

5. <article>: Defines a self-contained piece of content that can be independently distributed or syndicated. It is commonly used for blog posts, news articles, or forum posts.

6. <aside>: Represents content that is tangentially related to the main content, such as sidebars, pull quotes, or advertisements.

7. <footer>: Represents the footer section of a webpage, typically containing copyright information, legal notices, contact details, or links to related pages.

8. : A generic container element used to group and style other elements. It is widely used for layout purposes and provides flexibility in organizing content.

9. : Similar to , is an inline container used to apply styles or manipulate specific sections of text or other inline elements.

10. <table>: Defines a tabular data structure. It can be used to organise data into rows and columns, such as pricing tables or data comparisons.

Practical Activity 2.3.2: Use of html tag to Organize page layout

Tasks:

1. Referring to the previous theoretical activities (2.3.1) you are requested to go to the computer lab to use html tags to organise page layout. This task should be done individually.

2. Read key reading 2.3.2 and ask clarification where is necessary
3. Apply safety precautions
4. Present out the steps to use html tags to organize page layout.
5. Referring to the steps provided on task 3, use html tags to organise page layout.
6 Present your work to the trainer and whole class

Key readings 2.3.2:

To organize the layout of a web page using HTML tags, you can follow these steps:

1. **Understand the Structure:**
   - Start by understanding the basic structure of an HTML document, including the <html>, <head>, and <body> tags.
   - Identify the main sections of your webpage, such as header, navigation, main content, sidebar, and footer.

2. Use Semantic HTML Tags:
   - Utilise semantic HTML tags like <header>, <nav>, <main>, <section>, <article>, and <footer> to define the different parts of your webpage.
   - Semantic tags provide meaning to the content and help search engines and screen readers understand the structure of your page.

3. Divide Content into Sections:
   - Use elements to create divisions or sections within your web page layout.
   - Group related content together within elements to organize and style them collectively.

4. Create a Navigation Menu:

- Use <nav> for your navigation menu to provide links to different sections of your website.
- Structure your navigation links using lists (<ul> and <li>) for better organisation and styling.

5. Arrange Content with Containers:

- Use elements as containers to group and organize related content.
- Apply CSS styling to these containers to control their layout, positioning, and appearance on the webpage.

6. Use Heading Tags for Structure:

- Use heading tags (<h1> to <h6>) to define the hierarchy of your content.
- Headings help organize and structure your content, making it easier for users to navigate and understand.

7. Incorporate Lists and Tables:

- Use <ul>, <ol>, and <li> for lists of items within your content.
- Utilise <table>, <tr>, <th>, and <td> for tabular data to organize information in rows and columns.

8. Embed Multimedia Content:

- Use <img> for images, <video> for videos, and <audio> for audio files to enrich your content.
- Position multimedia elements within appropriate sections of your webpage layout.

9. Implement Forms for Interaction:

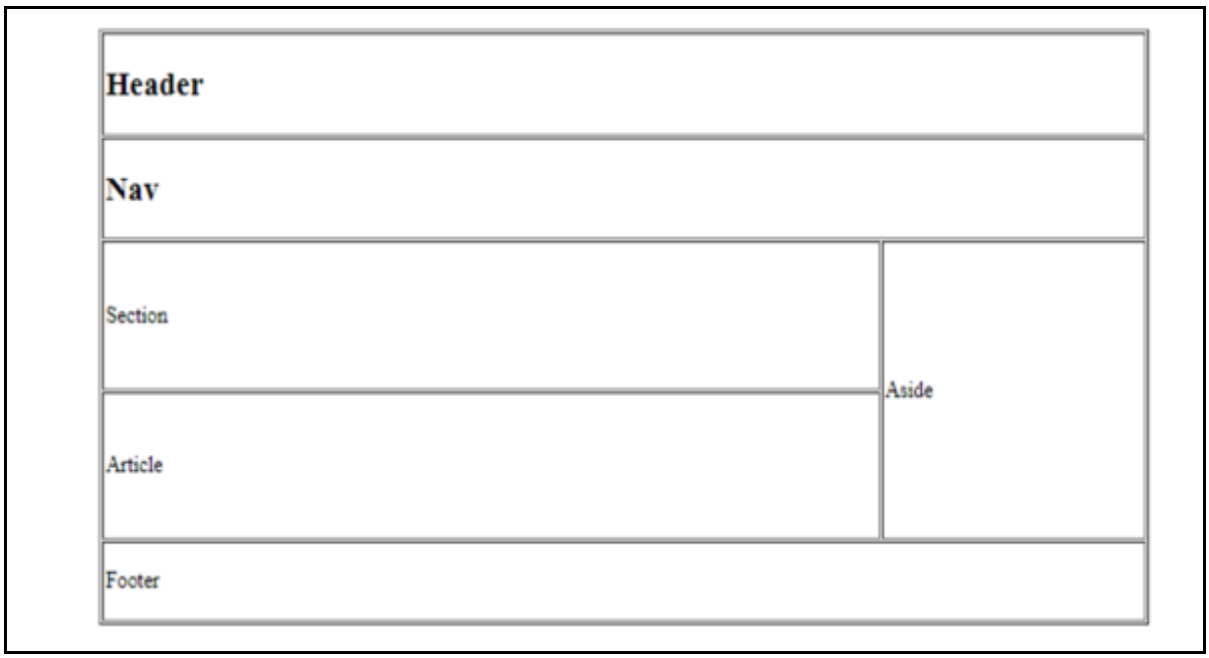
- Use form elements like <form>, <input>, <textarea>, <select>, and <button> to collect user input or feedback.
- Organize forms within your layout to make them easily accessible to users.

Example:

```
<!DOCTYPE html>
<html >
<head>
<title>CSS Template</title>
</head>
<body>
   <table border="1">
      <tr>
```

```
            <header><td colspan="2"><h2>Header</h2></td></header>
        </tr>
        <tr>
            <td colspan="2"><nav>
                <h2>Nav</h2>
            </nav>
            </td>
        </tr>
        <tr>
            <td width="600px" height="100px"> <section>
                Section
             </section></td>
            <td rowspan="2" width="200px" height="100px"> <aside>
            Aside
             </aside>
            </td>
        </tr>
        <tr>
            <td height="100px"> Article</td>
        </tr>
        <tr>
            <td colspan="2"><footer>
                <p>Footer</p>
             </footer></td>
        </tr>
    </table>
    </body>
    </html>
```

Display it by using web browser. A result will be:



Points to Remember

- **Page layout** refers to the arrangement and organisation of elements on a web page. It involves structuring and positioning various components such as headers, navigation menus, content sections, sidebars, footers, and more
- **Benefits of a good page layout:**
  - Improved user experience
  - Enhanced brand recognition
  - Increased engagement
  - Better conversion rates
- **Key parts of a webpage**
  - Header
  - Navigation Menu
  - Content Sections
  - Sidebar
  - Footer

Application of learning 2.3.

You have been hired as a web developer for a local community centre that is planning to launch a new website. The community centre offers various services, including educational programs, fitness classes, community events, and volunteer opportunities. They want a modern, easy-to-navigate website to provide information about their services and events to the community.

You are asked to create an HTML file that includes all the specified sections. Ensure that your HTML structure is well organized. Use appropriate HTML tags to create a clear and organized webpage layout.

Indicative content 2.4: Optimising web page in HTML

Duration: 15hrs

Theoretical Activity 2.4.1: Description of web page optimisation

Tasks:

1. In small groups, you are requested to answer the following questions related to the web page optimisation:
    a. What do you understand about web Optimization?
    b. Explain the benefit of applying SEO on webpage
    c. List the tags used to optimise web page
    d. Describe web accessibility

2. Provide the answer for the asked questions and write them on papers.

3. Present the findings/answers to the whole class

4. For more clarification, read the key readings 2.4.1. In addition, ask questions where necessary.

Key readings 2.4.1:

Definitions:

Website optimization is the process of improving your website's performance across various areas, such as web traffic, conversion rates, and usability.

Search Engine Optimization (SEO) refers to the practice of optimizing a website to improve its visibility and ranking on search engine results pages (SERPs) such as **Google, Bing**, and **Yahoo.**

The goal of SEO is to attract organic (non-paid) traffic to the website by ensuring it appears higher in search results when users search for relevant keywords and phrases.

SERP

The Search Engine Results Page **(SERP)** is the page that a search engine returns after a user submits a search query.

- **Importance of SEO:**

Increased Visibility: Higher rankings in search results lead to more visibility and organic traffic.

Cost-Effective: Compared to paid advertising, SEO provides a cost-effective way to attract long-term, sustainable traffic.

Credibility and Trust: Websites that appear at the top of search results are often perceived as more credible and trustworthy.

User Experience: Good SEO practices improve the overall user experience, making the website more user-friendly and efficient.

Competitive Advantage: Effective SEO can give a website an edge over competitors by attracting more visitors and potential customers.

❖ **Use of html tags to optimize website**

HTML tags have become so essential that no website can compete in today's search results if they ignore HTML tags.

The most important HTML tags for SEO are the following:

title tag, meta description tag, headings (H1 — H6), HTML5 semantic tags, Alt attribute, Open Graph tags, robots tag, canonical tag, href tag, and Schema markup.

HTML tags for Search Engine Optimization

HTML SEO tags are bits of code that can be used to describe content to search engines.

These are the HTML tags still crucial for your website SEO:

1. **Title tag**

In HTML, a title tag looks like this:

```
<title>Your Title Goes Here</title>
```

While in SERPs, a title tag looks like this:

Tips for Title Tag Optimization:

Title tag optimization not only enhances your website's SEO but also its usability and conversions.

The following are some tips to optimize your website's title tag:

- Brand Your Title Tag:
- Enter Keywords Toward the Beginning
- Use Key Phrases Wherever Possible
- Use No More than Two Keywords
- Use Unique Page Titles
- Use Modifier

Example of optimized HTML TITLE TAG

```
<title>How to Get 5 of the Most Valuable SERP Features</title>
```

```
<title>Our Web Development Projects - Case Studies & Success Stories | DevPortfolio</title>
```

Primary Keyword: "Web Development Projects"

Modifiers: "Case Studies," "Success Stories"

Brand Name: "DevPortfolio"

These examples demonstrate how to incorporate primary keywords, modifiers, and brand names effectively into title tags to improve SEO, click-through rates, and user experience.

2. Headings (H1 — H6)

In simple terms, header tags are headers in your page. Use headers to show the flow of your content and break up large chunks of text so that it's more digestible for readers. Headers also highlight the important aspects of your content.

While header tags do not directly influence search rankings, they do serve an important function for SEO and your site visitors: Headers make your content easy to read and engage with. And, for search engine bots, they also provide vital context around the keywords on the page.

It makes sense when you think of how readers and search engines interact with your page. The header is the first thing that gets the reader's attention. It tells them what to expect when they click on a link or visit your web page.

At the same time, search bots use it to understand the primary keyword for the page. This can affect where your content is displayed in search results.

Tips to improve your SEO with header tags

01. Use one H1 header tag per page
02. Stick to the traditional header hierarchy
03. Match search intent to headers
04. Avoid keyword stuffing

05. Use headers to break up text

Here's an example of a webpage with optimized heading tags (H1 to H3) for an e-commerce product page selling running shoes. Each heading tag is used to structure the content logically and includes relevant keywords to improve SEO.

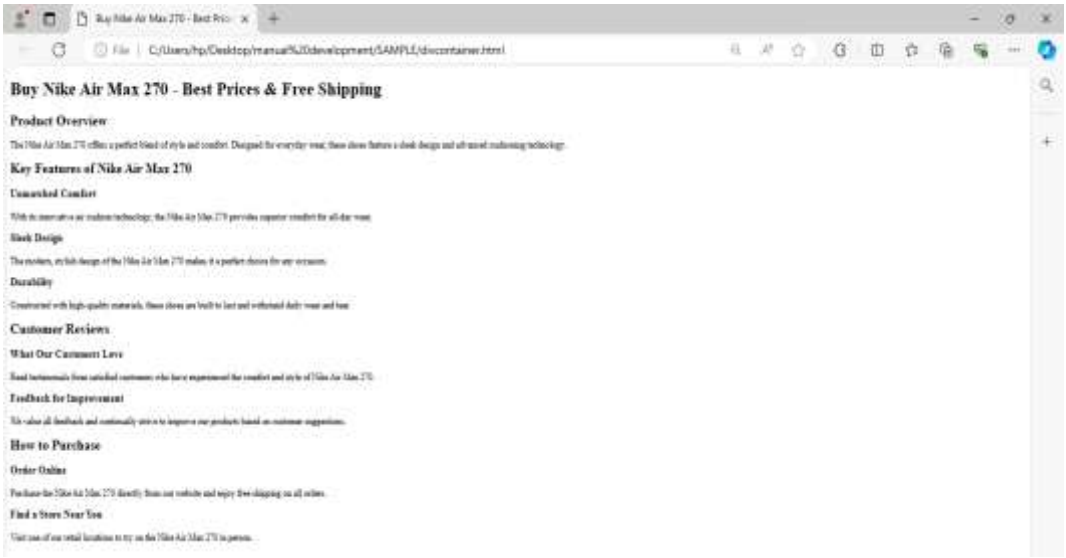
Example: Optimized Heading Tags for an E-Commerce Product Page

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Buy Nike Air Max 270 - Best Prices & Free Shipping | ShoeStore</title>
</head>
<body>
  <!-- Main Heading -->
  <h1>Buy Nike Air Max 270 - Best Prices & Free Shipping</h1>

  <!-- Section 1: Product Overview -->
  <section>
    <h2>Product Overview</h2>
    <p>The Nike Air Max 270 offers a perfect blend of style and comfort. Designed for everyday wear, these shoes feature a sleek design and advanced cushioning technology.</p>
  </section>

  <!-- Section 2: Features -->
  <section>
    <h2>Key Features of Nike Air Max 270</h2>

    <!-- Subsection: Comfort -->
    <div>
      <h3>Unmatched Comfort</h3>
      <p>With its innovative air cushion technology, the Nike Air Max 270 provides superior comfort for all-day wear.</p>
    </div>

    <!-- Subsection: Design -->
    <div>
      <h3>Sleek Design</h3>
      <p>The modern, stylish design of the Nike Air Max 270 makes it a perfect choice for any occasion.</p>
```

```
    </div>

    <!-- Subsection: Durability -->
    <div>
      <h3>Durability</h3>
      <p>Constructed with high-quality materials, these shoes are built to last and withstand daily wear and tear.</p>
    </div>
  </section>

  <!-- Section 3: Customer Reviews -->
  <section>
    <h2>Customer Reviews</h2>

    <!-- Subsection: Positive Reviews -->
    <div>
      <h3>What Our Customers Love</h3>
      <p>Read testimonials from satisfied customers who have experienced the comfort and style of Nike Air Max 270.</p>
    </div>

    <!-- Subsection: Constructive Feedback -->
    <div>
      <h3>Feedback for Improvement</h3>
      <p>We value all feedback and continually strive to improve our products based on customer suggestions.</p>
    </div>
  </section>

  <!-- Section 4: Purchase Information -->
  <section>
    <h2>How to Purchase</h2>

    <!-- Subsection: Online Purchase -->
    <div>
      <h3>Order Online</h3>
      <p>Purchase the Nike Air Max 270 directly from our website and enjoy free shipping on all orders.</p>
    </div>
```

```
    <!-- Subsection: Store Locations -->
    <div>
      <h3>Find a Store Near You</h3>
      <p>Visit one of our retail locations to try on the Nike Air Max 270 in person.
</p>
    </div>   </section></body></html>
```

Output



Explanation:

H1 Tag: The main heading includes the primary keyword "Nike Air Max 270" along with modifiers such as "Buy," "Best Prices," and "Free Shipping" to attract both search engines and users.

H2 Tags: These headings break down the content into key sections like "Product Overview," "Key Features of Nike Air Max 270," "Customer Reviews," and "How to Purchase." This helps with readability and SEO by organizing the content logically.

H3 Tags: These subheadings further categorize the content within each section, focusing on specific aspects such as "Unmatched Comfort," "Sleek Design," "What Our Customers Love," and "Order Online." This structure makes it easy for users to find relevant information quickly and also helps search engines understand the page's content better.

By using optimized heading tags, you can improve the SEO of your webpage while also enhancing the user experience by providing a clear and organized content structure.

3. **Meta Tag**

Meta tags are HTML tags that provide information about a webpage's content to search engines and users.

They play a crucial role in influencing how a website appears in search results and can impact click-through rate (CTR).

Meta tags are usually added in the <head> section of your HTML.

Here's one example:

<!DOCTYPE html>

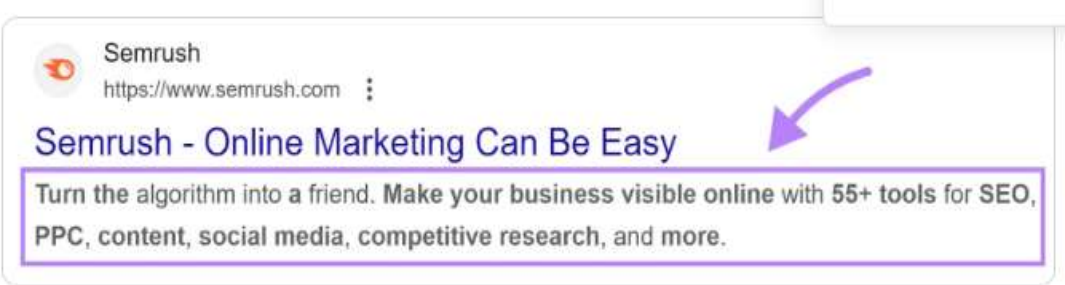
<html>

<head>

<meta name="description" content="Turn the algorithm into a friend. Make your business visible online with 55+ tools for SEO, PPC, content, social media, competitive research, and more.">

</head></html>

In the example above, a meta tag called "meta description" is used. In search results, that will look like this:



Here are some tips to help you craft effective meta descriptions:

- Keep them under 120 characters
- Use sentence case
- Only include your target keyword where it makes sense; avoid keyword stuffing
- Be accurate, descriptive, and concise
- Match search intent
- Avoid duplicate meta descriptions on your website

Example of how to optimize Meta tag

You can optimize:

- **Meta Description:**

  Provides a concise and compelling description that includes relevant keywords and encourages clicks.

  <meta name="description" content="Shop the latest Nike Air Max 270 running shoes. Enjoy the best prices and free shipping on all orders at ShoeStore. Find your perfect pair today!">
- **Meta Robots Tag:**

  Ensures that search engines index the page and follow the links.

```
<meta name="robots" content="index, follow">
```

Benefits of Optimized Meta Tags:

- **Improved Search Engine Rankings:** Properly optimized meta tags can help improve your webpage's rankings on search engines.

- **Higher Click-Through Rates (CTR):** Compelling and relevant meta descriptions can attract more clicks from search engine results pages (SERPs).
- **Better Social Media Engagement:** Open Graph and Twitter Card tags ensure that your content looks appealing when shared on social media platforms, leading to higher engagement.

  By optimizing these meta tags, you can enhance your webpage's visibility, attract more organic traffic, and improve user engagement.

4. **Image alt**

   Image Alt Also called alt tags and alt descriptions, alt text is the written copy that appears in place of an image on a webpage if the image fails to load on a user's screen.

   While optimizing website using image alt tag, Provide a clear, concise description of the image. Avoid keyword stuffing and make sure the description is relevant to the image content.

   <img src="nike-air-max-270.jpg" alt="Nike Air Max 270 running shoes in black and white color">

5. **HTML Semantic Tags**

   A semantic element clearly describes its meaning to both the browser and the developer. Examples of non-semantic elements: and - Tells nothing about its content. Examples of semantic elements: <form> , <table> , and <article> - Clearly defines its content.

   HTML semantic tags are essential for creating meaningful and well-structured web pages. They enhance SEO, accessibility, and code readability, making your website more user-friendly and easier to maintain. Using these tags properly ensures that both humans and machines can understand and navigate your web content effectively.

❖ **Web accessibility**

web accessibility means that websites, tools, and technologies are designed and developed so that people with disabilities can use them. More specifically, people can: perceive, understand, navigate, and interact with the Web.

Accessibility means creating a website that provides a good experience for everyone, regardless of how they must access the internet. When you ensure a good experience for users with disabilities, you can rest assured that your overall website experience and quality will improve.

For example, users with low vision or other visual impairments can increase font size or change color schemes to their preferences. Multiple navigation options, including keyboard navigation, provide users with multiple ways in which to navigate your site. Both practices ensure users have equal access to your content

❖ **Html Role in Web Accessibility**

- HTML (Hypertext Markup Language) plays a crucial role in web accessibility by providing the structural foundation that ensures content is accessible to all users, including those with disabilities.
- HTML provides the foundational structure needed to create accessible web content. By using **semantic HTML**, **ARIA** attributes, proper form labelling, alternative text for images, and ensuring keyboard accessibility, developers can create websites that are more inclusive and usable for everyone, including people with disabilities. This not only enhances the user experience but also ensures compliance with accessibility standards and regulations.

❖ **Test and standard validation**

- Web accessibility ensures that websites and web applications can be used by people with various disabilities. Testing and standard validation are crucial steps in making sure web content is accessible

Test and Standard Validation in Web Accessibility

1. Manual Testing

Manual testing involves human testers evaluating a website for accessibility issues. This can include:

Screen Reader Testing: Using screen readers like JAWS, NVDA, or VoiceOver to navigate the website and ensure content is readable and navigable.

Keyboard Navigation Testing: Ensuring all functionality is accessible using a keyboard alone, without a mouse.

Color Contrast Testing: Checking color contrast ratios to ensure text is readable for users with visual impairments.

Content Structure Testing: Reviewing the use of headings, lists, and other semantic elements to ensure a logical structure.

Tools for Manual Testing:

WAVE: A web accessibility evaluation tool that provides visual feedback about the accessibility of web content.

Axe DevTools: A browser extension that helps identify and resolve accessibility issues.

Chrome DevTools Accessibility: Built-in tools in Chrome for inspecting accessibility properties.

1. **Automated Testing**

Automated tools can quickly scan a website and identify many accessibility issues. However, they cannot catch everything, so they should be used in conjunction with manual testing.

Popular Automated Testing Tools:

Lighthouse: An open-source tool included in Chrome DevTools for auditing web pages for accessibility.

Axe: A comprehensive accessibility testing engine that integrates with various development tools.

Tenon: A web-based tool that checks for compliance with accessibility standards.

2. **User Testing**

Involving users with disabilities in the testing process is invaluable. This real-world testing provides insights into how actual users interact with your website and can uncover issues that other testing methods might miss.

Types of User Testing:

Observational Testing: Watching users as they interact with your website.

Feedback Sessions: Gathering feedback from users about their experience and any difficulties they encountered.

Example: Accessibility Testing Workflow

1. **Run Automated Tests:**
   - Use Lighthouse in Chrome DevTools to get an initial accessibility report.
   - Scan your site with Axe DevTools for more detailed findings.
2. **Conduct Manual Tests:**
   - Navigate your site using only the keyboard to ensure all interactive elements are accessible.
   - Use a screen reader to navigate through your site and ensure all content is properly announced.
3. **Perform User Testing:**
   - Recruit users with disabilities to test your site and provide feedback.
   - Observe how these users interact with your site and note any difficulties.
4. **Validate Against Standards:**
   - Use the W3C Markup Validation Service to check your HTML for compliance.
   - Review your site against WCAG guidelines to ensure all criteria are met.
5. **Implement Improvements:**
   - Address the issues identified in testing and validation.
   - Re-test to ensure the issues are resolved and no new issues have been introduced.

Practical Activity 2.4.2: Use html to Optimize webpage

Notes to the trainer

1. Referring to the previous theoretical activities (2.4.1) you are requested to go to the computer lab to use html tags to optimize a webpage. This task should be done individually.

2. Apply safety precautions
3. Present out the steps to use html tags to optimize a webpage.
4. Referring to the steps provided on task 3, use html tags to optimize webpage.
5. Present your work to the trainer and whole class
6. Read key reading 2.4.2 and ask clarification where is necessary

Key readings 2.4.2:

Optimization may include such tasks as: minimizing the file size of images and files; adding appropriate META description tags; using valid HTML code; limiting the number of redirects; and making sure the page loads quickly. All of these tasks will help to ensure a smooth user experience.

When optimizing a webpage in HTML, you'll want to focus on improving both the front-end and back-end aspects of the webpage to enhance performance, accessibility, and search engine rankings, here are some steps to follow:

1. Optimize the HTML Structure

Use Semantic HTML:

Organize content using semantic tags like <header>, <nav>, <main>, <section>, <article>, <footer>, etc.

This improves accessibility and SEO by clearly defining the purpose of different parts of your content.

Clean Up the Code:

Remove any unnecessary tags, duplicate code, or inline styles/scripts.

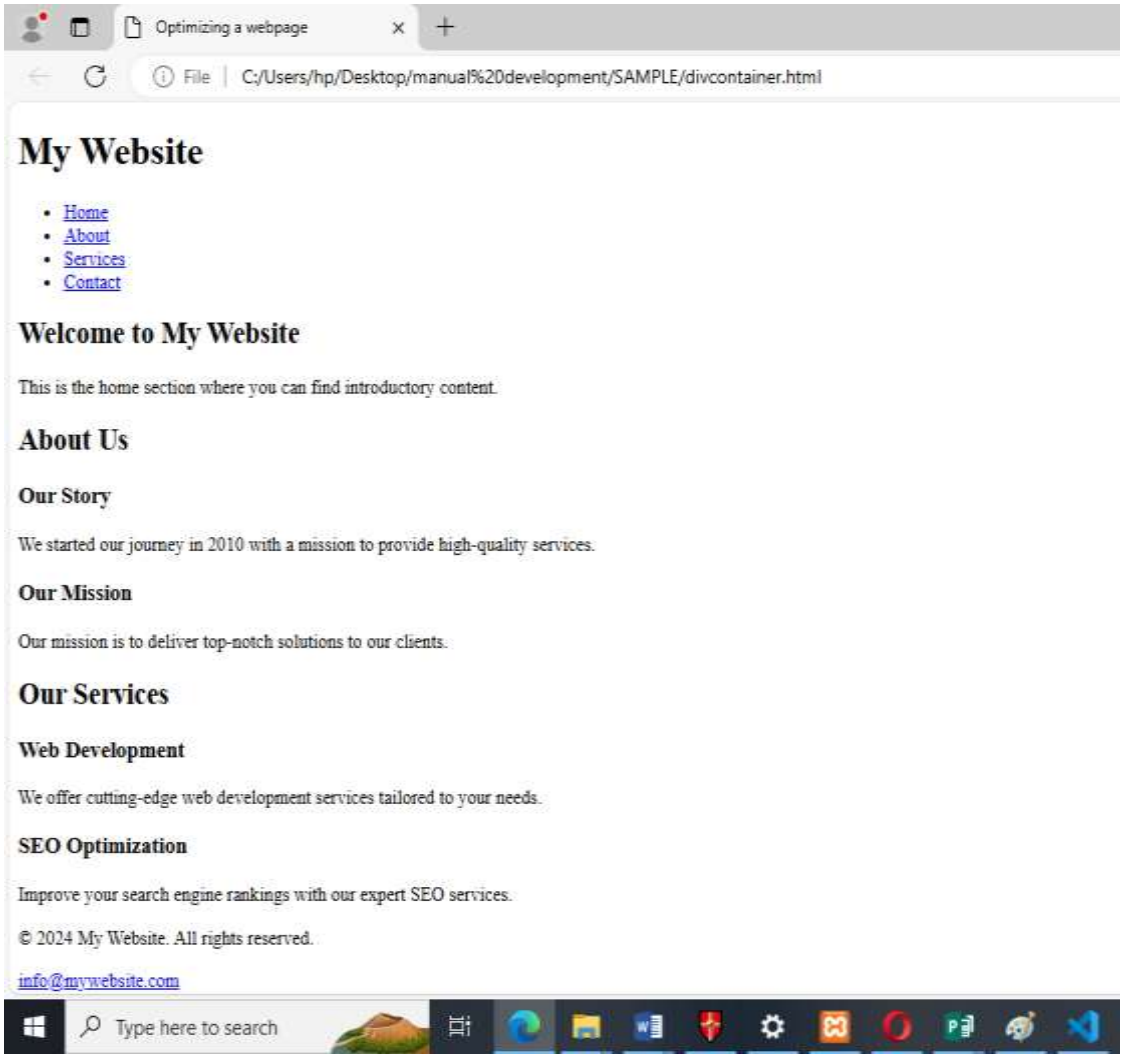
Ensure the HTML is well-structured and easy to read, even after minification.

Example:

```
<!DOCTYPE html>
<html lang="en"><head> <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <meta name="description" content="An example of using semantic HTML
tags to organize webpage content.">
  <title>Optimizing a webpage</title>
  <link rel="stylesheet" href="styles.css">
</head>
<body>
  <!-- Header Section -->
  <header>
    <h1>My Website</h1>
    <nav>
      <ul>
        <li><a href="#home">Home</a></li>
        <li><a href="#about">About</a></li>
        <li><a href="#services">Services</a></li>
        <li><a href="#contact">Contact</a></li>
      </ul>
    </nav>
  </header>
```

```
        <!-- Main Content -->
        <main>
          <!-- Hero Section -->
          <section id="home">
            <h2>Welcome to My Website</h2>
            <p>This is the home section where you can find introductory content.</p>
          </section>
          <! -- About Section -->
          <section id="about">
            <h2>About Us</h2>
            <article>
              <h3>Our Story</h3>
          <p>We started our journey in 2010 with a mission to provide high-quality services.
</p>
            </article><article>
           <h3>Our Mission</h3>
              <p>Our mission is to deliver top-notch solutions to our clients. </p>
            </article> </section>
          <! -- Services Section -->
          <section id="services">
            <h2>Our Services</h2>
            <article>
              <h3>Web Development</h3>
              <p>We offer cutting-edge web development services tailored to your needs.
</p>
            </article>
            <article>
              <h3>SEO Optimization</h3>
              <p>Improve your search engine rankings with our expert SEO services. </p>
            </article>   </section> </main>

        <! -- Footer Section -->
        <footer>
          <p>&copy; 2024 My Website. All rights reserved. </p>
          <p><a href="mailto:info@mywebsite.com">info@mywebsite.com</a></p>
        </footer></body></html>
```

Output



2. Minify HTML

Remove Unnecessary Characters:

Minify your HTML by eliminating unnecessary white spaces, comments, and line breaks to reduce file size.

Use tools like HTMLMinifier to automate this process.

3. Optimize Images

Choose the Right Image Formats:

Use modern formats like WebP for better compression.

Stick to PNG for images with transparency and JPEG for photographs.

Resize Images:

Ensure that images are no larger than they need to be in terms of dimensions.

Compress Images:

Use tools like TinyPNG or ImageOptim to compress images without losing noticeable quality.

Use srcset:

Implement the srcset attribute in <img> tags to provide different image sizes for different devices, ensuring faster load times on smaller screens.

Lazy Load Images:

Use lazy loading to defer the loading of images that are not in the initial viewport.

```
<img src="image.jpg" alt="Description of image" loading="lazy">
```

loading="lazy": This attribute tells the browser to load the image only when it is about to be displayed in the viewport

4. Improve Page Load Speed

Minimize HTTP Requests:

Combine CSS and JavaScript files where possible to reduce the number of requests.

Inline critical CSS to reduce the time to first render.

Enable Gzip Compression:

Compress HTML, CSS, and JavaScript files using Gzip to reduce the size of data transferred.

Leverage Browser Caching:

Set expiration dates on resources so that the browser can cache them and reduce load times on subsequent visits.

Reduce Server Response Time:

Choose a reliable hosting provider, and optimize server-side code and database queries.

5. Use Meta Tags Effectively

META Description:

Write concise, keyword-rich META descriptions that summarize the page content.

Viewport Meta Tag:

Use <meta name="viewport" content="width=device-width, initial-scale=1.0"> to ensure your page is responsive and scales correctly on mobile devices.

Title Tag:

Use descriptive and unique titles for each page to improve SEO and user experience.

6. Optimize CSS and JavaScript

Minify CSS and JavaScript:

Use tools like CSSNano and UglifyJS to minify CSS and JavaScript files, removing unnecessary spaces, comments, and code.

Load JavaScript Asynchronously:

Use the async or defer attributes on <script> tags to prevent blocking the HTML rendering.

Defer Non-Critical JavaScript:

Load non-essential scripts after the main content has been loaded.

Optimize CSS Delivery:

Inline critical CSS to reduce render-blocking and load non-critical CSS asynchronously.

7. Reduce Redirects

Direct Linking:

Avoid unnecessary redirects by linking directly to the final destination URLs.

Clean Up Redirect Chains:
Eliminate redirect chains (e.g., A redirects to B, B redirects to C) as they slow down page load time.

8. Implement Responsive Design

Use CSS Media Queries:
Implement media queries in your CSS to ensure the webpage adapts to different screen sizes and devices.
Flexible Images and Videos:
Use fluid grid layouts and ensure images and videos scale according to the screen size.

9. Improve Accessibility

Alt Text for Images:
Ensure all images have descriptive alt attributes for accessibility and better SEO.
ARIA Landmarks:
Use ARIA (Accessible Rich Internet Applications) landmarks and roles to enhance accessibility for screen readers.
Keyboard Navigation:
Ensure the webpage is fully navigable via keyboard, which benefits users with disabilities.

10. Validate HTML and CSS

Use Validators:
Validate your HTML and CSS using the W3C Markup Validation Service to ensure they meet web standards and have no errors.
Cross-Browser Compatibility:
Test your webpage on different browsers to ensure consistent performance and appearance.

11. Monitor and Test Performance

Use Performance Testing Tools:
Regularly check your webpage's performance using tools like Google PageSpeed Insights, Lighthouse, or GTmetrix.
A/B Testing:
Implement A/B testing (also known as split testing) to experiment with different optimizations and choose the best performing version.

Points to Remember

- Web optimization involves improving various aspects of a website, such as speed, performance, and user experience, to ensure it loads quickly, ranks well in search engines.
- The main benefits of SEO include improved visibility, cost-effectiveness, and enhanced credibility and trust.
- Most used html tags for Search engine optimization are: title tag, meta description tag, headings (H1 — H6), Title, meta tags, HTML5 semantic tags, image Alt attribute.
- Within application of web accessibility, people with disabilities can perceive, understand, navigate, and interact with the Web.

Application of learning 2.4

You work for an e-commerce company that sells a variety of products online. The company's management has identified the need to optimise the product pages to improve user experience and increase search engine visibility.

Your task is to optimize this website for both performance and accessibility.

You will need to:

- Improve the loading speed.
- Ensure the website is accessible to users with disabilities.
- Optimize the website for search engines (SEO).

Learning outcome 2 end assessment

Theoretical assessment

Choose the correct:

Q1: What is a webpage?

A. A collection of related web pages located under a single domain name

B. A software application for accessing information on the World Wide Web

C. A document on the World Wide Web that is identified by a unique URL

D. A tool used for writing and editing text and code

Q2: What is a website?

A. A single document on the World Wide Web

B. A collection of related web pages located under a single domain name

C. A tool used for browsing the internet

D. The address used to access a webpage

Q3: What is a web browser?

A. A document on the World Wide Web that is identified by a unique URL

B. A collection of related web pages located on server and delivered to the client

C. A software application for accessing and viewing information on the World Wide Web

D. A tool used for reading music and video on computer displays

Q4: What is a text editor?

A. A document written using symbols and graphics

B. A collection of software that are used to write complicated scripts in notepad

C. A tool used for drawing graphics on webpage the internet

D. A tool used for writing and editing text and code

Q5: What is a hyperlink?

A. A tool used for writing and editing text and code

B. A collection of related web pages located under a single domain name

C. The address used to access a webpage

D. A reference or navigation element in a document that links to another section of the same document or to another document

Question 6: Which of the following is not a web browser?

A) Safari

B) Edge

C) Atom

D) Opera

Question 7: When installing software from a storage drive, what is the first step?

A) Open a web browser

B) Insert the storage drive into the computer and locate setup

C) Run an antivirus scan

D) Open the command prompt

Complete the sentence below with appropriate term

Question 8: Search Engine Optimization (SEO) refers to the practice of optimizing a website to improve its …… and ranking on search engine results pages (SERPs) such as Google, Bing, and Yahoo.

Question 9: Web accessibility ensures that websites and web applications can be used by people with various ……

Question 10: HTML (HyperText Markup Language) plays a crucial role in web accessibility by providing the structural foundation that ensures content is accessible to all ……., including those with disabilities.

Match the questions in column A to its corresponding in column B

ANSWER	COLUMN A Accessibility testing workflow	COLUMN B How it is done
1……	1.Run Automated Tests	A. Re-test to ensure the issues are resolved and no new issues have been introduced
2……	2.Conduct Manual Tests	B. Use the W3C Markup Validation Service to check your HTML for compliance.
3……	3.Perform User Testing	**C.** Recruit users with disabilities to test your site and provide feedback.
4……	4. Validate Against Standards	**D.** Navigate your site using only the keyboard to ensure all interactive elements are accessible.
5……	5. Implement Improvements	**E.** Scan your site with Axe DevTools for more detailed findings.

Practical assessment

BZM Company Ltd. is a construction company in Kigali city with many customers, but they have recently discovered that the number of their customers is decreasing because many of the customers do not have enough information about the company, so the company manager decided to hire a web developer to build for them the structure of a website that will be used to make available the company's information including its daily activities, company profile, contacts, and any other information that the company manager deems necessary. You are tasked to design a web structure for BZM company.

Refences

Mobility Lab - Ultra-Slim Wired PC Keyboard Space Grey - French AZERTY USB connection : Amazon.com.be: Electronics

What is a Computer Keyboard? - Parts, Layout & Functions - Lesson | Study.com

What is a keyboard layout? (with 90+ list of them) - TypingDoneWell.com

QWERTY, QWERTZ, and AZERTY - All you need to know about them - TypingDoneWell.com

What is a Key Combination? How to Use Them on Mobile Devices | Lenovo US


<!DOCTYPE html>
<html lang="EN">
<head>
<meta charset="UTF-8" />
<meta http-equiv="X-UA-Compatible"
content="IE=edge" />
<meta name="viewport" content="width=de-
vice-width, initial-scale=1.0" />
<link rel="stylesheet" href="https://cdnjs.cloud-
flare.com/ajax/libs/font-awesome
/6.0.0-beta2/css/all.min.css" integrity="-
sha512-YWzhKL2whUzgiheMoBFwW8CKV4qpHQAEuvilg9
FAn5VJUDwKZZxkJNuGM4XkWuk94WCrrwslk8yWNG
mY1EduTA==" crossorigin="anonymous" referrerpolicy="no-re-
ferrer" />
<link rel="apple-touch-icon" sizes="180x180"
href="favicon.ico" />
<link rel="apple-touch-icon" sizes="57x57"
href="./images/fa
<link rel="apple-touch-icon" sizes="120x120"
href="./images/
<meta property="og:title" content="text" />
<meta property="og:url" content="http://won-
HTML
Home
Info
Contact us
HTML
* {
margin: 0;
padding: 0;
}
html {
overflow-x: hidden;
}
#wrap {
width: 100vw;
height: 100%;
background: radial-gradient(
circle at top left,
hsla(var(--hue1), 80%, 30%, 0.2),
transparent 20%
),
radial-gradient(
circle at top right,
hsla(var(--hue2), 90%, 80%, 0.5),
transparent 28%
),
radial-gradient(
at bottom left,
hsla(var(--hue3), 90%, 80%, 0.5),
transparent 40%
)
CSS

Indicative contents

3.1. Introduction to CSS

3.2. Creating CSS files

3.3. Applying typography on web page

3.4. Setting media query rules

Key Competencies for Learning Outcome 3: Style web elements

Knowledge	Skills	Attitudes
• Definition of CSS • Description types of CSS • Description of typography • Description of media query • Identification of basic structure and syntax of CSS	• Creating CSS syntax • Using different CSS style types • Using CSS display and positioning properties • Using CSS box model • Using CSS scripts for typography • Using breakpoint	• Be critical thinker • Be creative • Be detail-oriented • Have Passion for technology

Duration: 60 hrs

Learning outcome 2 objectives:

By the end of the learning outcome, the trainees will be able to:

1. Identify clearly the basic structure and syntax of CSS file as used in web development
2. Apply correctly the basic CSS rules to style HTML elements based on its function
3. Implement successful responsive typography techniques to ensure text looks good on various devices and screen sizes.
4. Apply correctly media query rules to create responsive layouts that adjust flexibly to various screen sizes.

Resources

Equipment	Tools	Materials
● Computer ● Projector/smartboard	● Text editors ● web browsers	● Internet

Indicative content 3.1: Introduction to CSS

Duration: 10 hrs

Theoretical Activity 3.1.1: Description of CSS

Tasks:

1. In small groups, you are requested to answer the following questions related to the CSS:
    a. What do you understand about CSS?
    b. Explain the utility of CSS.
    c. Describe types of CSS.

2.Provide the answer for the asked questions and write them on papers.

3.Present the findings/answers to the whole class

4.For more clarification, read the key readings 3.1.1. In addition, ask questions where necessary.

Key readings 3.1.1.:

❖ **Introduction to CSS (Cascading Style Sheets)**

CSS, or Cascading Style Sheets, is a style sheet language used for describing the presentation of a document written in HTML or XML. CSS is a cornerstone technology of the World Wide Web, alongside HTML and JavaScript.

Purpose of CSS

The primary purpose of CSS is to separate the content of a web page from its presentation. This separation allows web designers and developers to change the appearance of a website without altering its underlying HTML structure.

CSS enhances the visual and aesthetic aspects of a web page, making it more appealing and user-friendly.

❖ **Key Features of CSS (utility of CSS)**

1. **Separation of Content and Presentation**

CSS allows for a clear distinction between the structure of a web document (HTML) and its visual presentation (CSS). This separation enhances maintainability and flexibility.

2. **Reusability**

CSS styles can be reused across multiple web pages. By defining styles in an external CSS file, you can apply the same styles to multiple HTML documents, ensuring consistency throughout a website.

CSS is an essential tool for web designers and developers, enabling them to create visually appealing, maintainable, and accessible websites. By mastering CSS, you

can enhance the user experience and ensure your web content is presented effectively across various devices and screen sizes. Understanding the basics of CSS, including its syntax, types, and selectors, is the first step towards creating well-designed and responsive web pages.

3. **Improved Accessibility**
   By separating content from presentation, CSS contributes to better accessibility. Assistive technologies, such as screen readers, can access the content without being hindered by presentational elements.
4. **Responsive Design**
   CSS supports responsive design techniques, allowing web pages to adapt to different screen sizes and devices. Media queries enable designers to apply different styles based on screen size, orientation, and resolution.
5. **Design Flexibility**
   CSS provides a wide range of styling options, including colours, fonts, layouts, spacing, animations, and transitions. This flexibility enables designers to create visually engaging and interactive websites
6. **Styling and Layout**
   CSS allows for application of styles to HTML elements, controlling aspects like colours, fonts, spacing and positioning.
7. **Performance**
   External CSS files can be cashed by browsers, which improves loading times of web pages.

❖ **Types of CSS**

CSS defines the front-end appearance of your website. There are several types of CSS, among them are inline, internal and external CSS.

▪ **Inline CSS**

Inline CSS is used to style a specific HTML element. For this CSS style, you'll only need to add the style attribute to each HTML tag, without using selectors.

This CSS type is not really recommended, as each HTML tag needs to be styled individually. Managing your website may become too hard if you only use inline CSS.

Advantages of Inline CSS

- You can easily and quickly insert CSS rules into an HTML page. That's why this method is useful for testing or previewing the changes and performing quick fixes to your website.
- You don't need to create and upload a separate document as in the external style.

Disadvantages of Inline CSS

- Adding CSS rules to every HTML element is time-consuming and makes your HTML structure messy
- Styling multiple elements can affect your page's size and download time

❖ **Internal CSS**

Internal or embedded CSS requires you to add a <style> tag in the <head> section of your HTML document.

This CSS style is an effective method of styling a single page. However, using this style for multiple pages is time-consuming as you need to put CSS rules on every page of your website.

Here's how you can use internal CSS:

- Open your HTML page and locate <head> opening tag.
- Put the following code right after the <head> tag

<style type="text/css">

Advantages of Internal CSS

- You can use class and ID selectors in this style sheet.
- Since you'll only add the code within the same HTML file, you don't need to upload multiple files.

Disadvantages of Internal CSS:

Adding the code to the HTML document can increase the page's size and loading time.

❖ **External CSS**

With external CSS, you'll link your web pages to an external .css file, which can be created by any text editor in your device (e.g., Notepad++).

This CSS type is a more efficient method, especially for styling a large website. By editing one .css file, you can change your entire site at once.

to use consider the following points external CSS:

1. Create a new .css file with the text editor, and add the style rules.
2. In the <head> section of your HTML sheet, add a reference to your external .css file right after <title> tag:

```
<link rel="stylesheet" type="text/css" href="style.css" />
```

Don't forget to change style.css with the name of your .css file.

Advantages of External CSS:

- Since the CSS code is in a separate document, your HTML files will have a cleaner structure and are smaller in size.
- You can use the same .css file for multiple pages.

Disadvantages of External CSS

- Your pages may not be rendered correctly until the external CSS is loaded.
- Uploading or linking to multiple CSS files can increase your site's download time.

Points to Remember

- Primarily role of css is to format and structuring web elements CSS provides Key Features(utility) such as:
  - ✓ Separation of Content and Presentation
  - ✓ Reusability
  - ✓ Improved Accessibility
  - ✓ Responsive Design
  - ✓ Design Flexibility
  - ✓ Styling and Layout
  - ✓ Performance
- There are three basic types of css according to where it is placed in document, those types are:
  - ✓ Internal or embedded
  - ✓ External
  - ✓ Inline

Indicative content 3.2: Creating CSS files

Duration: 20 hrs

Theoretical Activity 3.2.1: Description of CSS files

Tasks:

1: In small groups, you are requested to answer the following questions related to cascading stylesheet:

a. What do you understand by the syntax of css?
b. Describe the way to use different types of CSS.
c. What do you understand by css box model?
d. What are the visual rules to consider when working with css?
e. what is the use of Use CSS Display and positioning?

2.Provide the answer for the asked questions and write them on papers/flipchart.

3.Present the findings/answers to the whole class

4.For more clarification, read the key readings 3.2.1. In addition, ask questions where necessary.

Key readings 3.2.1.:

- **Introduction to CSS Syntax**

  CSS syntax refers to the way we write CSS code. In order to write CSS, you need to first identify the element in your HTML page that you want to style before adding styles using a plethora of built-in CSS properties.

  There are multiple ways to identify and tell CSS which element in your HTML page it should style. CSS syntax follows the format demonstrated here.

  In CSS, **Selector** points to the HTML element you want to style, **Property** is the built-in CSS property to use to style this element, **Value** is the value we want to apply for this CSS property, and **Declaration** consists of the **CSS property** and **CSS value**, separated by a colon **(:)**

  CSS declarations are separated by a semicolon (;). Declaration blocks are all contained within curly braces.

- **CSS Selectors**

  CSS Selectors are used to describe which HTML element(s) you want to style. CSS provides multiple ways to describe the selectors:

- **Element selectors**
- **ID selectors**
- **Class selector**

  Element selectors are the simplest form of defining which HTML elements to style.

you need to tell CSS just the tag name of the element that you want to style**. For example** – h1, a, p or div.

In order to target all **<p>** tags, we use the **p selector.**

In order to target all ** ** tags, we use the **div selector.**

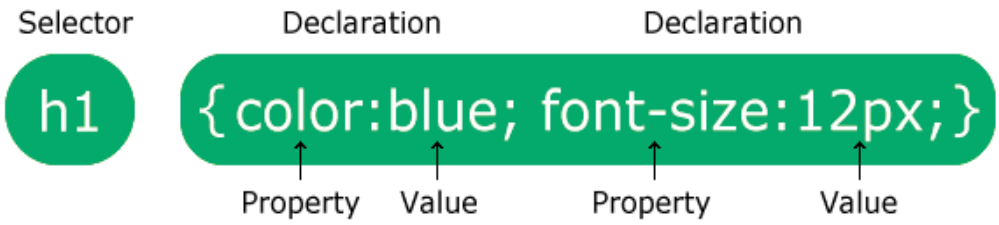
- **CSS Properties**

Using specific CSS properties is how we can change how our elements appear on a web page. Remember the syntax above, we need to use a selector first, and then we describe what styles we wanted to apply for a particular element.

Some common CSS properties that we can change are listed below:

Description	CSS Property Name	Value Type	Example
Text Color	`color`	Color name, Hex Code, RGB Value	`color: black;`
Text Size	`font-size`	Pixel Value	`font-size: 20px;`
Font Weight	`font-weight`	normal, bold, bolder, lighter, number from 100 to 900 for thinner to thicker fonts	`font-weight: bold;`
Font Style	`font-style`	normal, italic, oblique	`font-style: italic;`
Background Color	`background-color`	Color name, Hex Code, RGB Value	`background-color: red;`

❖ **Description of css syntax**



❖ **Describe CSS style types**

Let us have an example of the use of different styling

1. **Inline CSS** is applied directly to an element using the style attribute.

Inline CSS is best for quick styling or when you need to apply a specific style to a single element without affecting others.

2. **Internal CSS** is placed within a <style> tag in the HTML document's <head>.

• It is used for styling a single HTML document (webpage).

3.External CSS is linked via a separate .css file using the <link> tag in the HTML document's <head>. External CSS is applied using a separate CSS file that contains all the styles. This file is then linked to one or more HTML documents.

• It is the most common and efficient way to apply CSS to multiple web pages, External CSS is ideal for large websites where styles need to be consistent across multiple pages. It promotes reusability, as one CSS file can be linked to multiple HTML documents, making it easier to maintain and update styles.

❖ **CSS Visual Rules**

Visual rules in CSS encompass all the properties and techniques that influence the appearance and layout of elements on a webpage. By mastering these rules, you can create visually appealing, responsive, and user-friendly designs.

Here's a breakdown of the key types of visual rules in CSS:

1. Layout
2. Color and Background
3. Typography
4. Borders and Outlines
5. Spacing
6. Shadows
7. Visibility and Opacity
8. Transformations and Transitions
9. Overflow
10. Z-index

❖ **CSS Display and positioning**

The CSS position property defines, as the name says, how the element is positioned on the web page.

So, there are several types of positioning: static, relative, absolute, fixed, sticky, initial and inherit. First of all, let's explain what all of these types mean.

- **Static** - this is the default value; all elements are in order as they appear in the document.
- **Relative** - the element is positioned relative to its normal position.
- **Absolute** - the element is positioned absolutely to its first positioned parent.
- **Fixed** - the element is positioned related to the browser window.
- **Sticky** - the element is positioned based on the user's scroll position.

Relative Positioning

When you set the position relative to an element, without adding any other positioning attributes (top, bottom, right, left) nothing will happen. When you add position apart from relative, such as left: 20px the element will move 20px to the right from its normal position. Here, you can see that this element is relative to itself. When the element moves, no other element on the layout will be affected.

There is a thing you should keep in mind while setting position - relative to an element limits the scope of absolutely positioned child elements. This means that any element that is the child of this element can be absolutely positioned within this block.

After this brief explanation, we need to back it up, by showing an example.

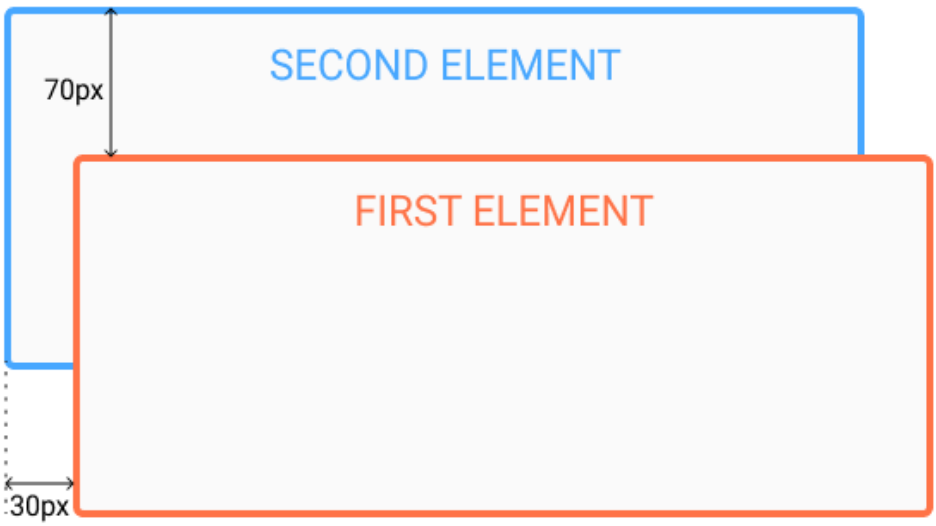
In this example, you will see how the relative positioned element moves when its attributes are changed. The first element moves to the left and top from its normal position, while the second element stays in the same place because none of the additional positioning attributes were changed.

HTML

```
<div id="first_element">First element</div>
<div id="second_element">Second element</div>
```

CSS

```css
#first_element {
  position: relative;
  left: 30px;
  top: 70px;

  width: 500px;
  background-color: #fafafa;
  border: solid 3px #ff7347;
  font-size: 24px;
  text-align: center;
}

#second_element {
   position: relative;

   width: 500px;
   background-color: #fafafa;
   border: solid 3px #ff7347;
   font-size: 24px;
   text-align: center;
}
```



Absolute Positioning

Absolute positioning allows you to place your element precisely where you want it.

Absolute positioning is done relative to the first relatively (or absolutely) positioned parent element. In the case when there is no positioned parent element, the element that has position set to absolute will be positioned related directly to the HTML element (the page itself).

An important thing to keep in mind while using absolute positioning is to make sure it is not overused, otherwise, it can lead to a maintenance nightmare.

The next thing, yet again, is to show an example of absolute positioning.

In the example, the parent element has the position set to relative. Now, when you set the position of the child element to absolute, any additional positioning will be done relative to the parent element. The child element moves relatively to the top of the parent element by 100px and right of the parent element by 40px.

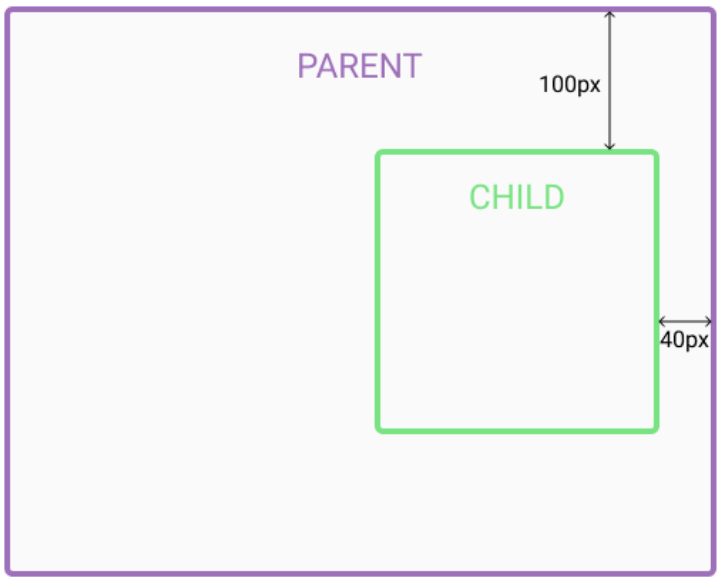
HTML

```
<div id=”parent”>
  <div id=”child”></div>
</div>
```

CSS

```css
#parent {
  position: relative;

  width: 500px;
  height: 400px;
  background-color: #fafafa;
  border: solid 3px #9e70ba;
  font-size: 24px;
  text-align: center;
}

#child {
  position: absolute;
  right: 40px;
  top: 100px;

  width: 200px;
  height: 200px;
  background-color: #fafafa;
  border: solid 3px #78e382;
  font-size: 24px;
  text-align: center;
}
```

Output



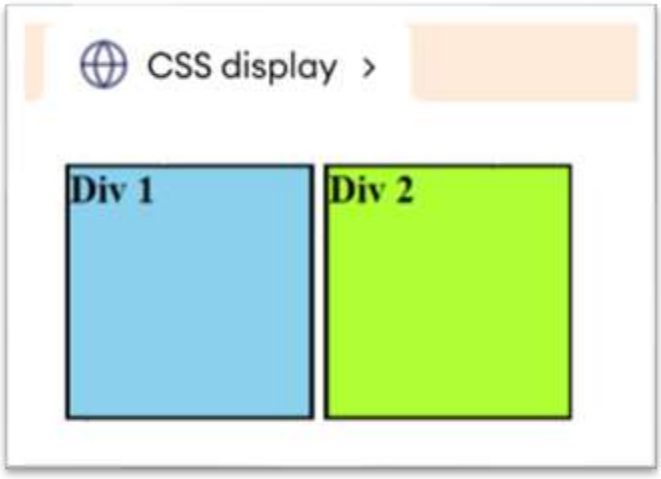
Through these examples, you have seen differences between absolute and relative positioned elements.

- **CSS Display Property**

  The CSS display property is used to adjust the layout of an element. For example:

```
div {
   display: inline-block;
}
```

  Browser Output



  Here, the inline-block value of the display property adds both div elements in the same horizontal flow. By default, the block-level elements like div, h1, etc., start a new line and take full width.

  The display property allows changing the display behaviour of inline and block elements.

Practical Activity 3.2.2: Creating CSS files

Task:

1. Referring to the previous theoretical activity (3.2.1) you are asked to go the computer lab to create CSS file. This task should be done individually.
2. Read key reading 3.2.2 and ask clarification where necessary
3. Apply safety precautions.
4. Outline the steps to create CSS file
5. Referring to the outlined steps provided on task 3, create CSS file.
6. Present your work to the trainer and whole class

Key readings 3.2.2

To create a CSS file, you must have text editor as development tools and you will need a web browser to test it or see the results:

Here are the steps to create your CSS file:

- **Steps to create CSS File:**
    1. Open a Text Editor
    2. Create a New File
    3. Save the File with a **.css** Extension choose save type as all types or css
    4. Write Your CSS Code
    5. Link the CSS File to Your HTML Document (if it is external css)
    6. Save and Test

❖ **Use of CSS style types**

▪ **Inline css**

Including CSS styles inline in an HTML document can be helpful in certain situations, such as when you want to apply a unique style to a single element on a page. You can use the style attribute on the relevant HTML element to include CSS styles inline**.**

When using inline CSS, follow these steps:

1. **Locate the HTML Element:** Identify the specific HTML element you want to style directly in your HTML file.

   Add the style Attribute: Inside the opening tag of the element, add the style attribute.
2. **Add the style Attribute**: Inside the opening tag of the element, add the style attribute
3. **Write CSS Rules:** Within the style attribute, write the CSS properties and values you want to apply to the element. Separate multiple properties with semicolons

   Example:

```
<p style="color: blue; font-size: 16px;">This is styled text.</p>
```

4. **Check for Syntax:** Ensure all properties and values are correctly written, with a colon separating them and a semicolon at the end of each rule.
5. **Apply and Test:** After adding the inline styles, save your changes and test the web page to see how the styles are applied.

   Example:

```html
<html>
  <body>
    <p style="font-size: 14px; color: red">
      This is a paragraph with inline CSS styles.
    </p>
    <p>This is a paragraph without any styles.</p>
  </body>
</html>
```

- **Internal CSS**

  Internal CSS can be applied to the whole web page but not on multiple web pages.

  When using internal CSS in an HTML document, follow these steps:

  Step1: Open html file using text editor

  Step 2: Open the <style> Tag:

- Place the <style> tag within the <head> section of your HTML document

```html
<head>
  <style>
    /* CSS code goes here */
  </style>
</head>
```

Step 3: Write CSS Rules:

- Inside the <style> tag, write your CSS rules to style elements. Define selectors (like elements, classes, or IDs) and specify properties and values.

```
<!DOCTYPE html><html lang="en">
<head>
<style>
  body {
    font-family: Arial, sans-serif;
    background-color: gray;
  }

  h1 {
    color: green;
  }

  .container {
    width: 80%;
    margin: 0 auto;
  }
</style>
</head>
<body>

</body></html>
```

Step 4: Apply the CSS to HTML Elements

Reference the defined classes, IDs, or elements in your HTML content to apply the styles

```
<body>

    <div class="container">
      <h1>Learn Internal CSS</h1>
      <p>This is a paragraph styled with internal CSS.</p>
    </div>
</body></html>
```

Step 5: Save and Preview:

- Save your HTML file and preview it in a browser to see the styles applied

You will get the following Result.

- **External Css**

  When using external CSS to style your HTML documents, follow these steps:

  Step1: Create a CSS File

- Write your CSS rules in a separate file with a **.css** extension (e.g., styles.css).
- Place all your styles in this file, such as selectors, properties, and values

  Step2: Link the CSS File to HTML

- In your HTML document, use the <link> tag within the <head> section to link to your external CSS file.
- The <link> tag should include the **href** attribute pointing to the CSS file, and the **rel** attribute set to "**stylesheet**"

  Example:

```
<link rel="stylesheet" href="styles.css">
```

  Step 3: Organize Your CSS

- Organize your CSS code logically by grouping related styles together (e.g., typography, layout, colors).
- Use comments to annotate different sections of your CSS file for better readability.

  Step 4: Apply CSS Rules

  The styles defined in your external CSS file will automatically apply to the HTML elements that match the selectors.

Step5: Test Across Devices

Ensure the external CSS file is properly linked and test the styles across different devices and browsers to verify that everything displays correctly.

❖ **CSS Visual Rules**

Visual rules in CSS encompass all the properties and techniques that influence the appearance and layout of elements on a webpage. By mastering these rules, you can create visually appealing, responsive, and user-friendly designs.

Here's a breakdown of the key types of visual rules in CSS:

1. **Layout**

- Display: Determines how an element is displayed on the page (e.g. block, inline, flex, grid).

Example:

```
.container {
  display: flex;
}
```

- Positioning**:** Controls the position of an element (e.g: static, relative, absolute, fixed, sticky).

```
.header {
  position: fixed;
  top: 0;
  width: 100%;
}
```

- Box Model: Manages the size and spacing of elements, including margin, border, padding, and width

```
.box {
  width: 300px;
  padding: 20px;
  margin: 10px;
  border: 1px solid #ccc;
}
```

2. **Color and Background**

- **Color:** Sets the color of text

```
.text
 {
 color:  green;
}
```

- **Background:** Defines the background color or image of an element

```
.background {
  background-color: #f0f0f0;
  background-image: url('image.jpg');
  background-size: cover;
}
```

3. **Typography**

- **Font:** Specifies the font family, size, weight, and style

```
.text {
  font-family: Arial, sans-serif;
  font-size: 16px;
  font-weight: bold;
}
```

- **Text Alignment:** Controls the alignment of text (e.g., left, right, center, justify).

```
.text {
  text-align: center;
}
```

- **Line Height and Letter Spacing:** Adjusts the spacing between lines of text and characters

```
.text {
  line-height: 1.5;
  letter-spacing: 0.05em;
}
```

4. **Borders and Outlines**

- **Border**: Defines the border around an element, including its width, style, and color.

```
.box
  bor .box:focus {
}       outline: 2px dashed #00f;
      }
```

- **Outline:** Similar to borders but does not take up space in the layout

5. **Spacing**

- **Margin:** Controls the space outside an element's border

```
.box {
  margin: 20px;
}
```

- **Padding:** Controls the space inside an element's border, between the content and the border.

```
.box {
  padding: 10px;
}
```

6. **Shadows**

- **Box Shadow:** Adds a shadow around an element's box

```
.box {
  box-shadow: 0 4px 8px rgba(0, 0, 0, 0.2);
}
```

- **Text Shadow:** Adds a shadow to text.

```
.text {
  text-shadow: 1px 1px 2px #999;
}
```

7. **Visibility and Opacity**

- **Visibility:** Controls whether an element is visible or hidden (visible, hidden).

```
.hidden {
  visibility: hidden;
}
```

- **Opacity:** Sets the transparency level of an element

```
.transparent {
  opacity: 0.5;
}
```

8. **Transformations and Transitions**

- **Transform:** Applies 2D or 3D transformations (e.g., rotate, scale)

```
.rotate {
  transform: rotate(45deg);
}
```

- **Transition**: Smoothly animates changes in CSS properties

```
.button {
  transition: background-color 0.3s ease;
}
```

9. **Overflow**

- **Overflow:** Controls what happens when content overflows an element's box (e.g., hidden, scroll).

```
.content {
  overflow: auto;
}
```

10. **Z-Index**

- **Z-Index: Controls the stacking order of elements that overlap**

```
.box {
  z-index: 10;
}
```

❖ **CSS Display and positioning**

When applying display and positioning properties in CSS, follow these steps to control the layout and placement of elements on a webpage:

1. Understand the Element's Role

- Determine the role of the element in the layout (e.g., is it a block-level container, an inline text, or something else?).

- Decide whether the element should flow naturally with the rest of the content or require specific positioning.

2. Apply the display Property

- Choose the appropriate display value based on how you want the element to behave.

3.Apply the position Property

- Choose the appropriate position value to control the element's placement

4.Use Offsets with Positioned Elements

- When using relative, absolute, fixed, or sticky, apply offsets (top, right, bottom, left) to move the element from its original position.

5. Combine with Other CSS Properties

- Combine display and position with other properties like z-index to control stacking order, or overflow to manage content overflow.

6. Test Across Different Screen Sizes

- Test how the element behaves across different screen sizes and devices, adjusting your CSS as needed to ensure a responsive design.

Example:

HTML FILE (index.html)

<!DOCTYPE html> <html lang="en"> <head> <meta charset="UTF-8"> <meta name="viewport" content="width=device-width, initial-scale=1.0"> <title>CSS Position Property Example</title> <link rel="stylesheet" href="styles.css"> </head> <body> <h1>CSS Position Property Example</h1> Static Position (default) Relative Position I am positioned relative to my parent.   Absolute Position Fixed Position (always at the bottom-right corner) Sticky Position (sticks to the top as you scroll) Scroll down to see the sticky effect. <!-- Just to create scrollable space --> </body> </html>

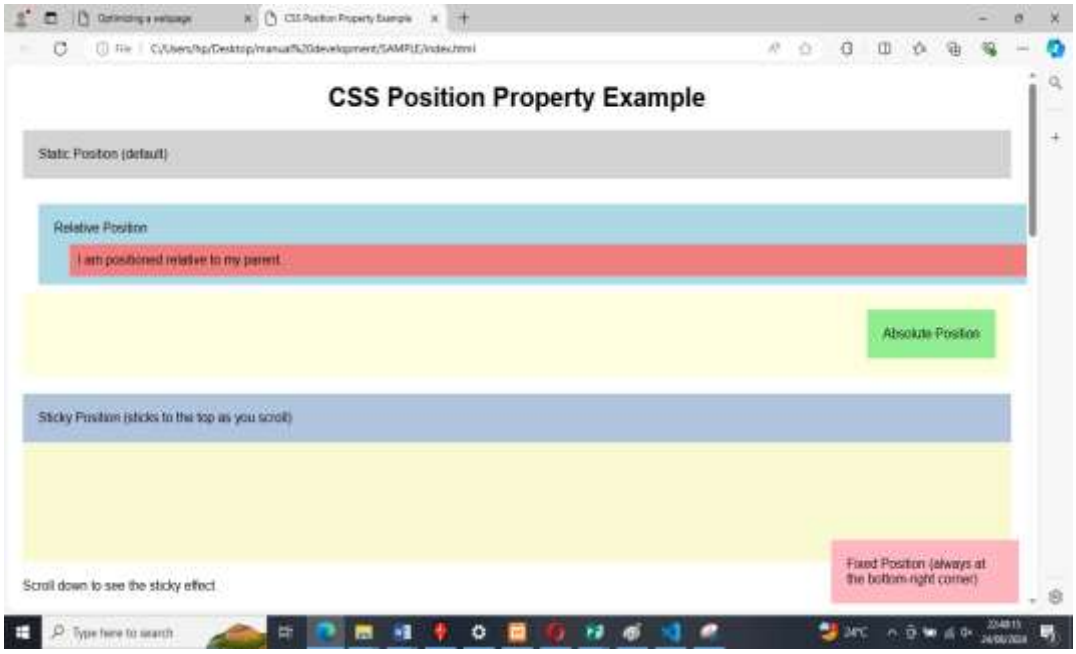
CSS FILE (styles.css)

```
body {
  font-family: Arial, sans-serif;
  margin: 20px;
}

h1 {
  text-align: center;
  margin-bottom: 20px;
}
```

```
/* Static Position (default) */
.static-box {
   position: static;
   background-color: lightgray;
   padding: 20px;
   margin-bottom: 20px;
}

/* Relative Position */
.relative-box {
   position: relative;
   background-color: lightblue;
   padding: 20px;
   margin-bottom: 20px;
   top: 10px;
   left: 20px;
}

.relative-child {
   position: relative;
   background-color: lightcoral;
   padding: 10px;
   top: 10px; /* Moves 10px down from its normal position */
   left: 20px; /* Moves 20px to the right from its normal position */
}

/* Absolute Position */
.absolute-container {
   position: relative;
   background-color: lightyellow;
   padding: 50px;
   margin-bottom: 20px;
}
.absolute-box {
   position: absolute;
   top: 20px; /* 20px from the top of the nearest positioned ancestor */
   right: 20px; /* 20px from the right of the nearest positioned ancestor */
   background-color: lightgreen;
   padding: 20px;
```

```
}
/* Fixed Position */
.fixed-box {
    position: fixed;
    bottom: 10px; /* Stays 10px above the bottom of the viewport */
    right: 10px; /* Stays 10px from the right edge of the viewport */
    background-color: lightpink;
    padding: 20px;
    width: 200px;
}
/* Sticky Position */
.sticky-container {
    height: 200px;
    background-color: lightgoldenrodyellow;
    margin-bottom: 20px;
}
.sticky-box {
    position: -webkit-sticky; /* For Safari */
    position: sticky;
    top: 0; /* Sticks to the top of the container when scrolling */
    background-color: lightsteelblue;
    padding: 20px;
}
```

Output



Points to Remember

- There are main three ways to apply css in web documents such as using inline, internal and external
- CSS Visual Rules encompass all the key properties and techniques that influences the appearance and layout of elements on webpage, here are Key types of visual rules in CSS: Layout, color and Background, typography, Borders and Outlines, spacing, shadows, visibility and opacity, transform and transition, overflow, z-index.
- The commonly used values for the display property are like: Inline, block, inline-block, flex, grid, none…
- In CSS, element positioning refers to the placement and arrangement of HTML elements on a webpage. Those elements of CSS positioning are: Relative, Absolute, Fixed Sticky, Static.
- The CSS box model is essentially a box that wraps around every HTML element. It consists of: content, padding, borders and margins.
- To create a separate css file follow these steps**:**
    1. 1.Open a Text Editor
    2. Create a New File
    3. Save the File with a **.css** Extension choose save type as all types or css
    4. Write Your CSS Code
    5. Link the CSS File to Your HTML Document
    6. 6.Save and Test

Application of learning 3.2.

You've been hired as a frontend developer for a popular blogging platform. Your task is to enhance the platform's user interface and improve the overall user experience through effective use of CSS (use Font, Text Colours, background colours, Opacity, Background image, Display, positioning and CSS Box model).

Indicative content 3.3: Applying Typography

Duration: 15 hrs

Theoretical Activity 3.3.1: Description of typography

Tasks:

1. In small groups, you are requested to answer the following questions related to the typography.

   a. What do you understand by term typography?
   b. What are scripts used for typography?

2. Provide the answer for the asked questions and write them on papers.
3. Present the findings/answers to the whole class
4. For more clarification, read the key readings 3.3.1. In addition, ask questions where necessary.

Key readings 3.3.1

❖ **Introduction to typography**

Typography is the art of arranging letters and text in a way that makes the copy readable, clear, and visually appealing to the reader.

It involves font style, appearance, and structure, which aims to elicit certain emotions and convey specific messages. In short, typography is what brings the text to life.

Benefits of using typography

Typography is so much more than just choosing beautiful fonts; it's a vital component of user interface design.

Good typography will establish a strong visual hierarchy, provide a graphic balance to the website, and set the product's overall tone. Typography should guide and inform your users, optimize readability and accessibility, and ensure an excellent user experience.

The different elements of typography

▪ **Fonts and typefaces**

There's some confusion surrounding the difference between typefaces and fonts,

with many treating the two as synonymous.

A typeface is a design style that comprises a myriad of characters of varying sizes and weight, whereas a font is a graphical representation of a text character.

Put simply, a typeface is a family of related fonts, while fonts refer to the weights, widths, and styles that constitute a typeface.

There are three basic kinds of typeface: serif, sans-serif, and decorative.

Serif

As the visual example above demonstrates, serif typefaces are identified by the extra marks at the end of letters.

The addition of these small strokes and elements gives serif fonts an air of tradition, history, authority, and integrity.

Sans-serif

Just like the name suggests, sans-serif typefaces are defined by what they lack. Without the serif's more traditional strokes and dashes, the sans-serif font family is seen as much more modern and bolder.

Popular sans-serif fonts include Helvetica and Arial, the default font when you start writing in a Google Doc.

Decorative

Again, given away by its name, the function of this typeface is aesthetic more than readable. As a result, you're far more likely to see these used in brand names, logos, and short titles.

- **Contrast**

  Much like hierarchy, contrast helps to convey which ideas or messages you want to emphasize to your readers.

  Spending some time on contrast makes your text interesting, meaningful, and attention-grabbing. Most designers create contrast by playing around with varying typefaces, colors, styles, and sizes to create impact and break up the page.

- **Consistency**

  Keeping your typefaces consistent is key to avoiding a confusing and messy interface.

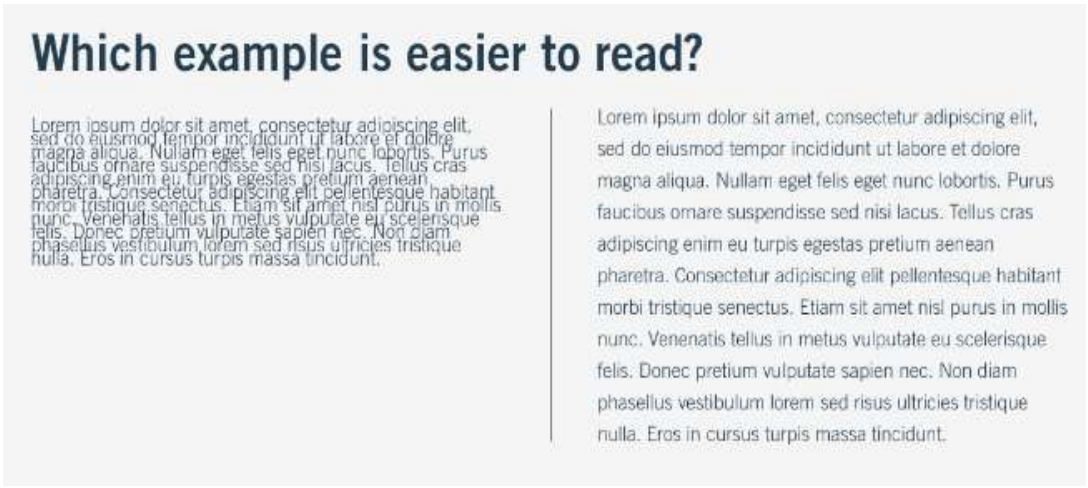
  When conveying information, it's essential to stick to the same font style so your readers instantly understand what they're reading and begin to notice a pattern.

- **White space**

  Often referred to as "negative space," white space is the space around text or graphics.

  It's often overlooked and tends to go unnoticed by the user, but proper use of white space ensures the interface is uncluttered and the text is readable.

  White space can even draw attention to the text and provide an aesthetically pleasing experience. White space often takes the form of margins, padding, or just areas with no text or graphics.



- **Alignment**

  Alignment is the process of unifying and composing text, graphics, and images to ensure equal space, size, and distance between each element.

  Many UI designers create margins to ensure that their logo, header, and body of the text are aligned with each other.

  When aligning your user interface, it's good practice to pay attention to industry standards. **For example**, aligning your text to the right will seem counterintuitive for readers who read left to right.

- **Color**

  Color is one of the most exciting elements of typography. This is where designers can really get creative and elevate the interface to a new level.

  Text color, however, is not to be taken lightly: nailing your font color can make the text stand out and convey the tone of the message—but getting it wrong can result in a messy interface and text that clashes with the site colors.

  Color has three key components: value, hue, and saturation.

  A good designer will know how to balance these three components to make the text both eye-catching and clearly legible, even for those with visual impairments.

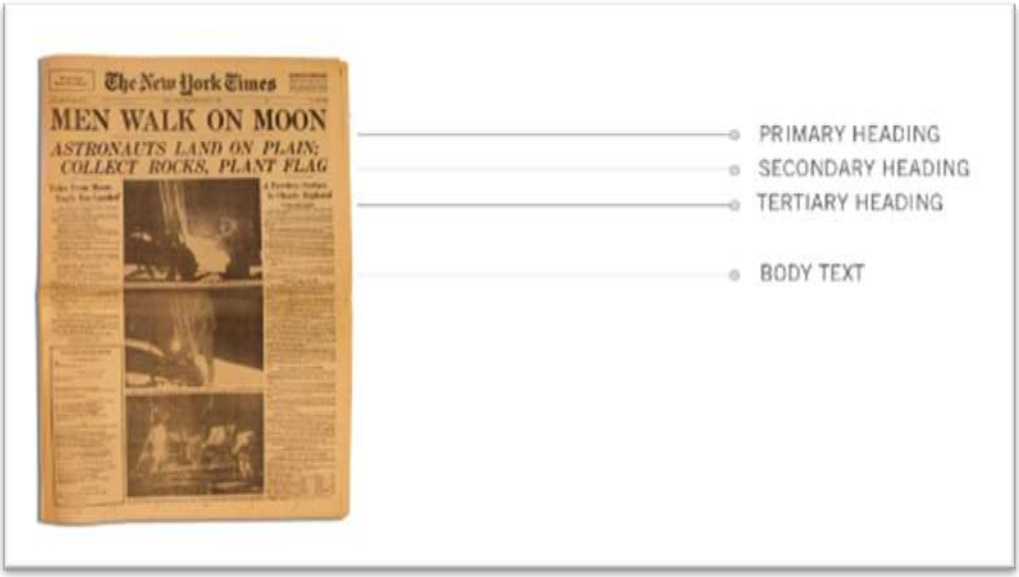
- **Hierarchy**

  Establishing a hierarchy is one of the most vital principles of typography.

  Typographical hierarchy aims to distinguish between prominent pieces of copy that should be noticed and read first and standard text copy.

  In an age of short attention spans brought about by social media, designers are urged to be concise and create typefaces that allow users to consume the necessary

information in short amounts of time.



❖ **CSS scripts for typography**

This is the first part of a three-part series of guides on CSS typography that will cover everything from basic syntax to best practices and tools related to CSS typography.

- **Typography CSS Properties**

There are two main groups of CSS properties that control typography style: **font** and **text.**

The font CSS property group dictates general font characteristics such as font-style and font-weight. Below you'll see the p element given a font-style and a font-weight property.

p { font-style: oblique; font-weight: bold; }

Default paragraph style

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Cras bibendum tortor ligula, non vehicula justo. Nunc sed elit tellus. Ut viverra facilisis lectus eget egestas. Mauris aliquet cursus commodo. Proin nibh lacus, scelerisque non vulputate sit amet, aliquet consequat felis. Nullam fringilla dictum lorem vel egestas.

Paragraph with font-style and font-weight properties

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Cras bibendum tortor ligula, non vehicula justo. Nunc sed elit tellus. Ut viverra facilisis lectus eget egestas. Mauris aliquet cursus commodo. Proin nibh lacus, scelerisque non vulputate sit amet, aliquet consequat felis. Nullam fringilla dictum lorem vel egestas.

The text CSS property group deals with the characters, spaces, words and paragraphs. For example, the text-indent property indents the first line of a text block. The letter-spacing property controls the spaces between a text block.

Below, you'll see the p element given a text-indent and a letter-spacing property.

p {text-indent: 50px; letter-spacing: 2px; }

Default paragraph style

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Cras bibendum tortor ligula, non vehicula justo. Nunc sed elit tellus. Ut viverra facilisis lectus eget egestas. Mauris aliquet cursus commodo. Proin nibh lacus, scelerisque non vulputate sit amet, aliquet consequat felis. Nullam fringilla dictum lorem vel egestas.

Paragraph with text-indent and letter-spacing properties

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Cras bibendum tortor ligula, non vehicula justo. Nunc sed elit tellus. Ut viverra facilisis lectus eget egestas. Mauris aliquet cursus commodo. Proin nibh lacus, scelerisque non vulputate sit amet, aliquet consequat felis. Nullam fringilla dictum lorem vel egestas.

There are other CSS properties that can affect web typography outside of the fonts

and text property groups.

For example, the color property controls an HTML element's foreground color, and can be used to change the color of text.

- **Font Sizing**

  Browser compatibility is a major issue in web design, even more so if you are attempting to make your web designs look the same in all browsers. Beginning web designers may mess around with a font's size until it looks just right, only to find it has completely changed among other browsers and platforms.

  Size fonts the correct way, and this problem can be minimized. A simple use of sizing text is as follows:

  h6 {font-size: 12pt; }

  The above sets the heading level-6 element to 12pt. For font-size values, there are 4 types of units of measurement.

  Absolute-size

  Standard keywords whose values are defined by the user agent (e.g., web browser). Values in W3C CSS 2.1 specifications are xx-small, x-small, small, medium, large, x-large, xx-large.

  Relative-size

  Standard keywords that size fonts based on the parent element.

  These are also defined by the user agent. Possible values are larger and smaller. Below you will see what a paragraph element looks like inside a parent element ( ) that has a size of 12pt (in Firefox).

  div { font-size: 12pt; } p { font-size: large; }

Default paragraph style

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Cras bibendum tortor ligula, non vehicula justo. Nunc sed elit tellus. Ut viverra facilisis lectus eget egestas. Mauris aliquet cursus commodo. Proin nibh lacus, scelerisque non vulputate sit amet, aliquet consequat felis. Nullam fringilla dictum lorem vel egestas.

Paragraph with font-size set to large

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Cras bibendum tortor ligula, non vehicula justo. Nunc sed elit tellus. Ut viverra facilisis lectus eget egestas. Mauris aliquet cursus commodo. Proin nibh lacus, scelerisque non vulputate sit amet, aliquet consequat felis. Nullam fringilla dictum lorem vel egestas.

Absolute Lengths

Absolute lengths are literal sizes.

For example, 12px is exactly 12 pixels and 2in is exactly 2 inches. Absolute lengths are often used by web designers. Possible units for absolute length units are pt, px, mm, cm, in, and pc. Millimetres (mm), centimeters (cm), and inches (in) are more suitable to physical, real-world dimensions and are units of measurement often used in print design. They are not very suitable for screen-based measurements because of the high variance of screen resolutions. Points (pt) and picas (pc) — while better than mm, cm and in — can also vary visually based on the browser or device's dpi.

Therefore, when using absolute lengths, pixels (px) are the least problematic. One issue with px, though, is that older versions of IE cannot resize them natively. If developing for an audience that will likely resize text manually through their web browser, the px unit of measurement may not be a good option.

Be sure when using px for font sizing that accessibility is not an issue.

Relative Lengths

The other type of length units are relative lengths. This means that their sizes are dependent on the font-size assigned to their parent element.

Possible units are em, % and ex. Not many web designers use ex — it is the height of the letter "x" in the current font. em and % values are far easier to work with.

em and % act identically, and it's just a matter of syntax:

0.5em is equal to 50%

1em is equal to 100%

0.2em is equal to 20%

0.73em is equal to 73%

2.21em is equal to 221%

html, **body { font-size: 85%; /* = (.85em) */ } h1 { font-size: 110%; /* = (1.1em) */ }**

The proportions are all relative to the element's parent font-size. So, if the base font size (in body or html) is set to 80%, a child with a font-size of 0.2em or 20% would be 20% in height of the original 80%.

Font Stacks

CSS font stacks are a list of fonts that include fonts that will work well in various operating systems and platforms, with the goal of making typography as consistent as possible.

Here is an example of a font stack:

body {font-family: Georgia, Times, "Times New Roman", serif; }

CSS Typography Pseudo-classes and Pseudo-elements

CSS pseudo-classes and pseudo-elements are great for targeting certain types of elements. Pseudo-classes start with a colon (:) followed by the class/element name**.**

There are several CSS pseudo-classes/pseudo-elements related to typography, such as :hover and :first-letter. Let's go over some that will help in styling our typography**.**

- **Links and Dynamic Pseudo-classes**

  The design of a website's hyperlinks is very important.

  Use anchor link pseudo-classes to create font styles for each link state. **:hover** is probably the most familiar, and it is good practice to create a separate (yet similar) style for it to provide a visual cue that a link element is interactive. Here are the link pseudo-classes:

  a:link { color: #666666; text-decoration: none; }

  a:visited { color: #333333; }

a:hover { text-decoration: underline; }

a:active { color: #000000; }

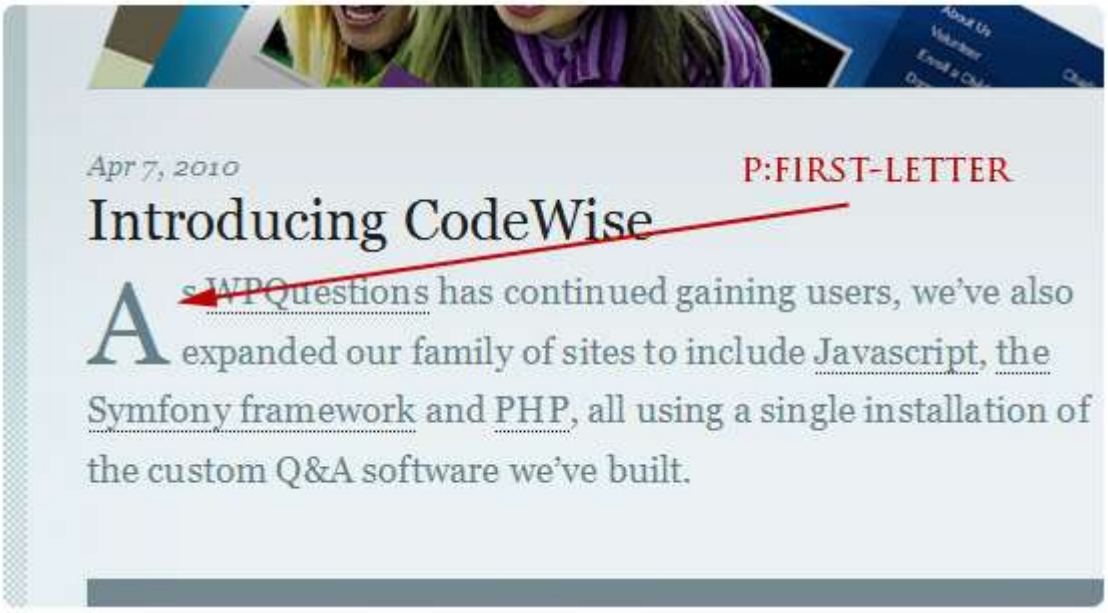
- **First, Last, and n-th Pseudo-elements**

The following pseudo-elements all relate to the position of an element relative to the HTML document and the other HTML elements in it.

: First-letter allows you to target the first letter of an element.

Here's an example:

p:first-letter { font-size: 30pt; display: block; float: left; margin: 0 5px 5px 0; }



As you can see, this pseudo-element can be helpful for creating drop caps.

The :: **first-line** CSS pseudo-element allows you to select the first line of an HTML element containing text.

Here's an example for bolding the first line of text and making its letters uppercased:

p::first-line { font-weight: bold; text-transform: uppercase; }

Embed and position project object in webpage content

When the content of one completely different webpage is embedded into another webpage, it is called a nested webpage. In simple words, a webpage that is inside another webpage of a completely different domain.

There are two methods to create a nested webpage.

- Using <iframe> tag
- Using <embed> tag

<iframe> tag:

The iframe in HTML stands for Inline Frame. The “iframe” tag defines a rectangular region within the document in which the browser can display a separate document, including scrollbars and borders. An inline frame is used to embed another document within the current HTML document.

Syntax:

```
<iframe src="URL"></iframe>
```

Example:

```
<!DOCTYPE html>
<html>
<head></head>
<body>
    <h1>
        Creating a Nested Webpage using
        'iframe' Tag: rtb.gov.rw
    </h1>

    <iframe src="https://www.rtb.gov.rw"
            height="500px" width="1000px">
    </iframe>
</body>
</html>
```

In this webpage we will also have nested homepage of rtb site.

- **<embed> tag:**

  The <embed> tag in HTML is used for embedding external applications which are generally multimedia content like audio or video into an HTML document. But other raw HTML content can be embedded using this tag. We can use this feature to create a nested webpage.

Syntax

```
<!DOCTYPE html>
<html>
<head></head>
<body>
    <h1>
        Creating a Nested Webpage using
        'embed' Tag:MINEDUC
    </h1>

    <embed src="https://www.mineduc.gov.rw/"
        type="text/html" />
</body>
</html>
```

The website can prevent you from embedding their content due to copyright issues. So, it is mandatory to have proper permission and authorization before using another webpage as a nested webpage inside your website.

Practical Activity 3.3.2: Use of Typography on web page

Task:

1. Referring to the previous theoretical activities **(3.3.1)** you are requested to go to the computer lab to perform the following task:
   Create a webpage for a magazine which must have the following characteristics:
   - Intuitive Navigation
   - Responsive Design
   - Clear Layout
   - Visual Appeal

On the magazine you have to embed other website for your choice. This task should be done individually.

2. Read key reading 3.3.2 and ask clarification where necessary

3. Apply safety precautions.

4 Develop the webpage by applying typography

5. Present your work to the trainer and whole class

Key readings 3.3.2.

Steps to design and develop the webpage by applying typography

Creating a well-designed webpage that effectively utilizes typography involves several steps, from initial planning to final development. Here's a comprehensive guide:

1. Planning and Research

Define Purpose and Audience: Understand the purpose of the webpage and identify the target audience.

Research and Inspiration: Look for inspiration from other websites and gather ideas on typography trends, font combinations, and layouts.

Content Planning: Outline the content that will be on the webpage, including headings, paragraphs, quotes, lists, and other text elements.

2. Choosing Fonts

Font Selection: Choose fonts that align with the brand identity and the message you want to convey. Consider using a combination of serif, sans-serif, and display fonts.

Web Fonts: Use web-safe fonts or integrate web fonts from sources like Google Fonts or Adobe Fonts.

Font Pairing: Select complementary font pairs for headings and body text to create visual hierarchy and readability.

3. Setting Up the Project

Create Project Folder: Set up a project folder containing an HTML file (**index.html**) and a CSS file (**styles.css**).

Link CSS File: Link the CSS file to the HTML file to apply styles.

4. HTML Structure

- To structure an HTML document effectively follow these steps:

i. Start with the DOCTYPE Declaration: Begin with <!DOCTYPE html> to define the document type as HTML5.

ii. Create the Root <html> Element: Wrap the entire content within the <html> tag and set the lang attribute to specify the language.

iii. Add the <head> Section: Inside the <head> tag, include the document's metadata, such as <title>, <meta> tags, and links to external resources like CSS and JavaScript files.

iv. Define the Document's <title>: Set the title of the page within the <title> tag, which appears in the browser tab.

v. Include Meta Tags: Add meta tags for character encoding, viewport settings for responsiveness, and other SEO-related information.

vi. Link to External Resources: Use <link> for stylesheets and <script> for JavaScript files to link external resources.

vii. Add the <body> Section: Place all visible content of the webpage within the <body> tag.

viii. Organize Content with Semantic Elements: Use HTML5 semantic tags like <header>, <nav>, <main>, <section>, <article>, <aside>, and <footer> to logically structure the content.

ix. Use Headings for Hierarchy: Apply heading tags (<h1> to <h6>) to create a clear content hierarchy.

x. Add Content Elements: Insert paragraphs, images, lists, links, and other content elements within the structured layout.

xi. Ensure Accessibility: Include attributes like alt for images and aria-* for better accessibility.

xii. Test and Validate: Check the HTML in different browsers and validate it using an HTML validator to ensure it meets web standards.

Example

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Typography Example</title>
  <link rel="stylesheet" href="styles.css">
</head>
<body>
  <! -- Content goes here -->
</body>
</html>
```

Add Content: Structure your content using semantic HTML tags (e.g., <header>, <main>, <section>, <article>, <footer>).

```
<header>
  <h1>Welcome to Typography Design</h1>
</header>
```

<main><section>

<h2>Introduction to Typography</h2>

Typography is the art and technique of arranging type to make written language legible, readable, and appealing...

</section>

<section>

<h2>Typography Principles</h2>

<blockquote> “Typography is the craft of endowing human language with a durable visual form.” – Robert Bringhurst</blockquote>

 Legibility

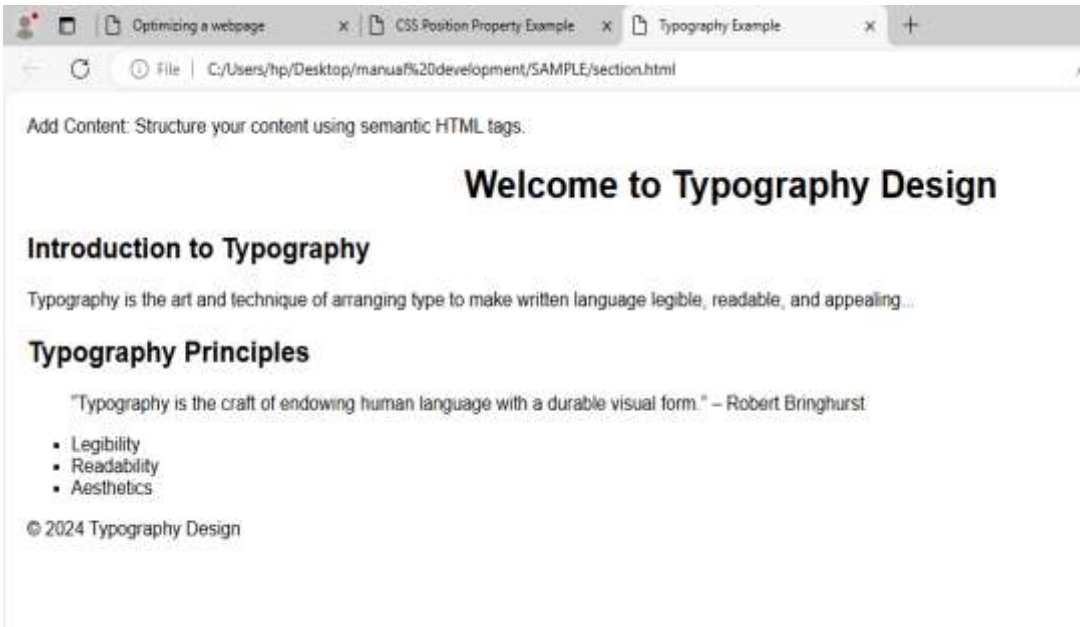
Readability

Aesthetics </section></main><footer>

© 2024 Typography Design

</footer>

Output



5. CSS Styling

- Create or Link the CSS File: Start by creating a new CSS file (e.g., styles.css) and link it to your HTML document using the <link> tag in the <head> section.

6. **Responsive Design**

- Use Media Queries to Ensure the webpage is responsive and looks good on different devices

7. **Testing and Debugging**

- **Cross-Browser Testing:** Test the webpage in different browsers to ensure consistent appearance and functionality.
- **Device Testing:** Test the webpage on various devices (desktop, tablet, mobile) to check responsiveness.
- **Validation**: Use HTML and CSS validators to check for syntax errors and best practices.

8. Final Adjustments

- **Fine-Tuning:** Make any final adjustments to the design, ensuring all text is legible and aesthetically pleasing**.**
- **Performance Optimization:** Minimize CSS and HTML files to improve load times.

9. Deployment

- **Upload to Server:** Upload the files to a web server or use a platform like GitHub Pages for deployment.
- **Share and Review:** Share the link with others for feedback and review**.**

Using typography scripts

Typography scripts refer to the various writing systems and character sets used to represent different languages and styles in web design.

Choosing the right font that supports multiple scripts is crucial for ensuring that text appears correctly across different languages. Web fonts like Google Fonts often provide options that cover a wide range of scripts.

Follow these steps to apply typography scripts

1. **Choose a Typography Script**

Select a Font: Start by choosing a font from a typography provider, such as Google Fonts, Adobe Fonts, or a custom font that you have.

Consider Readability: Make sure the font is readable and fits the design aesthetic of your webpage.

2. **Import the Font**

- **Using Google Fonts:**

- Go to Google Fonts.
- Select the font you want to use and click on "Select this style."
- In the sidebar, you'll see options to embed the font. Copy the <link> tag provided.

  Example:

  <link

  rel="stylesheet"
  href="https://fonts.googleapis.com/css2?family=Roboto:wght@400;700&display=swap">

- **Using Adobe Fonts:**

- Go to Adobe Fonts.
- Choose a font and add it to a web project.
- Adobe will provide a <link> or script tag that you can copy into your HTML**.**

  Self-Hosted Fonts:

- Download the font files and place them in your project directory.
- Use the @font-face rule in your CSS to load the font.

  Example:

```
@font-face {
    font-family: 'MyCustomFont';
    src: url('fonts/MyCustomFont.woff2') format('woff2'),
         url('fonts/MyCustomFont.woff') format('woff');
    font-weight: normal;
    font-style: normal;
}
```

3. **Link the Font in Your HTML**

- Add the <link> tag from Google Fonts or Adobe Fonts in the <head> section of your HTML file.

  <head>

  <link rel="stylesheet"

href="https://fonts.googleapis.com/css2?family=Roboto:wght@400;700&display=swap">

</head>

4. **Apply the Font Using CSS**

- Use CSS to apply the font to your webpage elements.

Example:

```
body {
   font-family: 'Roboto', sans-serif;
}

h1 {
   font-family: 'Roboto', sans-serif;
   font-weight: 700;
}
```

5. **Customize Typography Settings**

- Adjust font sizes, weights, line heights, and other typography-related properties to achieve the desired look and feel

Example:

```
p {
   font-size: 16px;
   line-height: 1.5;
}
h1 {
   font-size: 32px;
   font-weight: bold;
   line-height: 1.2;
}
```

6. **Test on Different Devices**

- Ensure that the typography looks good on different devices and screen sizes by testing your webpage on various platforms.

Points to Remember

- There are two main groups typography scripts (CSS properties) that control typography style which are: **font**, the font CSS property group dictates general font characteristics such as font-style and font-weight and text, the text CSS property group deals with the characters, spaces, words and paragraphs but also There are other CSS properties that can affect web typography outside of the fonts and text property groups like: Color.
- By following these steps (1. Choose a Typography Script,2. Import the Font, 3. Link the Font in Your HTML, 4. Apply the Font Using CSS, 5. Customize Typography, Settings and 6. Test on Different Devices), you will be able to effectively incorporate typography scripts into your webpage, enhancing its visual appeal and readability.

Application of learning 3.3.

AGASARO MAGAZINE Wishes to publish an article, they want you as web developer to Create a visually appealing magazine article layout that emphasizes the principles of typography, including font and typeface, contrast, consistency, white space, alignment, and color. Use CSS pseudo-elements: first-letter and first-line to enhance the design, your task is to design a one-page magazine article on a topic of your choice (e.g., travel, technology, fashion)

Indicative content 3.4: Setting media query rules

Duration: 15hrs

Theoretical Activity 3.4.1: Description of CSS media query rules

Tasks:

1. In small groups, you are requested to answer the following questions related to the CSS media query rules:
    a. What do you understand about media queries?
    b. Identify the device types and corresponding media types for media queries
2. Provide the answer for the asked questions and write them on papers.
3. Present the findings/answers to the whole class
4. For more clarification, read the key readings 3.4.1. In addition, ask questions where necessary.

Key readings 3.4.1

Media query in web design

- **CSS media queries** are a feature in CSS that allows you to apply styles to an element based on the characteristics of the device or screen displaying the content. This includes factors like screen width, height, resolution, orientation, and more.
- **CSS Media query** is a technique used for modifying the behaviour and appearance of a website, based on the user's device conditions such as display size, orientation, resolution, even type of device used, and other browser settings.

The way media queries work

Media queries work by using the @media rule to define styles that should be applied only when certain conditions are met. For example, you can apply specific styles when the screen width is less than 600px.

Some common uses of media query.

- Creating responsive web designs that adapt to different screen sizes and devices.
- Applying different styles for print and screen media.
- Adjusting the layout and visibility of elements based on the device orientation (portrait vs. landscape).
- Enhancing accessibility by providing alternate styles for high-contrast modes or reduced motion settings.

Common media features include:

width and height: Viewport or device dimensions.
min-width and max-width: Minimum and maximum viewport or device width.
orientation: Device orientation (portrait or landscape).
resolution: Pixel density of the display.
aspect-ratio: Ratio of the width to the height of the viewport.
prefers-color-scheme: User's preferred color scheme (light or dark mode).
prefers-reduced-motion: User's preference for reduced motion to enhance accessibility.

Identify the device types (Media Types in CSS3)

CSS3 is the latest version of CSS (Cascading Style Sheet) which is built upon its precedence versions. The media types in CSS3 specify the type of media or device the which a particular set of styles should apply. There are several media types available that describe the target devices for which the particular property works the best.

Media Types in CSS3	Description
all	Default media types, used for all devices
screen	Devices with screens, such as desktops, laptops, phones, and more
speech	For devices used in speech synthesis and screen reading
projection	Devices like projectors, and presentation screens
handheld	Handheld devices such as smartphones, tablets, and more
tv	Specific for TV screens
braille	Devices targeting Braille devices used for visual disablement
3d-glasses	Devices like 3D glasses
grid	Grid-based devices like e-book readers
print	Styles are applied when the document is printed

❖ **Description of Media Query**

- Media Query definition

CSS media queries are a feature in CSS that allows you to apply styles to an element based on the characteristics of the device or screen displaying the content.

This includes factors like screen width, height, resolution, orientation, and more.

Media queries are used for the following:

To conditionally apply styles with the CSS @media and @import at-rules.

To target specific media for the <style>, <link>, <source>, and other HTML elements with the media= or sizes=" attributes.

To test and monitor media states using the Window.matchMedia() and EventTarget.addEventListener() methods.

Media Query rules:

Media queries in CSS allow for the application of styles based on specific conditions of the viewport or device.

The following are some media query rules:

1. **Width and Height**

- **min-width:** Applies styles if the viewport's width is equal to or greater than the specified value.
- **max-width:** Applies styles if the viewport's width is equal to or less than the specified value.
- **min-height:** Applies styles if the viewport's height is equal to or greater than the specified value.
- **max-height:** Applies styles if the viewport's height is equal to or less than the specified value.

2. **Orientation**

- **Portrait:** Applies styles if the viewport is in portrait mode (height greater than width).
- **Landscape:** Applies styles if the viewport is in landscape mode (width greater than height).

3. **Resolution**

- **min-resolution:** Applies styles if the device's resolution is equal to or greater than the specified value.
- **max-resolution:** Applies styles if the device's resolution is equal to or less than the specified value

4. **Aspect Ratio**

- **min-aspect-ratio:** Applies styles if the aspect ratio is equal to or greater than the

specified value.

- **max-aspect-ratio**: Applies styles if the aspect ratio is equal to or less than the specified value.

5. **Device Width and Height**

- **min-device-width:** Applies styles if the device's width is equal to or greater than the specified value.
- **max-device-width:** Applies styles if the device's width is equal to or less than the specified value.
- **min-device-height:** Applies styles if the device's height is equal to or greater than the specified value.
- **max-device-height:** Applies styles if the device's height is equal to or less than the specified value.

6. **Color**

- **min-color:** Applies styles if the device can display at least the specified number of colors.
- **max-color:** Applies styles if the device can display no more than the specified number of colors.
- **color:** Applies styles based on the number of bits per color component of the device

7. **Reduced Motion**

- **Prefers-reduced-motion:** Applies styles if the user has requested reduced motion

8. **Scripting**

- **scripting:** Applies styles based on the scripting ability of the device (e.g., scripting: none for devices that do not support scripting).

9. **Hover and Pointer**

- **hover:** Applies styles if the primary input mechanism can hover over elements.
- **any-hover:** Applies styles if any available input mechanism can hover over elements.
- **pointer:** Applies styles based on the accuracy of the primary input mechanism (e.g., pointer: fine for a mouse).
- **any-pointer:** Applies styles based on the accuracy of any available input mechanism

Practical Activity 3.4.2: Use of CSS Media Query

Task:

1. Referring to the previous theoretical activities **(3.4.1)** you are requested to go to the computer lab to perform the following task:
    - Create responsive and adaptive webpage that provide an optimal user experience across a wide range of devices and conditions. This task should be done individually.
2. Read key reading 3.4.2 and ask clarification where necessary
3. Apply safety precautions.
4. Develop the webpage by applying media query rules and test your webpage
5. Present your work to the trainer and whole class

Key readings 3.4.2

A media query is a CSS technique used to apply styles only when certain conditions (like screen width) are met. It allows you to create responsive designs that adapt to different devices, ensuring that your website looks good on everything from mobile phones to large desktop monitors.

Application of Media Queries

Applying media queries in web documents is essential for creating responsive designs that adapt to different screen sizes and devices. Here's a step-by-step guide on how to apply media queries effectively:

Step1. Understand the Purpose of Media Queries

Media queries allow you to apply CSS rules conditionally, based on the characteristics of the device or viewport (like width, height, resolution, orientation, etc.).

Step 2. Set Up Your HTML and CSS

HTML Structure: Ensure your HTML is semantic and well-structured, as media queries will only affect the visual presentation.

Basic CSS: Start by defining the base styles for your web page, which apply to all devices by default**.**

Step 3. Determine Breakpoints

Identify Key Breakpoints: Decide on the viewport widths where your design should change. Common breakpoints include:

320px (small mobile devices)

480px (mobile devices)

768px (tablets)

1024px (small desktops/laptops)

1200px (large desktops)

Step4. Apply Media Queries in Your CSS

Syntax: Use the **@media** rule in your CSS to define styles that apply at specific breakpoints.

Example:

```css
/* Base styles for mobile-first approach */
body {
  font-size: 16px;
  padding: 10px;
}
/* Styles for screens larger than 768px (tablets and up) */
@media (min-width: 768px) {
  body {
    font-size: 18px;
    padding: 20px;
  }
}
/* Styles for screens larger than 1024px (desktops and up) */
@media (min-width: 1024px) {
  body {
    font-size: 20px;
    padding: 30px;
  }
}
```

Step 5. Customize Layouts and Styles per Breakpoint

Adjust Layouts: Use media queries to rearrange layouts, change column widths, or hide/show elements based on screen size.

Modify Typography: Scale text sizes, line heights, and other typography settings to maintain readability on different devices.

Optimize Images: Serve different image sizes or resolutions based on the device's screen size and resolution.

```
/* Change layout to two columns on larger screens */
@media (min-width: 768px) {
  .container {
    display: flex;
    flex-direction: row;
  }
  .sidebar {
    width: 30%;
  }
  .main-content {
    width: 70%;
  }
}
```

Step6. Test Responsiveness

Use Browser DevTools: Test the responsiveness of your design using the responsive design mode in browser developer tools.

Adjust as Needed: Make tweaks to your media queries and styles based on how the design looks and functions on different screen sizes.

CSS Media Queries Example

With a practical demonstration let's understand how to use CSS media queries to make a fully-responsive web application. In this example, we shall create a responsive navbar with a hamburger menu icon that appears on smaller screens.

Let us follow below steps to have an example of how media queries are applied within HTML Documents:

Step1: create a webpage using html

<html><head>

<link rel="stylesheet" href="styles.css">

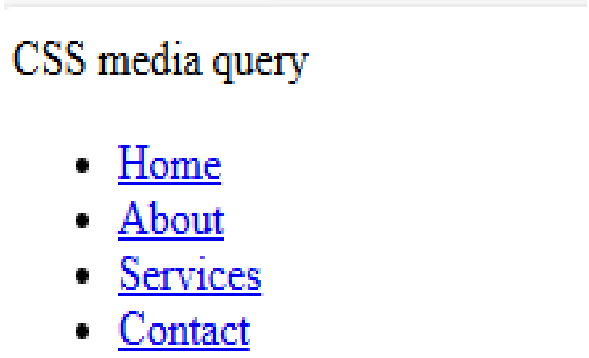
<title> Guide to CSS Media Query</title>

</head>

<body>

```
 <nav class="navbar">
  <div class="logo">CSS media query</div>
  <div class="menu-toggle">
   <span></span>
   <span></span>
   <span></span>
  </div>
  <ul class="nav-links">
   <li><a href="#">Home</a></li>
   <li><a href="#">About</a></li>
   <li><a href="#">Services</a></li>
   <li><a href="#">Contact</a></li>
  </ul>
 </nav>
</body>
</html>
```

The above html will produce this output



Step 2: Write CSS

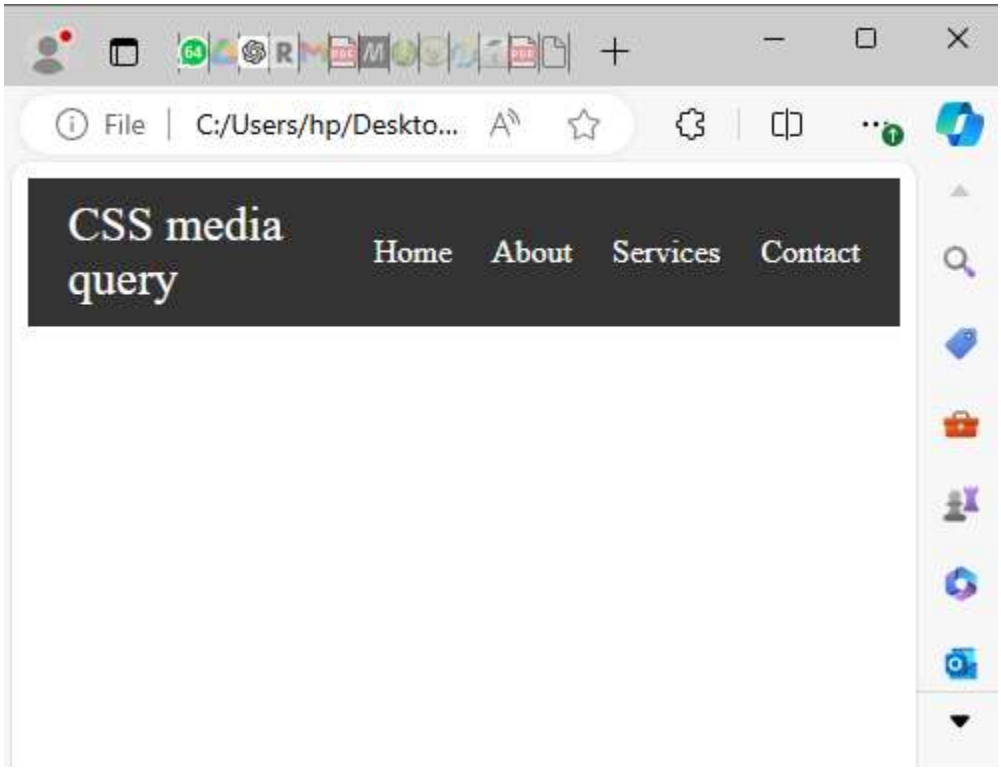
```
. navbar {
   background-color: #333;
   color: #fff;
   display: flex;
```

```
  justify-content: space-between;
  align-items: center;
  padding: 10px 20px;
 }
 .logo {
  font-size: 24px;
 }
.menu-toggle {
  display: none;
  flex-direction: column;
  cursor: pointer;
 }
. menu-toggle span {
  width: 30px;
  height: 3px;
  background-color: #fff;
  margin: 3px 0;
 }
.nav-links {
  list-style: none;
  display: flex;
  gap: 20px;
 }
.nav-links a {
  color: #fff;
  text-decoration: none;
 }
```

Step 3: Write CSS using Media Queries

```
@media screen and (max-width: 600px) {
   . menu-toggle {
     display: flex;
   }
   .nav-links {
     display: none;
     flex-direction: column;
     position: absolute;
     top: 60px;
     width: 100%;
   }
.nav-links.active {
     display: flex;
   }
 }
```

For small screen it will look like this:



Use of Breakpoint in media queries

Breakpoints in media queries are crucial for creating responsive web designs that adapt to different screen sizes and device types. Here's how breakpoints are used in media queries:

1. Adapting Layouts for Different Devices

Responsive Layouts: Breakpoints allow you to change the layout of a webpage based on the size of the user's screen. For example, a multi-column layout on a desktop might shift to a single-column layout on a mobile device.

```
/* Base layout for mobile devices */
.container {
  display: block;
}

/* Change layout for tablets and larger screens */
@media (min-width: 768px) {
  .container {
    display: flex;
  }
}
```

2. Optimizing Typography

Font Size and Line Height: Breakpoints let you adjust font sizes, line heights, and spacing to ensure readability on different screen sizes. Smaller screens might need larger fonts and more spacing to improve legibility.

```
body {
  font-size: 16px;
  line-height: 1.5;
}

@media (min-width: 768px) {
  body {
    font-size: 18px;
    line-height: 1.6;
  }
}
```

3. Adjusting Navigation Menus

Responsive Navigation: Breakpoints are used to toggle between different navigation styles, such as switching from a horizontal menu on desktops to a hamburger menu on mobile devices.

Example:

```css
/* Mobile navigation (hamburger menu) */
.nav {
  display: none;
}

.hamburger {
  display: block;
}

@media (min-width: 768px) {
  /* Desktop navigation (horizontal menu) */
  .nav {
    display: block;
  }

  .hamburger {
    display: none;
  }
}
```

4. Scaling Images and Media

Responsive Images: Breakpoints allow you to serve different image sizes or resolutions based on the device's screen size, optimizing loading times and ensuring that images look sharp on all devices.

Example:

```css
.hero-image {
  width: 100%;
  height: auto;
}

@media (min-width: 1024px) {
  .hero-image {
    width: 75%;
  }
}
```

5. Enhancing User Interface Elements

Button and Input Size: Breakpoints can be used to adjust the size of buttons, inputs, and other interactive elements to make them easier to use on smaller touch screens.

Example:

```
.button {
  padding: 10px 20px;
}

@media (min-width: 768px) {
  .button {
    padding: 15px 30px;
  }
}
```

6. Modifying or Hiding Content

Content Visibility: Use breakpoints to show or hide certain elements based on the screen size. For example, you might hide a large image or secondary content on smaller screens to prioritize primary content.

Example:

```
.sidebar {
  display: none;
}

@media (min-width: 1024px) {
  .sidebar {
    display: block;
  }
}
```

7. Controlling Grid and Flexbox Layouts

Reflowing Content: Breakpoints allow you to change the direction or order of grid and flexbox layouts based on screen size, ensuring content flows naturally across different devices.

Example

```css
.grid-container {
  display: grid;
  grid-template-columns: 1fr;
}

@media (min-width: 768px) {
  .grid-container {
    grid-template-columns: 1fr 1fr;
  }
}

@media (min-width: 1200px) {
  .grid-container {
    grid-template-columns: 1fr 1fr 1fr;
  }
}
```

8. Supporting Multiple Device Orientations

Portrait vs. Landscape: Breakpoints can also be used to apply different styles depending on whether the device is in portrait or landscape mode.

```css
@media (orientation: portrait) {
  .sidebar {
    display: none;
  }
}

@media (orientation: landscape) {
  .sidebar {
    display: block;
  }
}
```

Breakpoints in media queries are essential for creating a responsive design that adapts to different devices and screen sizes. By strategically applying breakpoints, you can ensure that your website is accessible, user-friendly, and visually appealing across a wide range of devices, from small mobile phones to large desktop monitors.

Points to Remember

- Breakpoints are used to create responsive designs that adapt to different devices, such as desktops, tablets, and mobile phones.
- **common uses of media queries**
    - Creating responsive web designs that adapt to different screen sizes and devices.
    - Applying different styles for print and screen media.
    - Adjusting the layout and visibility of elements based on the device orientation (portrait vs. landscape).
    - Enhancing accessibility by providing alternate styles for high-contrast modes or reduced motion settings.

Application of learning 3.4.

YNV Store is company which sells electronic devices through an online market, they have a website that use to publish those products but they have problems about the display of visual text and appealing contents across different devices, they want you to apply media queries rules to enhance the functionality of their website.

Learning outcome 3 end assessment

Theoretical assessment

Question 1. What is the primary purpose of CSS in web development?

a) To structure the content of a web page

b) To add interactive functionality to a web page

c) To separate the content of a web page from its presentation

d) To store data in a web page

Question 2. Which of the following is NOT a type of CSS?

a) Inline CSS

b) Internal CSS

c) External CSS

d) Exported CSS

Question 3. Which of the following methods is used to import an external CSS file within a CSS file?

a) @link

b) @include

c) @import

d) @stylesheet

Question 4. Which method is used to apply CSS to an HTML document by linking to an external stylesheet file?

a) Inline CSS

b) Internal CSS

c) External CSS

d) Embedded CSS

Question 5. Which of the following is NOT a valid value for the display property?

a) inline

b) block

c) flex

d) inheritance

Question 6. Which of the following concepts includes margins, borders, padding, and the actual content in its definition?

a) CSS Flexbox

b) CSS Grid Layout

c) CSS Box Model

d) CSS Positioning

Question 7. Complete the following sentence with appropriate terms

a) The art and technique of arranging type to make written language legible, readable, and appealing when displayed is known as _____

b) There are two main groups of CSS properties that control typography style: ---------- and ----------

c)Use ---------- link pseudo-classes to create font styles for each link state.

d)The ----------tag defines a rectangular region within the document in which the browser can display a separate document, including scrollbars and borders.

Question 8. Match the elements in column A with its corresponding meaning in column B

Answers	**Column A** (media query rules)	**Column B** (Explanation)
1……	7. min-width	A. Applies styles if the user has requested reduced motion.
2……	8. landscape	B. Applies styles based on the number of bits per color component of the device
3……	9. color	C. Applies styles if the viewport is in l mode where width is greater than height
4. ……	4. Reduced Motion	D. Applies styles if the viewport's width is equal to or greater than the specified value.

Practical assessment

FashionHub is an online store that sells a wide range of clothing items, including dresses, shirts, and accessories. They want to hire a Front-End developer to develop a website that includes media queries for responsiveness across different devices. That website should provide an optimal user experience on various devices, including desktop computers, tablets, and smartphones. The website should adapt its layout and content presentation based on the user's device.

Apply your knowledge of typography, media queries, and web optimization to create a fully functional, responsive webpage that demonstrates your ability to style web elements, create a web structure, and optimize the page for different devices and screen sizes so that it can satisfy the requirements of FashionHub.

References

A complete guide to CSS Media Query | Browser Stack

CSS Syntax (2024 Tutorial & Examples) | BrainStation®

CSS Syntax (w3schools.com)

Including CSS: External Styles | Basic Introduction (codefinity.com)

https://blog.hubspot.com/website/website-typography

CSS Position Relative vs Position Absolute | Kolosek

CSS Display Property (With Examples) (programiz.com)

A complete guide to CSS Media Query | BrowserStack

A complete guide to CSS Media Query | BrowserStack

Using media queries - CSS: Cascading Style Sheets | MDN (mozilla.org)

October 2024